

The Battery SOC Benchmark, explained end to end

Every part of the system on one page — what it does, how each piece works, why it was built that way, and where to look in the code. Written for developers and non-developers alike; each part opens with a plain-English summary.

CONTENTS

1. ⚡ The five-minute version
2. 🧰 The tools and libraries, and why each one
3. 📦 Life of a submission, end to end
4. ▶ The evaluation engine — running the model
5. 📊 The evaluation engine — scoring, complexity and outputs
6. 📄 The data model
7. 👤 Authentication and accounts
8. 💻 The web tier: submitting, results, leaderboard
9. 🏠 Administration, notifications, reports, monitoring
10. 🏗 Infrastructure and operations
11. 🛡 Security: threats and what stops them
12. ❓ Questions you will probably get
13. 🛠 How to change things — recipes
14. 🎬 A demo order that tells the story
15. 📖 Glossary

This is the whole system explained in one place: what it does, how each part works, why it was built the way it was, and where to look in the code when you want to check something.

You don't need to be a developer to follow it. Every part starts with a short plain-English summary, and the diagrams are drawn to make sense on their own, so you can read the summaries and the pictures and skip anything in `monospace` without losing the thread. If you are a developer and the codebase is new to you, read it in order once; each part ends with a list of the files to open, and the glossary at the end covers any term you haven't met. If you already know this stack, the parts most worth your time are the submission journey, the scoring, authentication and the recipes, because those hold the decisions you couldn't guess from the code — and the questions section near the end is the list people actually ask.

Every claim points at a file so it can be checked. The diagrams are Mermaid and render on GitHub.

Colour key used in every diagram

 Website
(Vercel)

 Worker
(lab VM)

 Sandbox
(Docker)

 Data
(database, bucket)

 External service
(MathWorks, GitHub, SMTP)

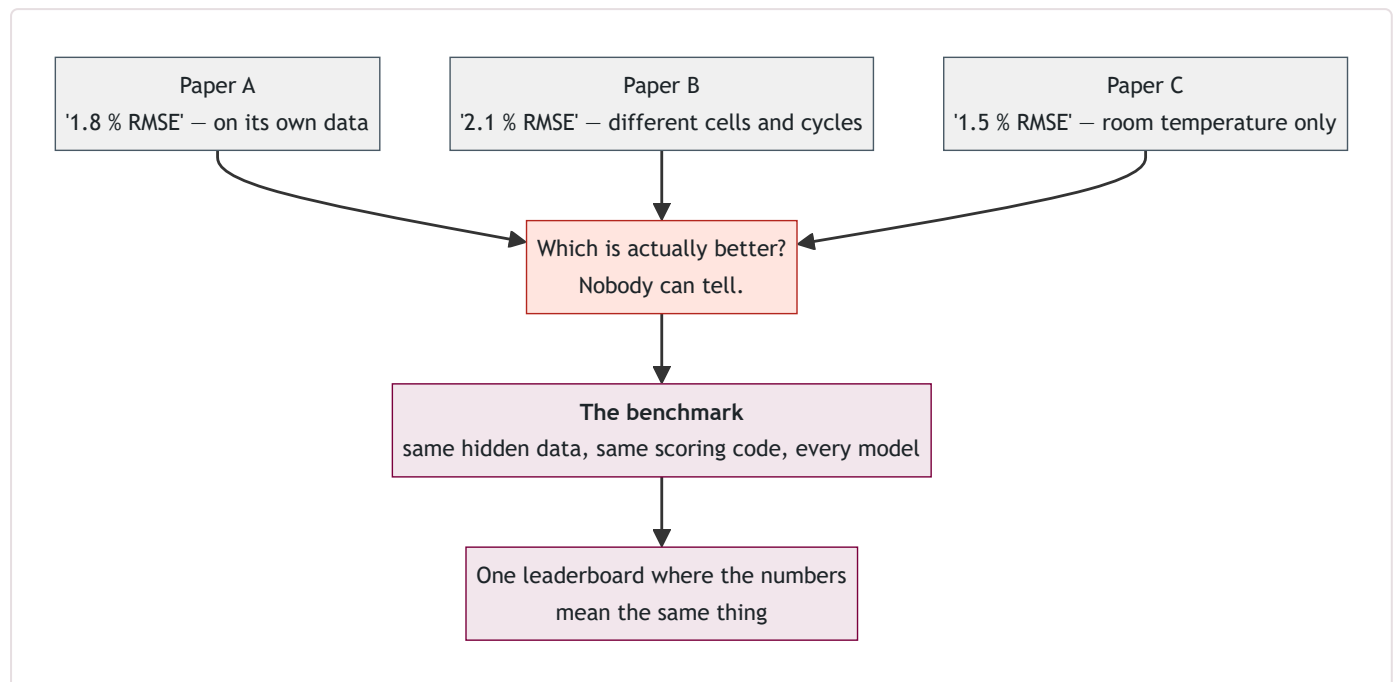
 A person

 Secret ·  danger

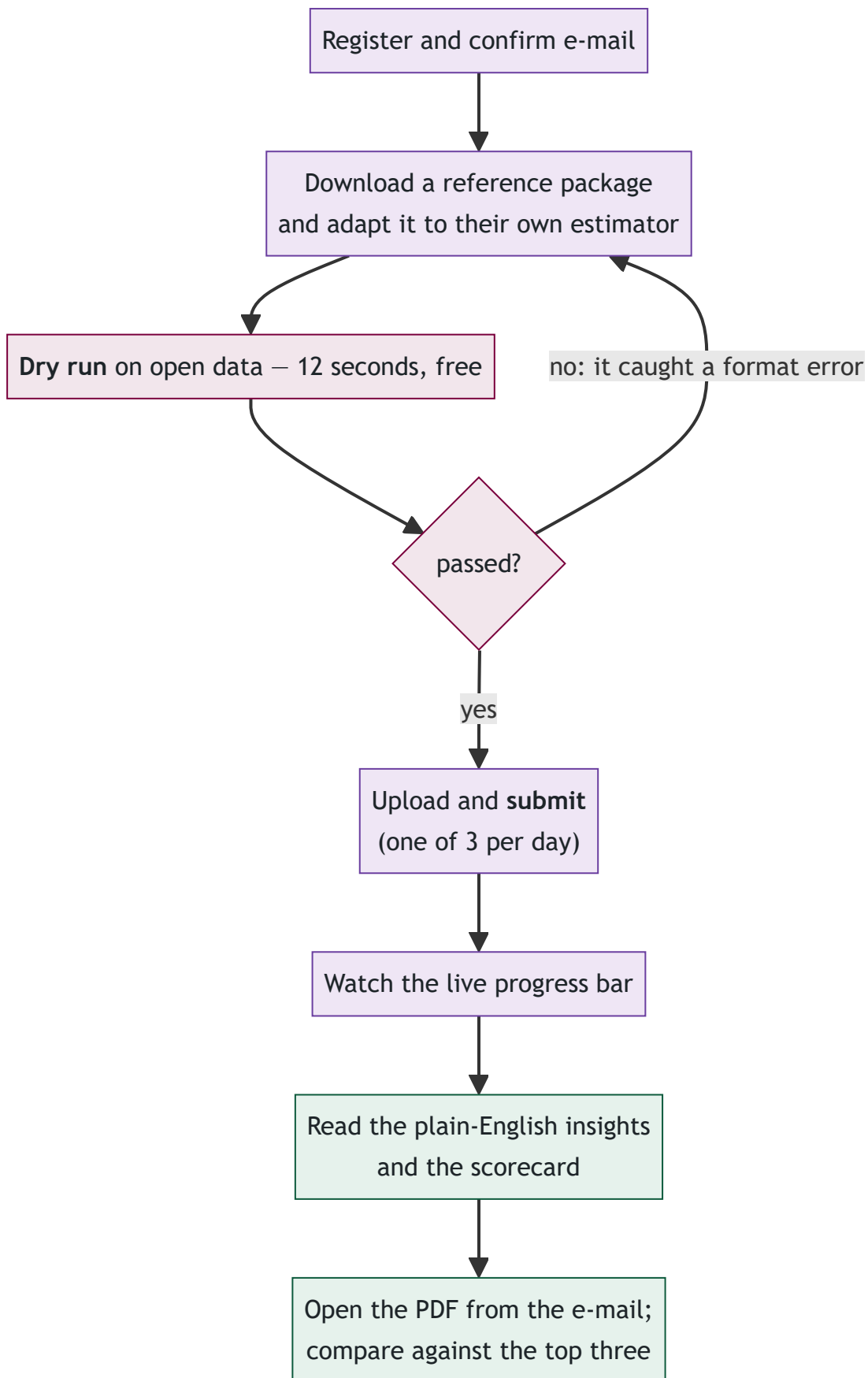
⚡ The five-minute version

💡 **Plain English.** Researchers upload a small program that guesses how full a battery is. We run that program against battery data that has never been published, score it with a fixed public formula, and put the score on a public leaderboard. Everyone is scored on the same hidden data with the same code, so for the first time the numbers are comparable. The uploaded program is deleted the moment it has been scored, and it never touches the website.

🎯 The problem it solves

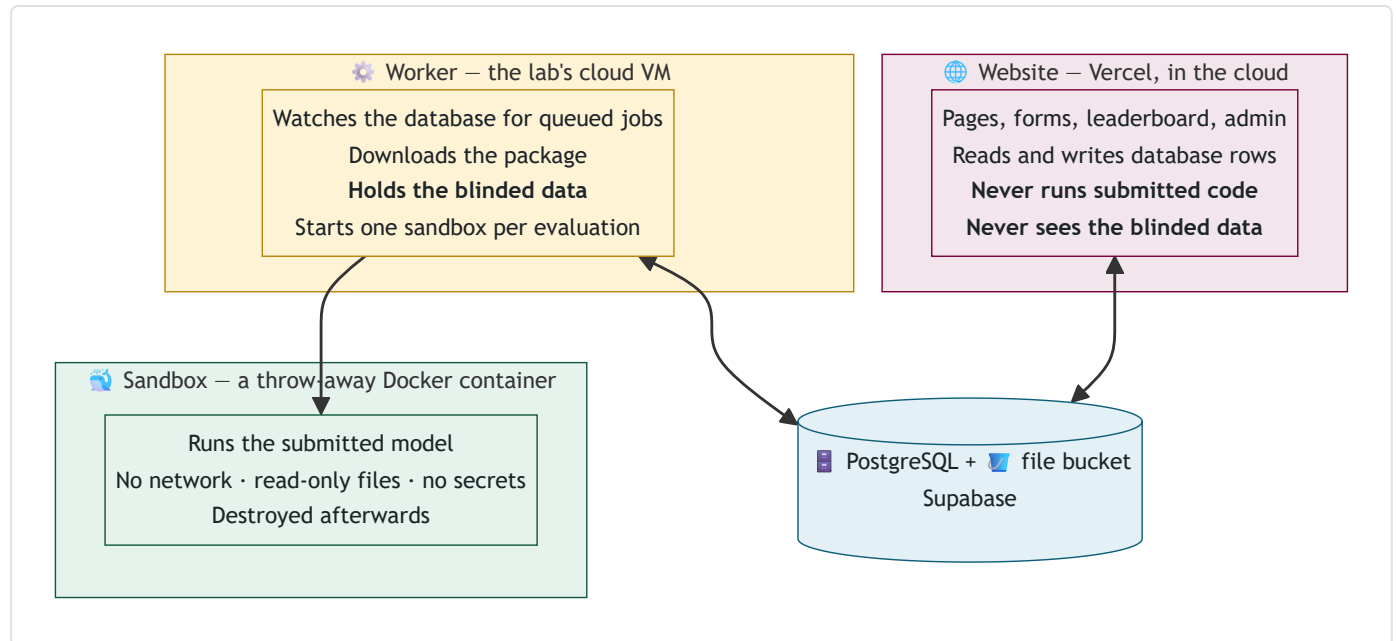


A researcher's experience, start to finish



🧱 The three tiers, and what each may touch

Everything in the system is one of three kinds of program. The most important fact about the design is *what each is allowed to reach*.



The website and the worker share **nothing but the database**. They never talk to each other. That one fact explains a great deal of what follows: why adding a second worker machine needs zero configuration (it simply starts claiming jobs), why the website stayed up during a ten-hour network outage that cut the worker off, and why the queue is a database table rather than a queue service.

⚠️ What could go wrong, and what stops it

WORRY	WHAT STOPS IT
A malicious model steals the blinded data	It can read it (it must, to be scored) but cannot send it anywhere: no network, and the container is destroyed afterwards
A malicious model attacks the website or database	It cannot reach them — they are on different machines, and the sandbox has no network
A model runs forever	Hard timeout (360 min), enforced both inside and outside the container
A researcher's code is kept and misused	The package is deleted the moment it is scored; only scores and traces remain

WORRY	WHAT STOPS IT
Someone floods the queue	3 submissions per person per day; uploads and dry runs rate-limited
Two workers evaluate the same model	Atomic claim (Part 3)
A worker crashes mid-run	Lock goes stale after 30 min; another worker picks the job up; two attempts per job
The worker loses its network	It retries every 2 s and resumes by itself; admins are e-mailed after 3 minutes of silence
Someone guesses passwords	bcrypt hashing plus 10 attempts per 15 min per account

📌 Where each thing physically is

WHAT	WHERE	WHY THERE
▲ Website	Vercel, free tier	Zero-ops hosting for Next.js; scales itself
⚡ Database + file bucket	Supabase, free tier	PostgreSQL and object storage in one account
Worker	Arbutus VM, Ubuntu 24.04, 8 vCPU / 12 GB	The lab controls it; MATLAB can be licensed there; the only place the blinded data exists
MATLAB	Inside a Docker image on that VM	So MATLAB models run in the same kind of sandbox as Python ones
🐙 Source code	GitHub, McMaster-Battery-Research-Group organization	Read-only collaborators can read and propose; only the owner merges
E-mail	Gmail SMTP today; Resend once a domain exists	Configuration only — the code is provider-agnostic

📌 Files to open, in order: [README.md](#) → [prisma/schema.prisma](#) → [src/evaluator/worker.ts](#).

PART 2



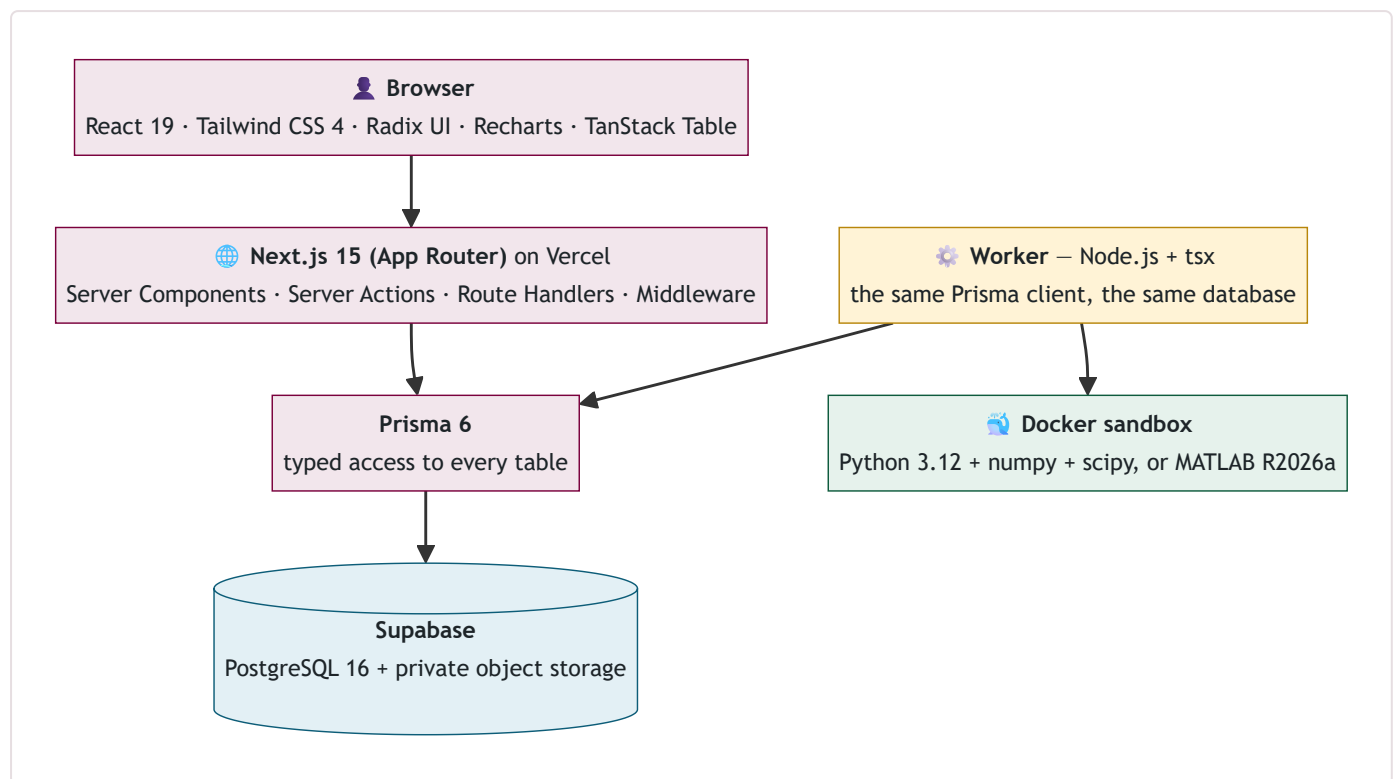
The tools and libraries, and why each one



Plain English. Software is assembled from existing building blocks. This part names each block, says what it is in one sentence, why we chose it over the alternatives, and — for non-developers — what it is analogous to. The short version: everything runs on free tiers, the website and the worker are written in the same language, and MATLAB is used for exactly one job.

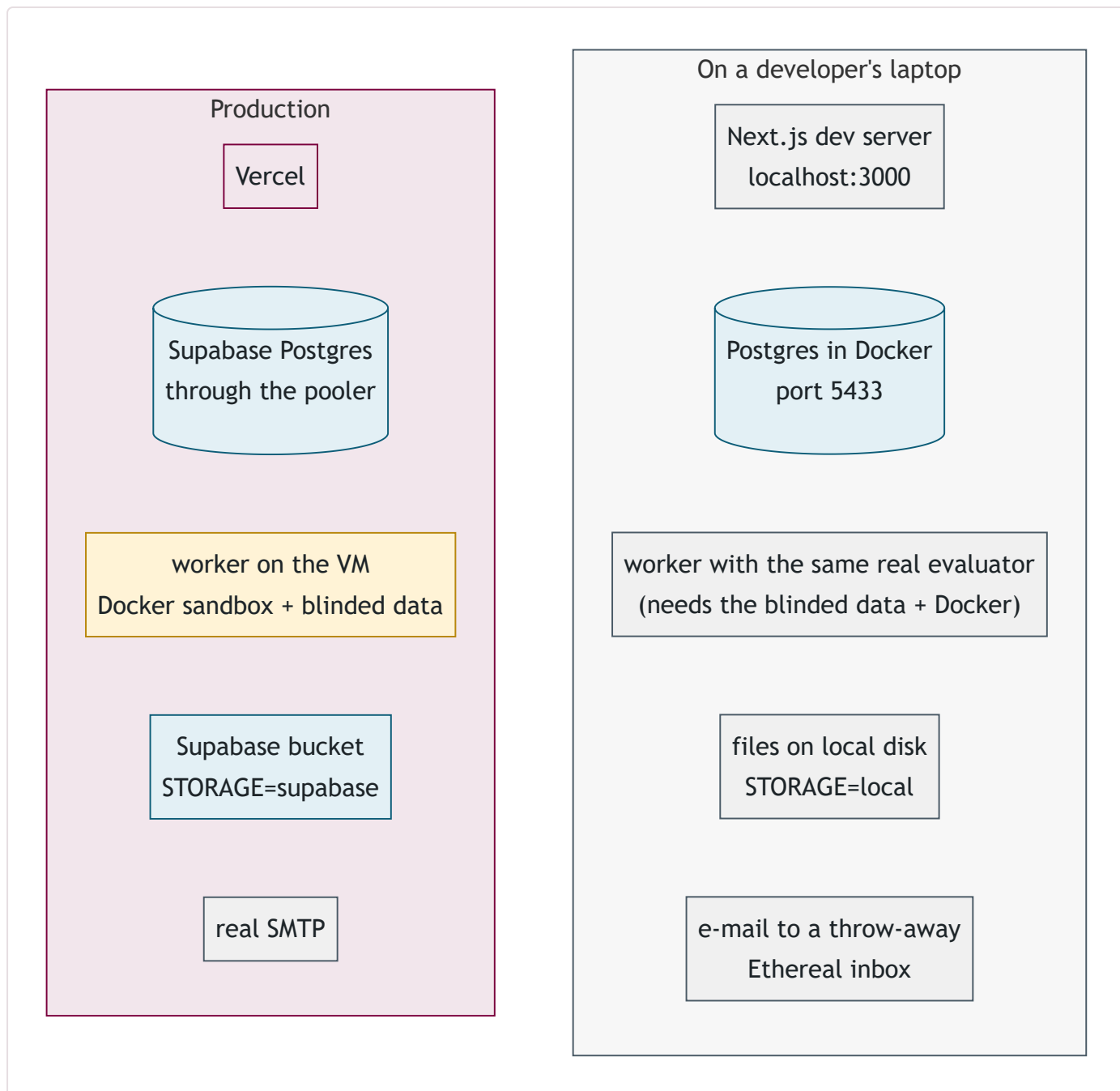


The stack as layers



Development versus production

The same code runs in two very different environments. Knowing which is which explains most "works on my machine" confusion.



Switching between them is entirely `.env` configuration — `STORAGE` , `DATABASE_URL` , `SMTP_HOST` , `SOCBENCH_BLIND_DATA` — never a code change. There is no fake scorer: a developer's worker runs the same evaluator as production, so local results are real results.

Web application

TOOL	WHAT IT IS	WHY WE USE IT	ANALOGY
 TypeScript 5	JavaScript with a type system that catches mistakes before the code runs	One language for the website <i>and</i> the worker; refactoring is safe	Spell-check for code

TOOL	WHAT IT IS	WHY WE USE IT	ANALOGY
 Node.js 22	Runs JavaScript/TypeScript outside a browser	The server side and the worker	The engine
 Next.js 15 (App Router)	The web framework: pages, forms and small APIs in one project, rendered on the server	Free hosting on Vercel; server rendering keeps database access off the browser; one project instead of "front end + API"	The chassis everything bolts onto
 React 19	The library for building interfaces from components	Next.js is built on it	Lego bricks for screens
Server Components	React components that run <i>only on the server</i> and can read the database directly	No API layer to build or secure for reading	A page that fills itself in before it is sent
Server Actions	Functions that run on the server but are called like ordinary functions from a form	No API layer for writing either; validation and auth sit beside the logic	A form that knows where to go
Route Handlers	Plain URL endpoints (<code>/api/...</code>) for the few things that need one	Status polling, PDF download, upload URLs, the health check	Doors for other programs
Middleware	Code that runs before a page, at the network edge	Redirects signed-out users away from protected pages	A bouncer at the door
 Tailwind CSS 4	Styling as small utility classes in the markup	Fast to build; McMaster maroon and gold defined once as tokens	Paint by numbers
 Radix UI	Unstyled, accessible dialogs, menus, tooltips, switches	Keyboard and screen-reader behaviour done right, our look on top	Door hinges that just work
Recharts 3	Charts as SVG	Vector charts, PNG export, a custom MATLAB-style zoom for SOC traces	The graph paper

TOOL	WHAT IT IS	WHY WE USE IT	ANALOGY
TanStack Table 8	A "headless" table engine: sorting, filtering, column visibility, no visuals	The leaderboard's column picker and filters	The spreadsheet brain without the spreadsheet
 zod 4	Declares what an input must look like and validates it	One schema per form, run in the browser <i>and</i> the server action	The form's rulebook
Auth.js v5	Login sessions for Next.js	E-mail + password with verification, no third-party identity provider	The ID card office
bcryptjs	Password hashing	Slow on purpose, so guessing passwords is expensive	A one-way scrambler
nodemailer	Sends e-mail over SMTP	Provider-agnostic — swapping Gmail for Resend is configuration	The post room
pdfkit	Draws PDFs programmatically, including vector charts	The report attached to results e-mails, no browser needed	A plotter
adm-zip	Reads zip files	Validating the package before anything runs	The parcel inspector
Playwright	Drives a real browser to test pages	Catches the class of bug only a real browser shows	A robot tester

Data

TOOL	WHAT IT IS	WHY WE USE IT
 PostgreSQL 16	The relational database	Robust, free on Supabase, and the data (users → submissions → results) is naturally relational
 Prisma 6	The ORM: <code>prisma/schema.prisma</code> describes every table; the generated client gives typed queries	Website and worker share one schema and one client; changing a column is one edit

TOOL	WHAT IT IS	WHY WE USE IT
 Supabase	Hosted PostgreSQL + object storage + a connection pooler, one free account	One dashboard; the bucket and the database live together
Supavisor (the pooler)	Sits in front of PostgreSQL and funnels thousands of short-lived connections through a few real ones	Serverless functions each open their own connection; Postgres would run out in seconds

Evaluation

TOOL	WHAT IT IS	WHY WE USE IT
 Python 3.12 + numpy + scipy	The numerical language and its array/science libraries	The benchmark maths is numpy: exact, fast, no licence, runs anywhere. scipy reads and writes <code>.mat</code> files
MATLAB R2026a	The lab's native language; many submissions are <code>.m</code> / <code>.p</code>	Used <i>only</i> to execute MATLAB models — 40 lines of MATLAB, nothing else
 Docker	Runs each evaluation in an isolated container	Submitted code is untrusted; isolation is non-negotiable
<code>mathworks/matlab-deep-learning:r2026a</code>	MathWorks' official MATLAB container with the deep-learning toolboxes	No MATLAB install on the VM; every toolbox submissions have needed is inside (24.5 GB)
MathWorks online licensing	Licensing MATLAB through a MathWorks account instead of a licence server	The only way to license MATLAB on a headless cloud machine without a campus licence server

Operations

TOOL	WHAT IT IS	WHY WE USE IT
 Vercel	Hosting for Next.js	Free tier, automatic deploys from GitHub, serverless scaling
 Arbutus / OpenStack	The Alliance's research cloud	Free allocation for the lab; a persistent VM we control

TOOL	WHAT IT IS	WHY WE USE IT
systemd	Linux service manager	Runs the worker on boot, restarts it on crash, applies sandbox hardening
 GitHub Actions	CI runner	One job: ping the health endpoint every 10 minutes
 ufw	Linux firewall	Default-deny inbound; SSH only
 tsx	Runs TypeScript directly	The worker starts as <code>node --import tsx src/evaluator/worker.ts</code> , no build step
 dotenv	Loads <code>.env</code> files into environment variables	Local development configuration

✦ **Files to open, in order:** [package.json](#) → [.env.example](#) → [next.config.ts](#).



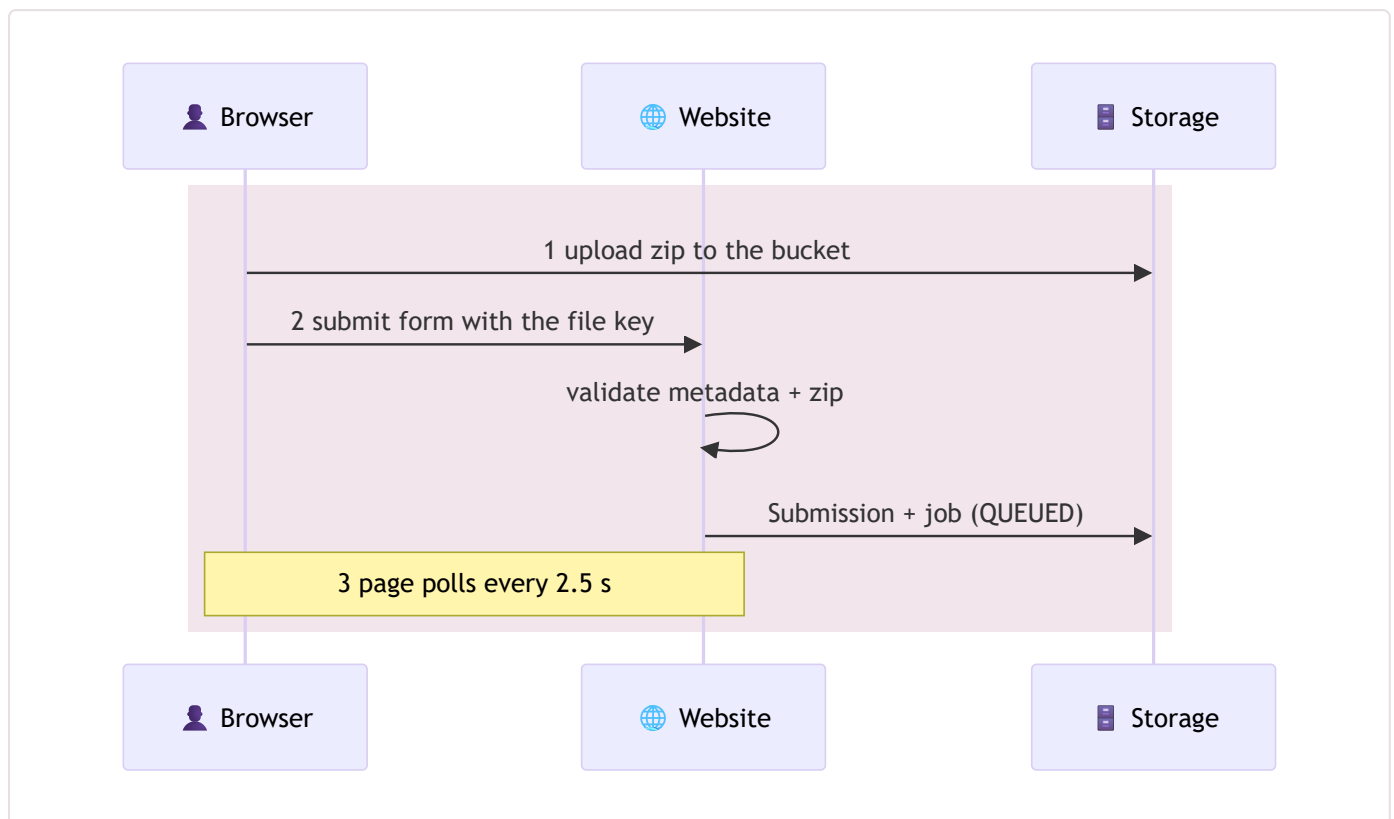
Life of a submission, end to end

Plain English. A researcher uploads a zip. The website checks that it is well-formed and puts a ticket in a queue. A worker machine takes the ticket, downloads the zip, runs the model inside a sealed box against the secret data, saves the scores, deletes the zip, and e-mails a PDF. If anything goes wrong the researcher is told exactly what. Follow this part once with the files open and you can answer almost any question about the system.

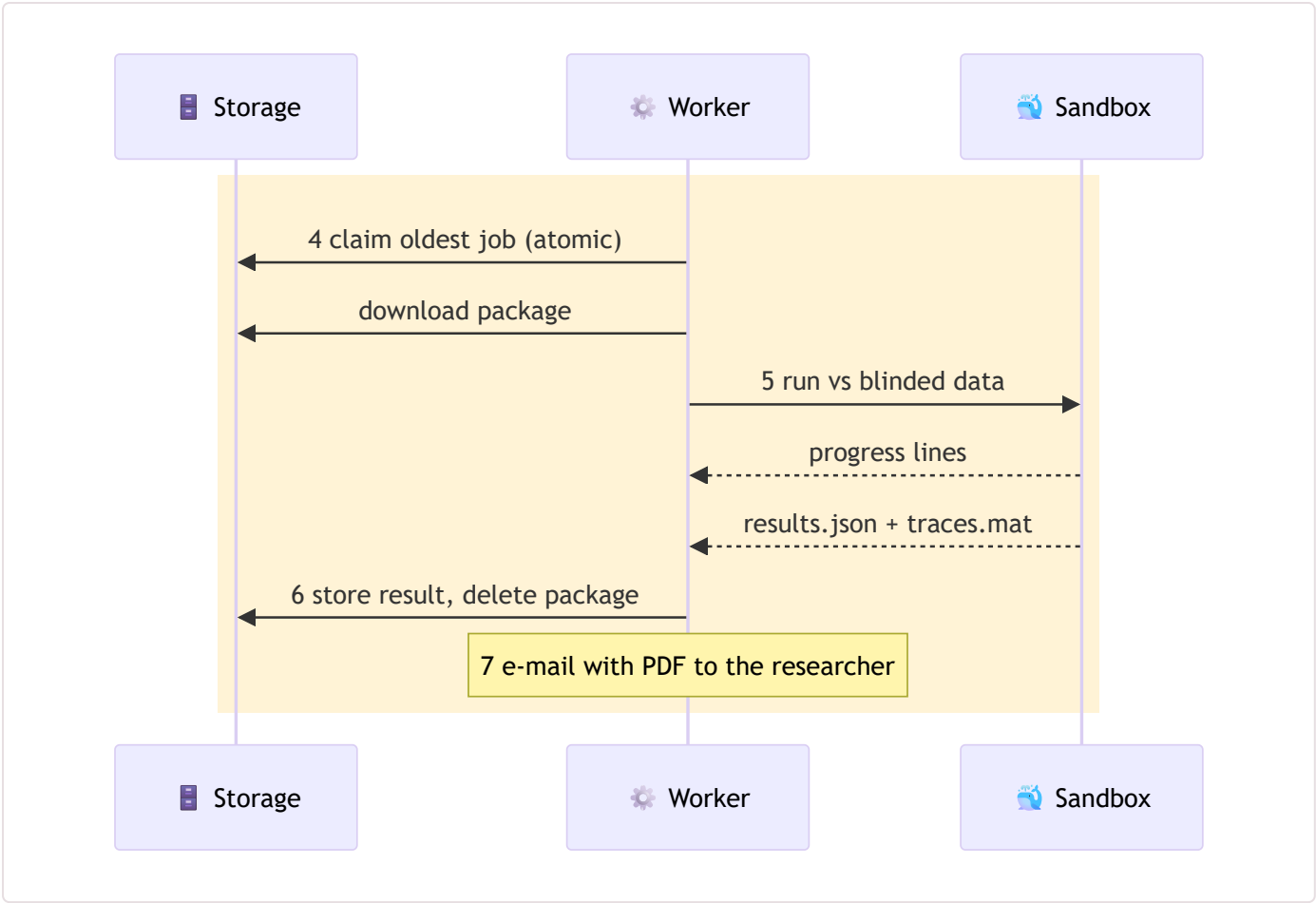


The whole journey in one picture

First half — on the website:



Second half — on the worker:

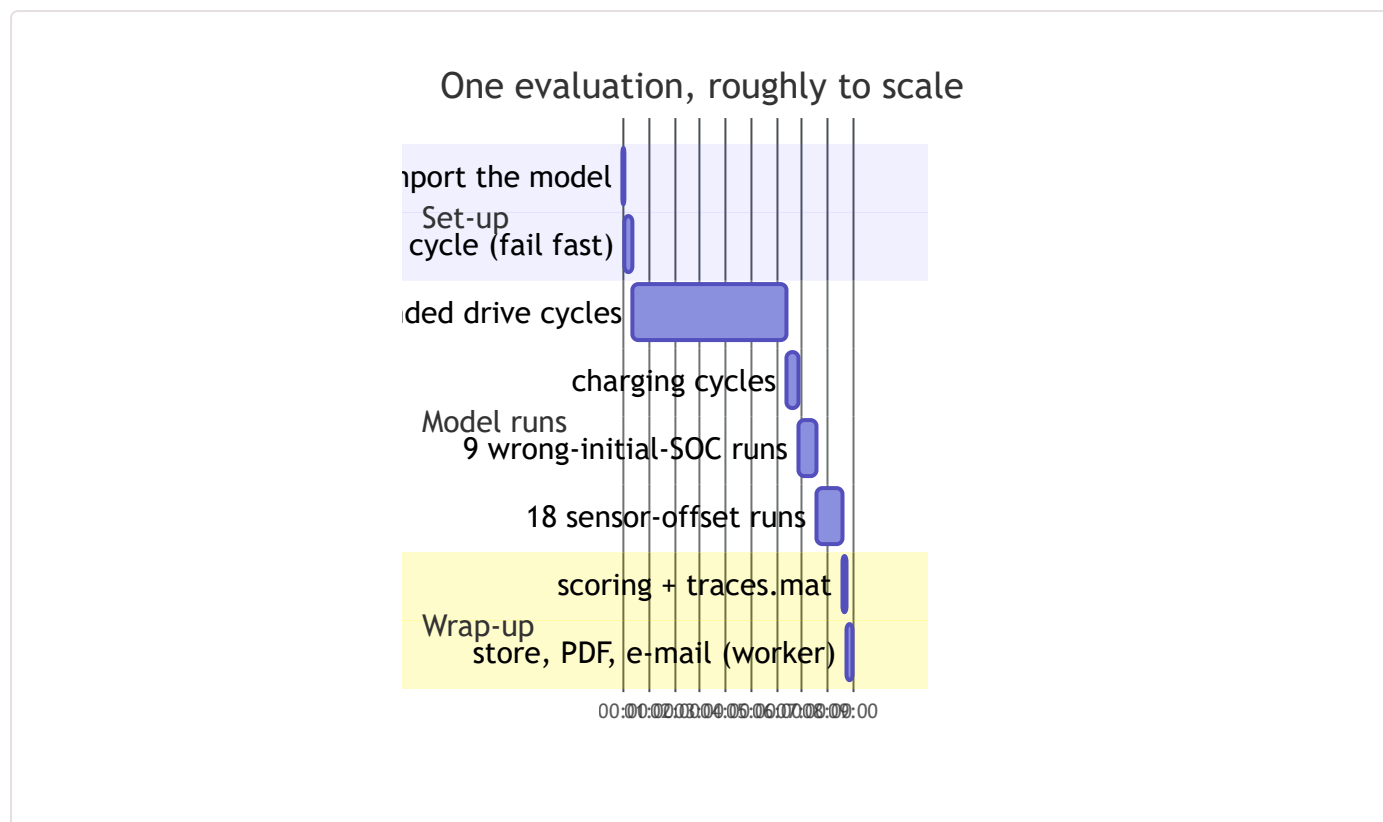


 What the researcher sees at each step

STEP	ON SCREEN	BEHIND IT
Upload	A progress bar to 100 %, then "checking the package structure"	Direct PUT to the bucket; then the server re-downloads and validates
Queued	"Position 2 in the queue · about 25 min · evaluator online"	Heartbeat table + job table + the ETA simulation
Running	A live console, a percentage, a stage name, an ETA	Every line the sandbox prints, streamed into the job log
Done	Confetti, the plain-English insights, the scorecard, an e-mail with a PDF	One database transaction, then package deletion, then mail
Failed	The exact error, with the model's own traceback lines	<code>EvaluationError</code> marked user-facing — never retried

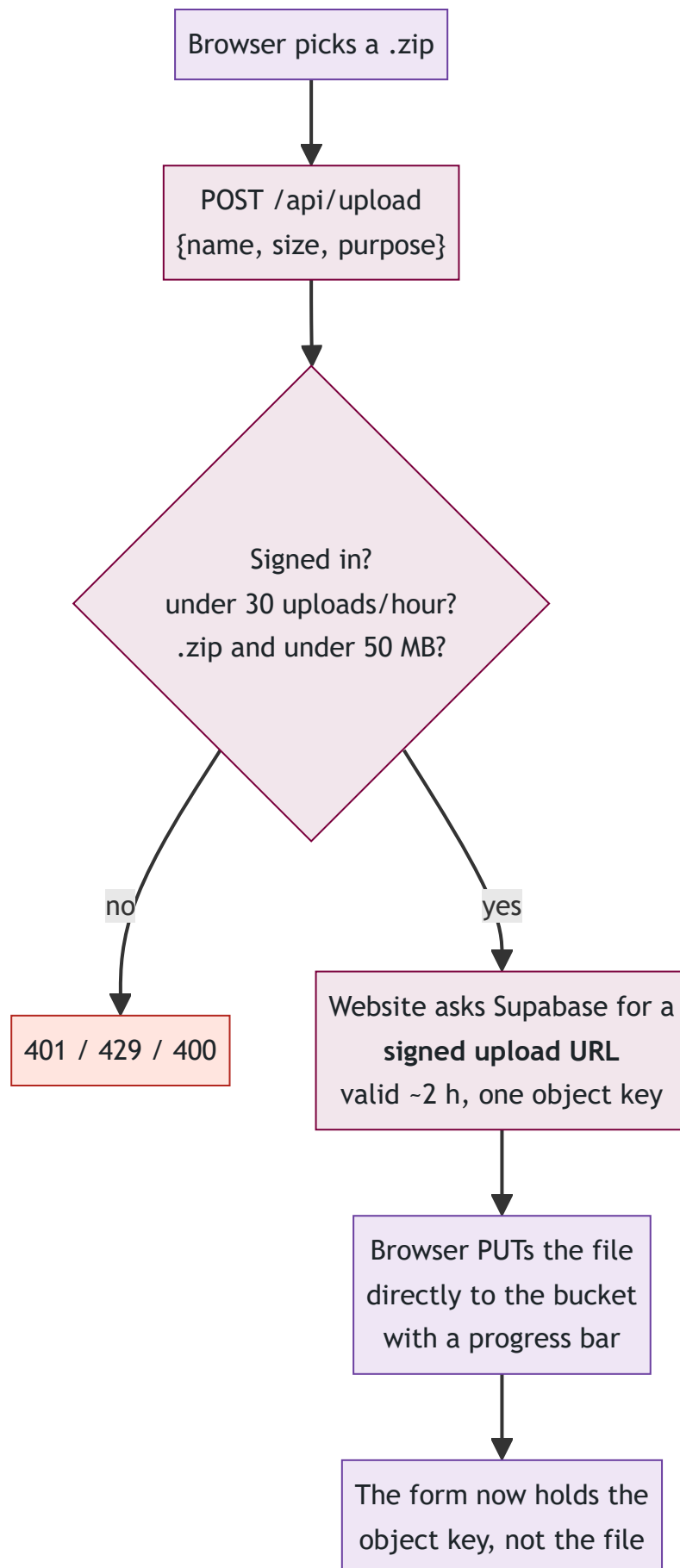
🕒 An evaluation on a clock

Rough proportions for a mid-weight model (a heavy LSTM in MATLAB can take 45 minutes; a Coulomb counter in Python takes 10 seconds):



📁 Step 1 — Upload: browser → bucket, bypassing the website

Vercel caps the body of a serverless request at 4.5 MB; packages can be 50 MB. So the file never passes through our server on the way in.



Files: [api/upload/route.ts](#), `uploadPackage()` in [upload-client.ts](#), `createSignedUpload()` in [storage.ts](#). Keys look like `submissions/1724600000000-ab12cd.zip`; the website accepts only keys matching `OBJECT_KEY_RE`, so a tampered form cannot point at some other object. In local development (`STORAGE=local`) the file simply travels inside the form.

✅ Step 2 — Validate and create

Everything below happens inside one server action, `createSubmissionAction` in [submit/actions.ts](#), in this order. Cheap checks come first, and the pre-uploaded object is deleted on any rejection so the bucket never fills with rejects.



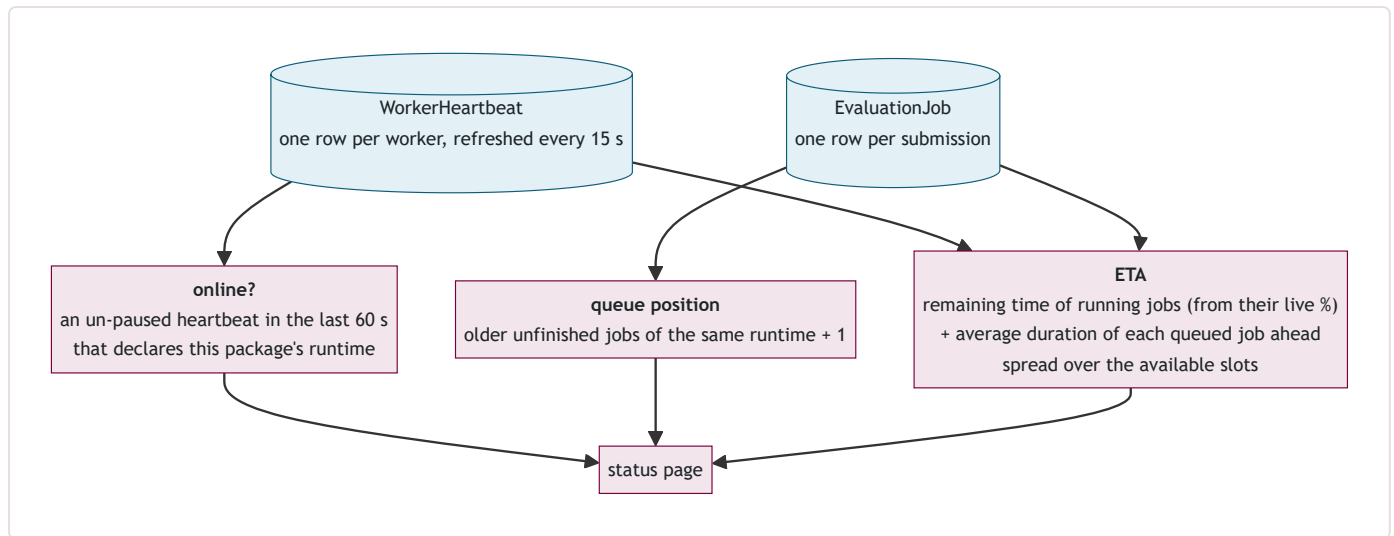
Zip hygiene (package-check.ts) is mirrored line for line by `safe_extract` in the Python evaluator, so a package that passes the website cannot surprise the worker:

CHECK	LIMIT	WHY
Entry count	≤ 500	Bounded work
Uncompressed total	≤ 512 MB	Disk
Single entry	≤ 256 MB	Disk
Compression ratio	$\leq 200 : 1$	Zip bombs are $\sim 1000 : 1$
Paths	no <code>/...</code> , no <code>C:</code> , no <code>..</code>	Path traversal
Symlinks	none (checked in the attribute bits)	Escaping the extraction folder
Sub-folders	none — zip the <i>files</i> , not the folder	The single most common user mistake
Model file	exactly one of <code>Model.m</code> / <code>Model.p</code> / <code>Model.py</code> at top level, with a regex check on the function signature	The contract

The `runtime` column (`python` or `matlab`) is set here, from which model file was found, so the right kind of worker claims the job later.

Step 3 — Waiting: where the numbers on the status page come from

The page polls `/api/submissions/[id]/status` — first after 1 s, then every 2.5 s. Everything it shows derives from two tables:



The average duration comes from the last 10 completed runs of the same model type (`averageRunSec` in [worker-status.ts](#)), falling back to 45 minutes. If no worker can run this runtime, the page says so plainly — "no evaluator for MATLAB packages is online" — rather than showing an ETA that will never arrive.

Worked example: two slots, one job running with 12 minutes left, two queued jobs ahead of yours averaging 20 minutes each. Slot A: 12 min → then queued job 1 (ends at 32). Slot B: queued job 2 (ends at 20). Your job starts on the first free slot: **about 20 minutes**.

🔒 Step 4 — Claim: how two workers never take the same job

The worker ([worker.ts](#)) polls every 2 s. Dry runs are claimed first — they are short and someone is watching. Then `claimJob()` in [run-job.ts](#):

SELECT the oldest EvaluationJob where
attempts under 2
AND (lockedAt is null OR lockedAt older than 30 min)
AND submission is QUEUED or RUNNING
AND runtime is one this worker declares

found one?

no

sleep 2 s, try again

yes

UPDATE that row
SET lockedAt = now, lockedBy = me, attempts + 1
WHERE id = X AND lockedAt = the value I just read

rows updated?

1

It is mine. Run it.

0

Another worker changed lockedAt
a millisecond before me. Try again.

In code, the whole trick is one `WHERE` clause:

```
// run-job.ts - claimJob()
const candidate = await db.evaluationJob.findFirst({ where: { /* the SELECT above */
const { count } = await db.evaluationJob.updateMany({
  where: { id: candidate.id, lockedAt: candidate.lockedAt }, // only if nobody to
  data: { lockedAt: new Date(), lockedBy: workerId(), attempts: { increment: 1 } }
});
return count === 1 ? candidate : null; // 0 means I lost t
```

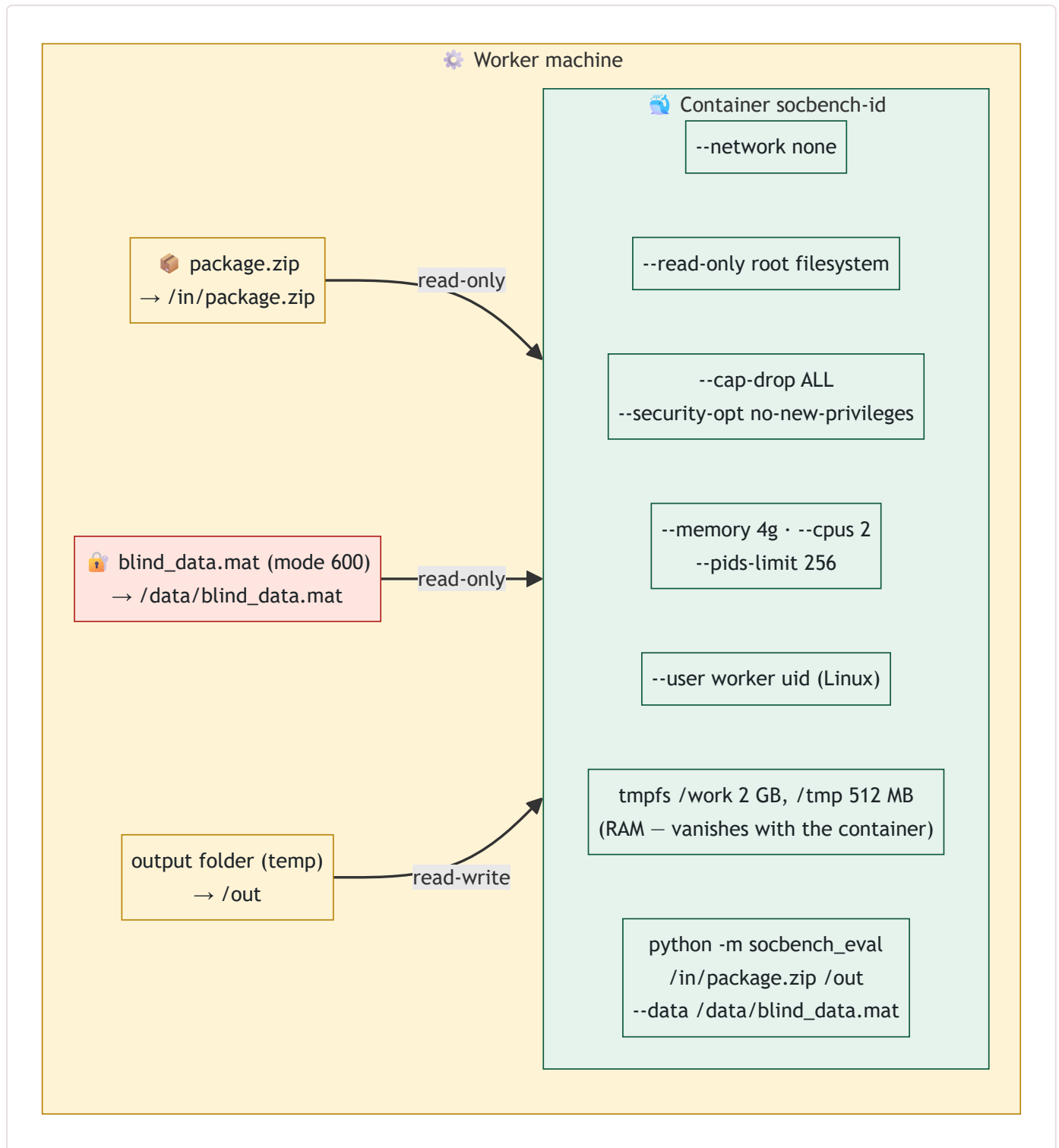
No transaction, no advisory lock, no queue service — a **compare-and-swap** on one column. If another worker won, the row no longer matches and the update touches zero rows.

Why a 30-minute stale lock is safe for a 6-hour evaluation: every progress line the evaluator prints is written to the job log, and *that write also refreshes* `lockedAt`. A live job is never stale. A job whose worker died stops being refreshed and becomes reclaimable after 30 quiet minutes.



Step 5 — Evaluate: the sandbox

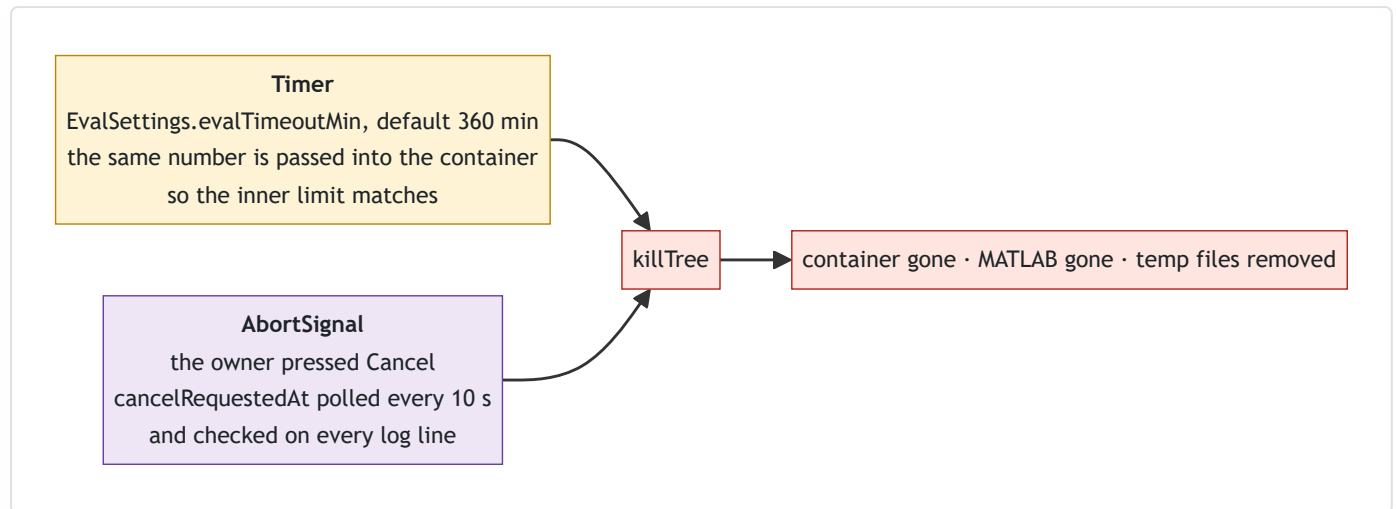
`runJob` marks the submission `RUNNING`, downloads the package to a temporary file (`storage.materialize`), and hands off to `PythonEvaluator.spawn()` in [python-evaluator.ts](#), which builds exactly one `docker run` command.



Three mounts and nothing else. The container never sees the host's environment variables, so no database password, SMTP credential or storage key can leak. For MATLAB packages the image is `socbench-eval-matlab`, the network is `bridge` (only so MATLAB can check out its licence), a `tmpfs /home/matlab` is added because MATLAB insists on writing preferences at start-up, and a 24-hour licence token is passed in — never the year-long one. The full command is written to the job log with licence values **redacted**.

Every line the container prints is streamed into the job log by `input.log`. That log is what the website shows as the live console, and where the progress bar comes from.

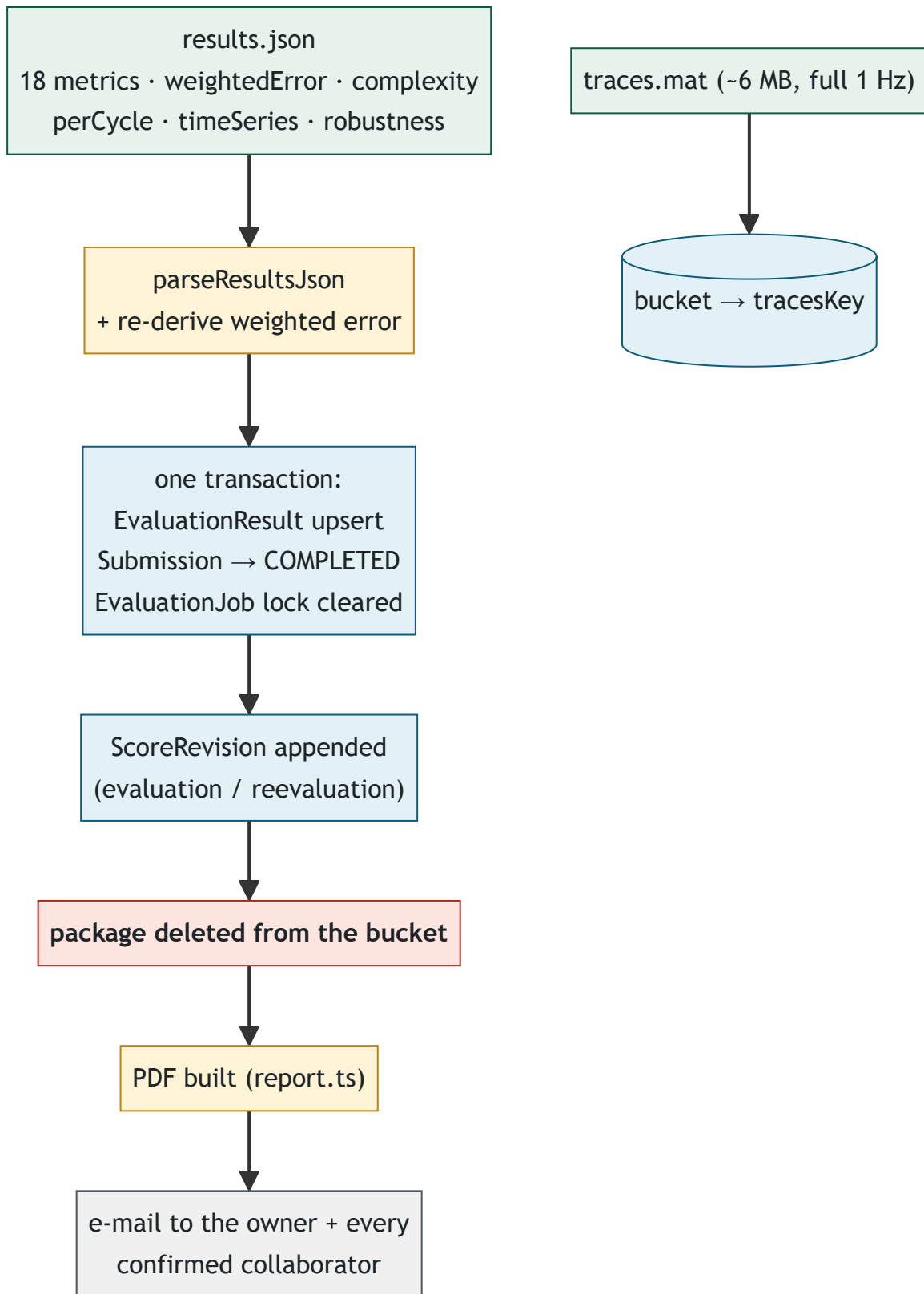
Two independent kill switches, both calling `killTree` (a `docker kill` plus a process-group kill so a MATLAB child dies with its parent):



Host mode (`EVAL_SANDBOX=none` , or a MATLAB package on a machine with no MATLAB image — this is how the laptop ran MATLAB natively during the outage): the same Python entry point, run directly, with an *allow-listed environment* — only `SOCBENCH_*` , `MATLAB_*` , `PATH` and locale variables reach it. Faster, not isolated. Dedicated low-privilege account only.

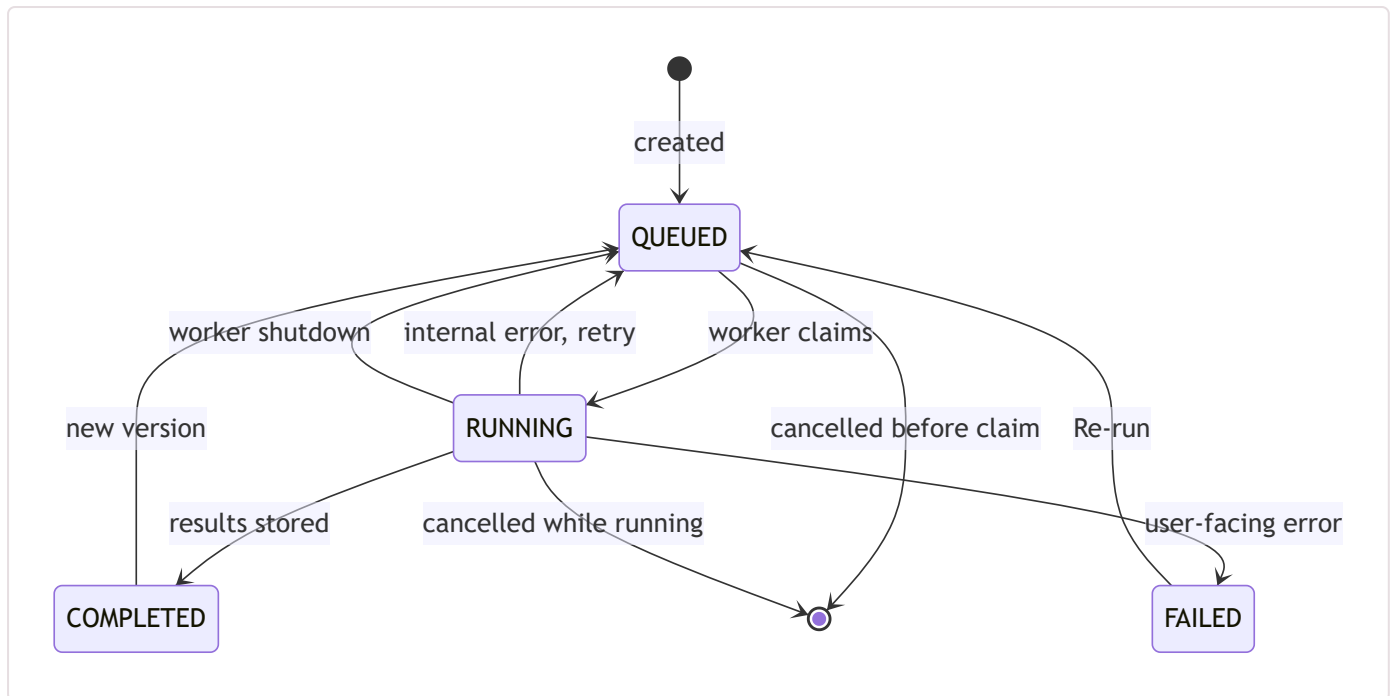
❏ Step 6 — Score and store

Back in `runJob` , `results.ts` parses `results.json` : all 18 metric keys must be finite numbers. The website **re-derives** the weighted error from them with the *active* weights and logs a note if it disagrees with the evaluator by more than 0.01 (it never has, with default weights; when an admin has overridden the weights, the website's value wins).



The large trace file goes to the bucket, not the database row, and replaces the previous one on a re-evaluation. The package is deleted *before* the e-mail step — nothing that happens afterwards can leave it behind.

Step 7 — The branches people forget



- **Retry happens only for our faults.** An `EvaluationError` marked `userFacing` — bad package, the model threw, timeout — is final; the exact message is stored and e-mailed. An internal error (Docker daemon down, a crash in our code) with `attempts < 2` unlocks the job and someone claims it again.
- **Cancel** while unclaimed → deleted outright. Once claimed → `cancelRequestedAt` is set; the worker notices within 10 s, aborts, and then either deletes the submission or — if this was a resubmitted v2+ — restores the previous version's score.
- **Worker shutdown** (Ctrl+C, `systemctl stop`, admin "stop"): in-flight evaluations are aborted with the reason `SHUTDOWN`; the job is handed back *with the attempt refunded* and the heartbeat row is deleted so the admin page does not show a ghost.

🔴 **Files to open, in order:** [submit/actions.ts](#) → [package-check.ts](#) → [worker.ts](#) → [run-job.ts](#) → [python-evaluator.ts](#) → [results.ts](#) → [worker-status.ts](#).

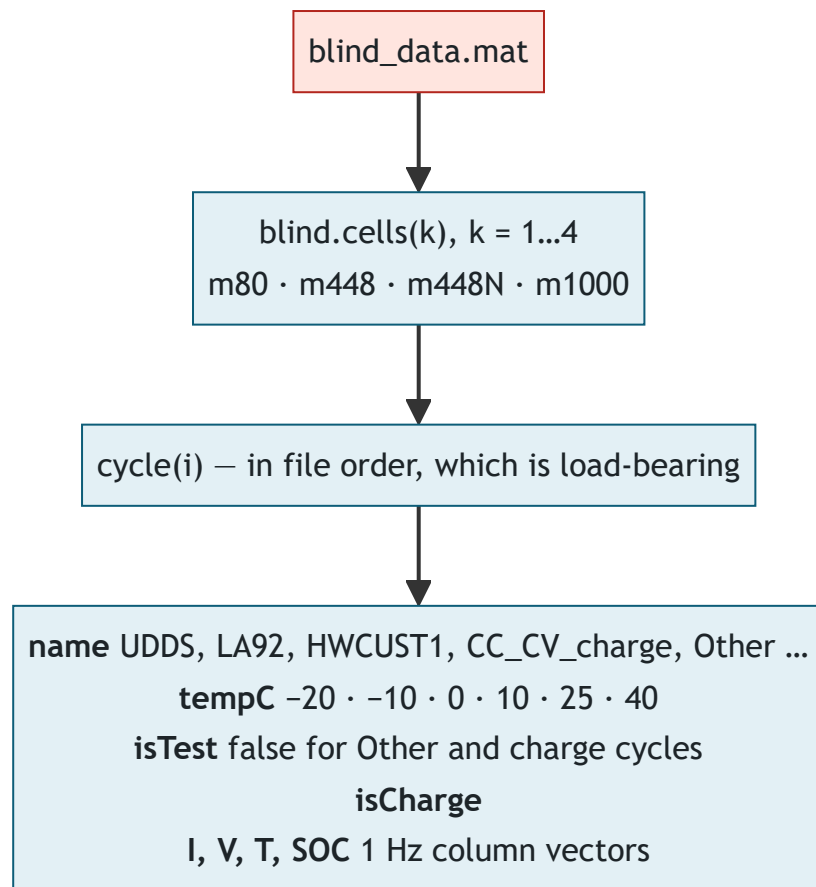
The evaluation engine — running the model

 **Plain English.** This and the next part describe the ~800 lines of Python that *are* the benchmark. They reproduce the lab's original MATLAB scoring tool exactly — checked against the real hidden data: the four reference models match every column of the old leaderboard to three decimal places, and the Python and MATLAB paths agree with each other. This part is about *running* the model: what data it is fed, in what order, and how. The next part is about turning the errors into a score.

Directory: [evaluator/python/socbench_eval/](#) — `__main__.py` (the run), `data.py` (load the .mat), `runner.py` (execute models), `pipeline.py` (jobs, scoring, complexity, traces).

 **The answer key:** `blind_data.mat`

The lab's data lives in MATLAB *tables*, which Python cannot read. [Export Blind Data.m](#) is run **once** to convert them to plain arrays in a version-7 `.mat` that scipy understands. **The cycle order is preserved and load-bearing** — every grouping in the scorer is positional, exactly as the original tool relied on.



The test grid

Every model faces the same grid. Each square is one drive cycle of roughly one to three hours of one-second measurements.

4 cells
m80 — 80 kg payload
🔒 m448 — 448 kg, fully blinded
m448N — 448 kg, open data exists
m1000 — 1000 kg, highest currents



× 6 temperatures
-20 · -10 · 0 · 10 · 25 · 40 °C

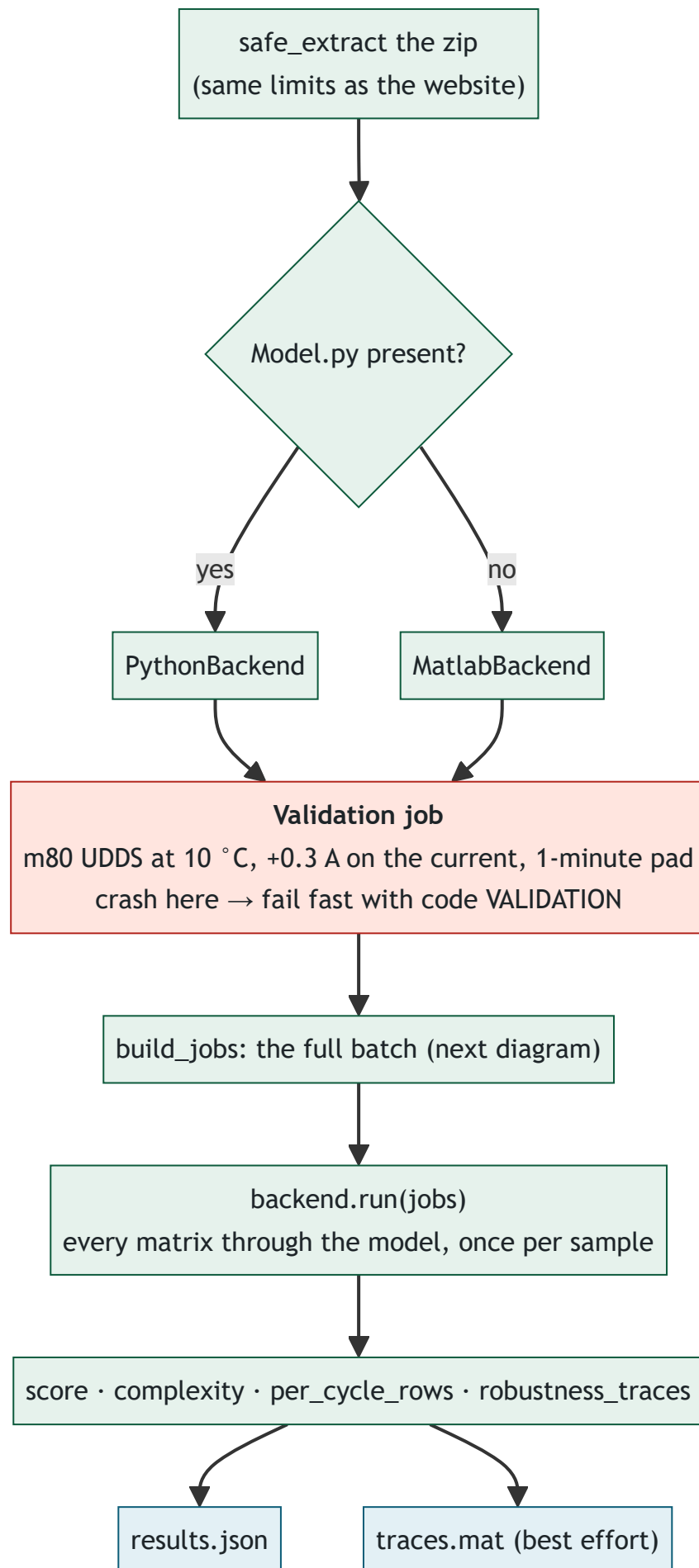


× 6 drive cycles
standard: UDDS · HWFET · LA92 · US06
non-standard: HWCUST · HWGRADE



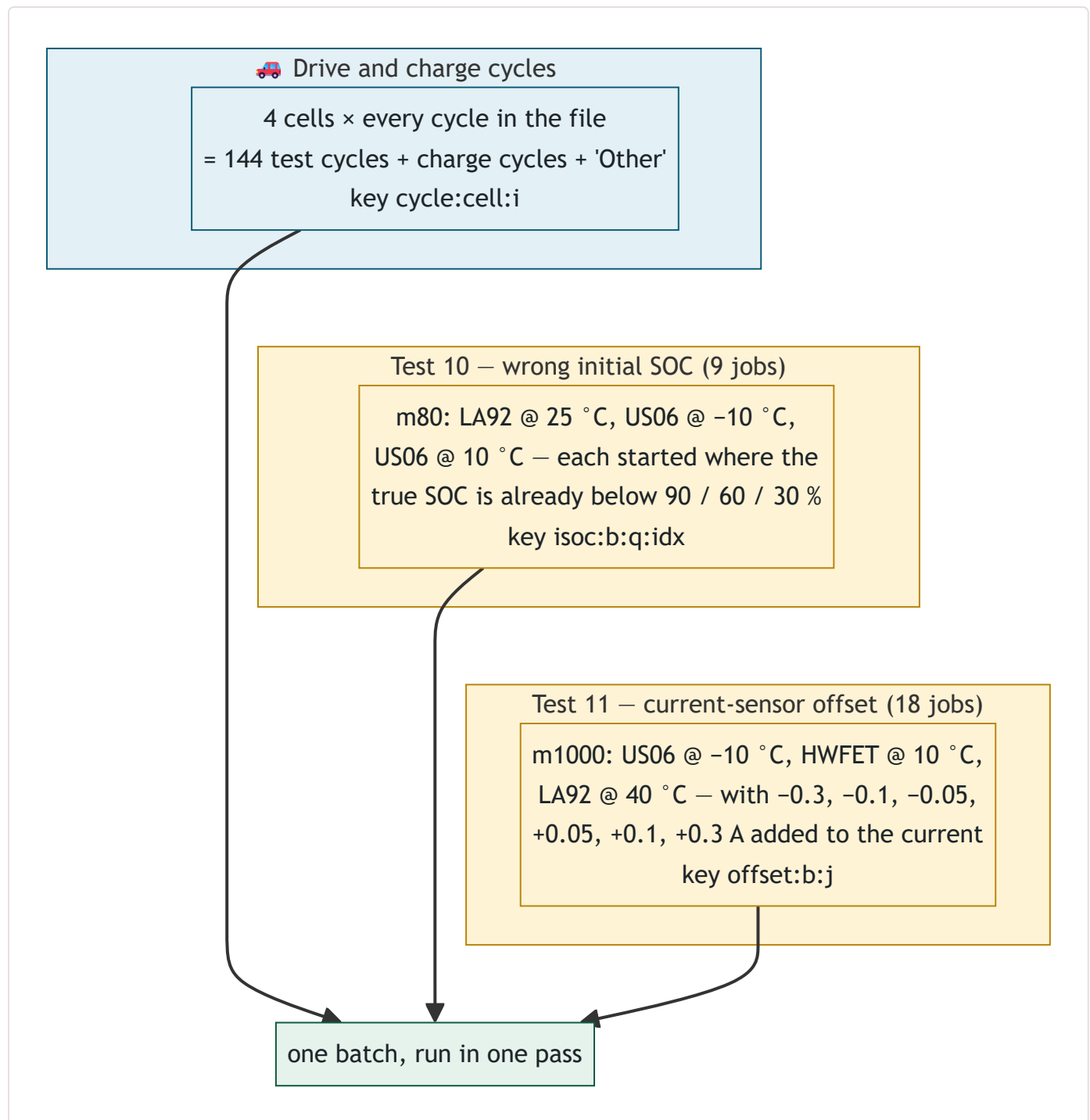
= 144 test cycles
+ CC-CV charging cycles

▶ The run, in order (`__main__.py`)



The validation job exists for one reason: a broken model fails in seconds with its real error message instead of after 45 minutes.

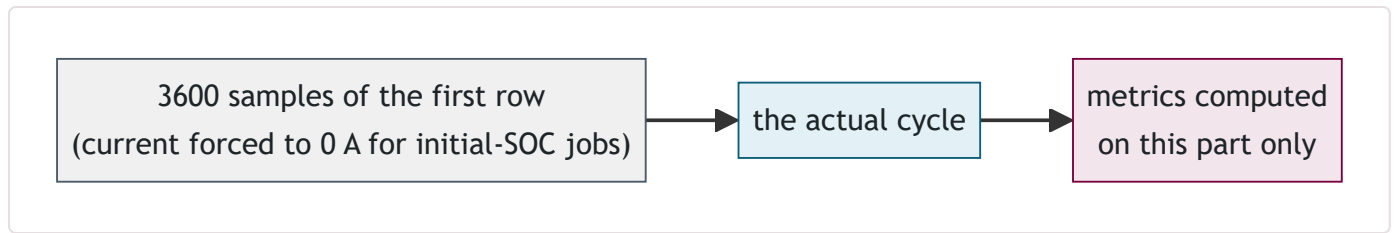
What `build_jobs` produces



Only the ± 0.3 A offset runs enter the score; the others are kept for the RMSE-vs-offset chart.

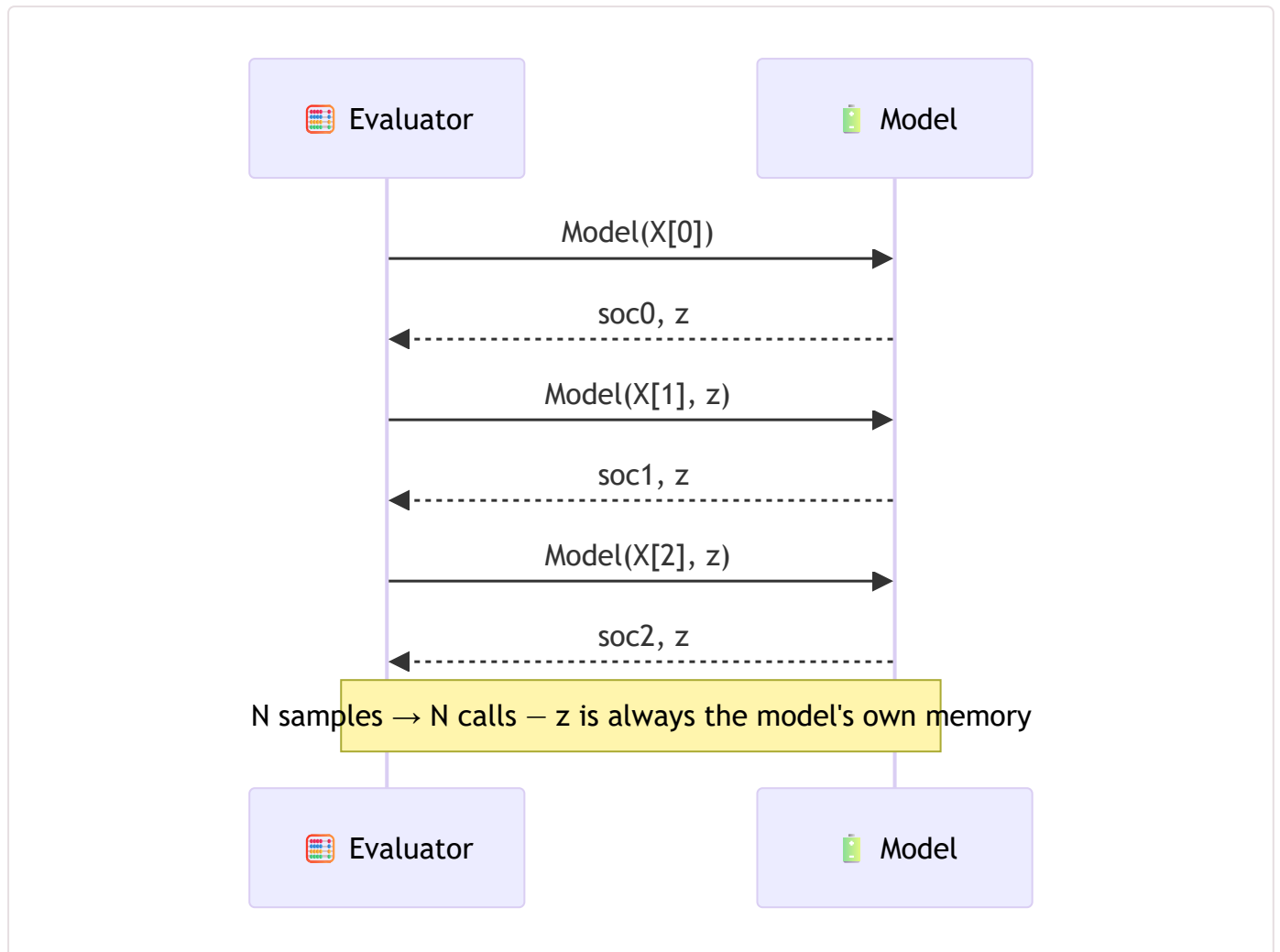
Padding. Every job is prefixed with one hour (3 600 samples) of its first row repeated, so estimators with internal memory — Kalman filters, RNNs — settle before scoring begins.

The pad is stripped from the prediction before metrics.



How the model is executed (`runner.py`)

Both backends honour the same contract: **call** `Model(X[i], z)` **once per sample, in order, carrying** `z` **forward**. That is how a BMS runs an estimator, and it is why a model cannot cheat by looking ahead.



This is what a submitted model looks like — the reference Coulomb counter, in full:

```
# Model.py
CAPACITY_AH = 4.6

def Model(X, z=None):
    """X = [current (A, negative = discharge), voltage (V), temperature (C)]
    Returns (SOC estimate 0..1, memory z for the next call)."""
    current = float(X[0])
    if z is None:
        soc = 1.0
    else:
        soc = float(z) + current * (1 / 3600) / CAPACITY_AH
    return soc, soc
    # memory z = previous SOC
```

BACKEND	HOW	ERRORS
PythonBackend	imports <code>Model.py</code> with <code>importlib</code> , loops in-process	re-raised as <code>ModelError</code> with the traceback trimmed to the submitter's own frames
MatlabBackend	writes every input matrix to one <code>in.mat</code> , starts one <code>matlab -batch</code> session running Run_Model.m (40 lines: loop over matrices, <code>tic/toc</code> around each), reads <code>out.mat</code> back	"Undefined function 'butter'" is translated to "that is the Signal Processing Toolbox, not installed here" via a lookup table

`Run_Model.m` prints `[Run_Model] k/n done` after every matrix; the Python side turns that into the same `NN.N% | key` progress lines the Python backend emits, so the website's progress bar is identical for both. Progress is weighted by *samples*, not matrix count, so long cycles move the bar proportionally.

Dry run

`--dry-run` uses **open** data shipped with the repo (`dryrun_data.mat`, 2 h of public m80 data): the +0.3 A validation, then one padded cycle. No blinded data is mounted, no leaderboard row. This replaced the lab's downloadable "Model Submission Test Tool".

There is deliberately **no mock evaluator**: every number the site has ever shown — dry run or full evaluation, on a laptop or on the VM — came from this pipeline. Developing without the blinded data means developing without evaluations, which keeps a fabricated score from ever being mistaken for a real one.

🚀 **Files to open, in order:** [main.py](#) → [data.py](#) → [runner.py](#) → [Run_Model.m](#) → [Export_Blind_Data.m](#).

PART 5



The evaluation engine — scoring, complexity and outputs

 **Plain English.** After the model has run on every cycle, the evaluator has one error number per cycle. This part shows how those become the 18 numbers on the scorecard, how the 18 become one leaderboard score, what "complexity" means, and what gets sent back to the website for the charts.



From one number per cycle to 18 metrics

Per-cycle RMSE (in % SOC) is arranged as a matrix `R` — rows are test cycles in file order, columns are the four cells — and the metrics are positional means over it, mirroring the original `Obtain_Output_Data.m`.

Matrix R – per-cycle RMSE
rows = test cycles in file order
columns = m80 · m448 · m448N · m1000

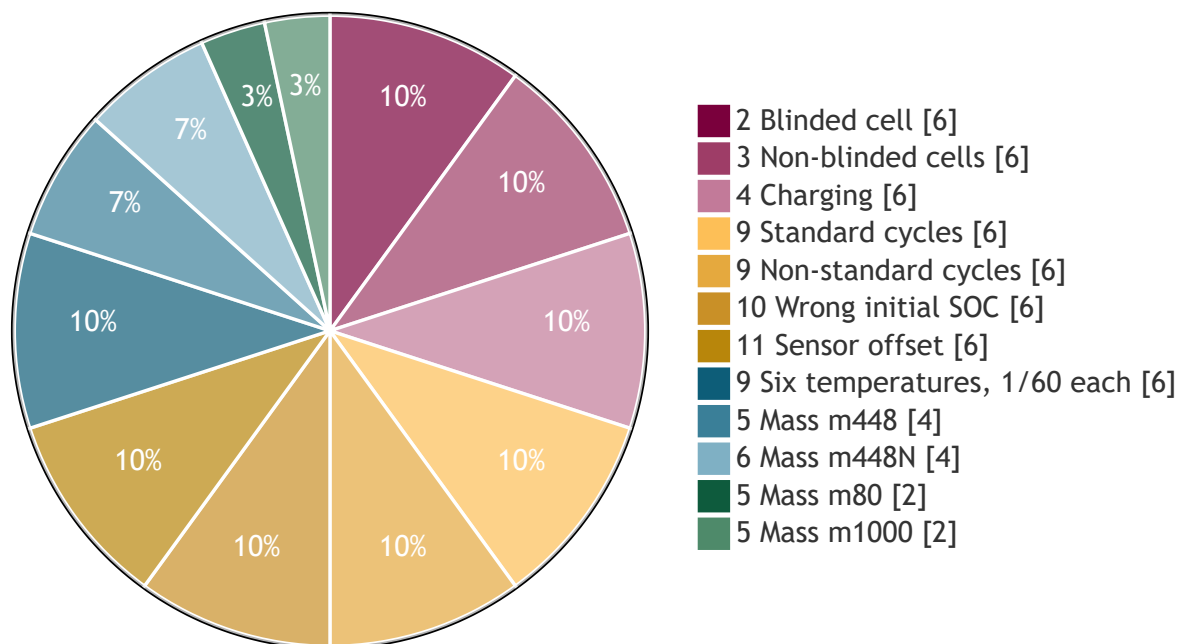
the other runs
charge-cycle RMSEs
9 initial-SOC RMSEs
18 offset RMSEs

Positional means → 18 metrics
1 allCells – mean of non-zero R (weight 0)
2 blindedCell – the m448 column
3 nonBlindedCells – mean of the other three
4 charging – the charge cycles
5-8 mass m80 / m448 / m448N / m1000
– the four column means
9 standard vs non-standard cycles
– blocks of 4 and 2
10 six temperatures – m80 in blocks of 6
(-10 and -20 swapped)
11 initialSocError – 3 : 2 : 1 for 90 / 60 / 30 %
12 currentSensorOffset – mean of the ± 0.3 A runs

weighted error = $\sum \text{weight} \times \text{metric}$
weights sum to exactly 1

The weights

Share of the weighted error, by test case



The weights in metric order: `[0, 1/10, 1/10, 1/10, 1/30, 2/30, 2/30, 1/30, 1/10, 1/10, 1/60 ×6, 1/10, 1/10]`. Seven categories carry a tenth each; the four payloads share a fifth; the six temperatures share a tenth. `allCells` has weight zero — it is the headline number people recognise from the paper, but it would double-count everything else.

A worked example. A model with these RMSEs (all in % SOC): blinded cell 3.0, non-blinded 2.0, charging 4.0, standard 2.5, non-standard 3.5, initial-SOC 5.0, offset 6.0, the four masses 2.0 / 2.5 / 3.0 / 4.0, the six temperatures 6, 4, 3, 2, 1.5, 1.5:

GROUP	CONTRIBUTION
Seven categories at 1/10	$0.1 \times (3.0 + 2.0 + 4.0 + 2.5 + 3.5 + 5.0 + 6.0) = \mathbf{2.60}$
Four masses at 1/30, 2/30, 2/30, 1/30	$(2.0 + 2 \times 2.5 + 2 \times 3.0 + 4.0) / 30 = \mathbf{0.57}$
Six temperatures at 1/60	$(6 + 4 + 3 + 2 + 1.5 + 1.5) / 60 = \mathbf{0.30}$
Weighted error	3.47

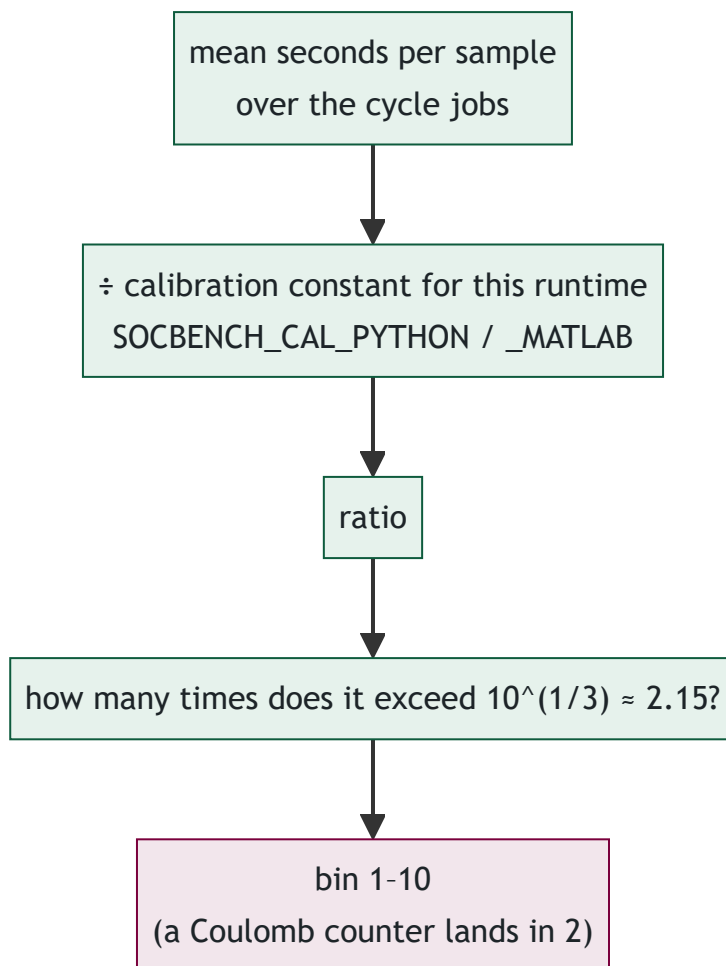
Two extras the scorer emits: `suspicious = mean(R) > 25` (the original tool *withheld* such results; we flag it in the log and leave the decision to an administrator), and

`maxError` = the single worst instantaneous error across all test cycles.

RMSE, in words

For one cycle: take the difference between the estimated and true SOC at every second, square each difference (so misses in both directions count and big misses count more), average them, and take the square root to get back to % SOC. A model that is always 2 % off scores an RMSE of 2. A model that is perfect most of the time but 30 % off for a minute scores worse than that minute's share would suggest — which is the point.

Complexity

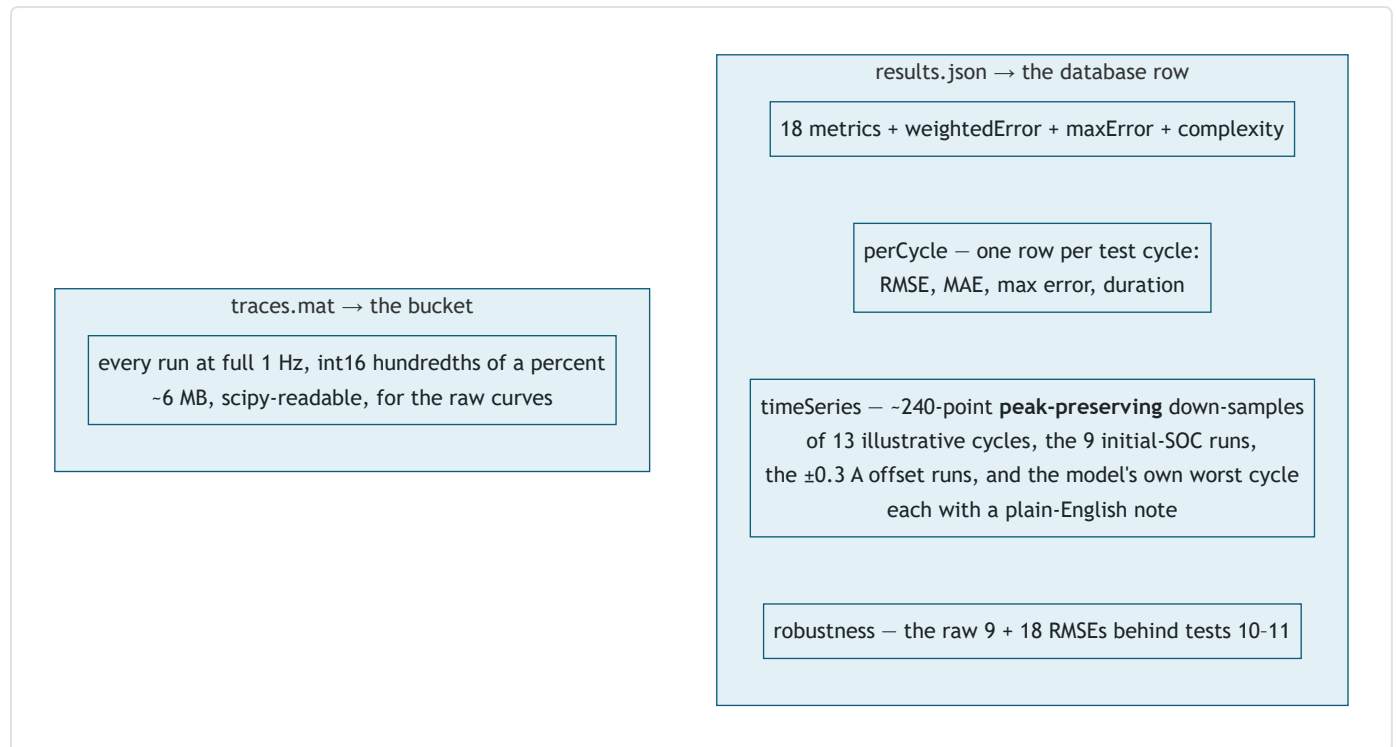


Informational only — it never enters the weighted error. The constants are per evaluation host. A caveat worth knowing: MATLAB's JIT is ~20× faster inside a function than in a script, so the MATLAB constant can only be re-measured by calling the model the way `Run_Model.m` does. For a researcher, complexity answers "would this fit on a real

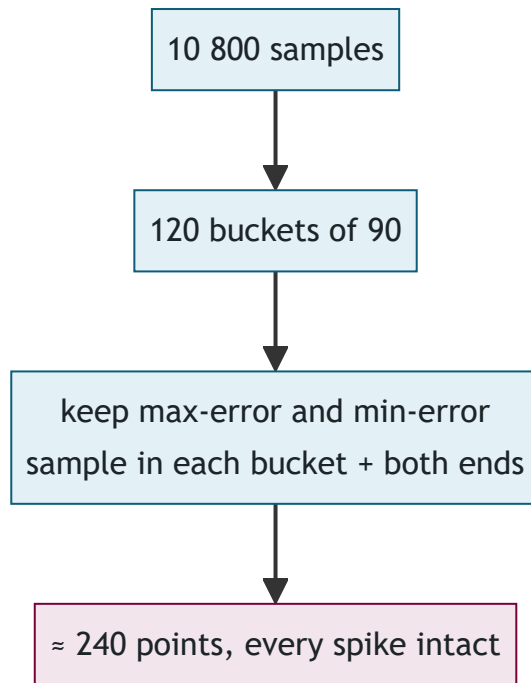
battery controller?" — a bin of 2 is a few arithmetic operations per second; a bin of 9 is a neural network that would need a much larger processor.



What leaves the engine



Peak-preserving down-sampling. A three-hour cycle has 10 800 points; a chart can show about 240. Taking every 45th point would hide the very spike that produced the max-error number. Instead `display_indices` splits the cycle into 120 buckets and keeps the sample with the largest *and* the smallest signed error in each — so every spike survives and the plotted line agrees with the scorecard.

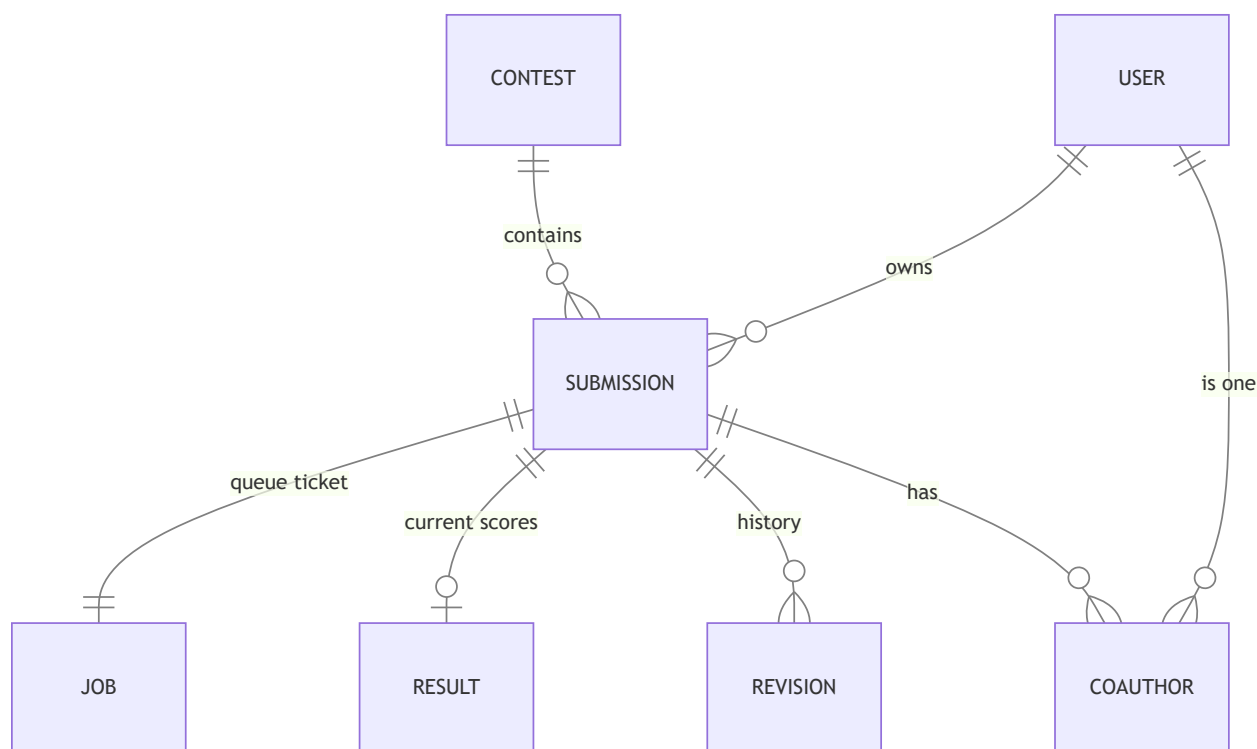


✦ **Files to open, in order:** [pipeline.py](#) (`score` , `complexity` , `display_indices` , `per_cycle_rows` , `robustness_traces` , `write_full_traces`) → [test-cases.ts](#) (the same 18 with labels and descriptions for the website).



The data model

Plain English. The database is a set of tables. A person has submissions; each submission has one queue ticket, one current result, and a history of everything that ever happened to it. Separate tables track contests, worker machines, settings and the admin activity feed. One file — `prisma/schema.prisma` — describes all of it, and both the website and the worker read that same description.

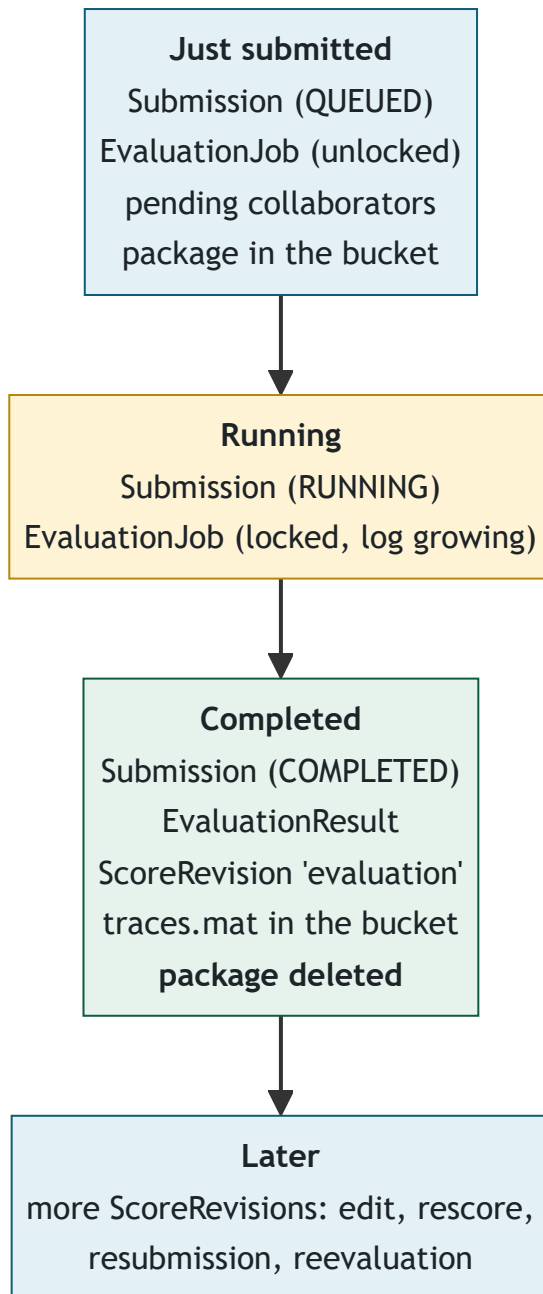


The picture shows only how the tables relate; the names are shortened for space — **JOB** is `EvaluationJob`, **RESULT** is `EvaluationResult`, **REVISION** is `ScoreRevision`, **COAUTHOR** is `SubmissionCollaborator`. Each table's columns are listed just below.



Which rows exist at each moment

One submission's footprint grows as it moves through its life:



The columns that matter

TABLE	KEY COLUMNS
User	email , passwordHash (bcrypt), role USER/ADMIN, emailVerified (a <i>timestamp</i> , not a boolean), public-profile fields, adminNotify JSON of e-mail toggles, avatar bytes ≤ 400 KB

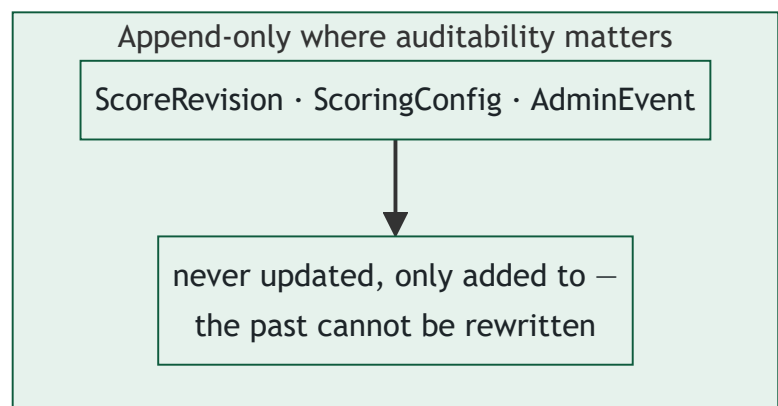
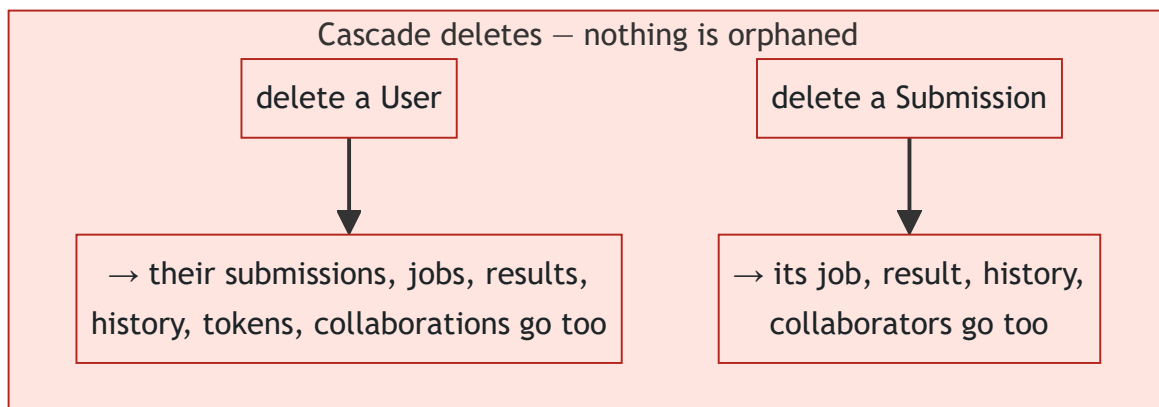
TABLE	KEY COLUMNS
Submission	seq (the human-visible #), modelName / description / modelType , isPrivate , isHidden (admin moderation), fileKey into the bucket, runtime (python / matlab , set at upload), status , version (bumped by "submit new version"), optional contestId
EvaluationJob	attempts , lockedAt / lockedBy (the compare-and-swap lock), log (the live console text), cancelRequestedAt
EvaluationResult	weightedError , complexity , the 18 metric columns allCells ... currentSensorOffset , maxError , perCycle JSON, down-sampled timeSeries JSON, robustness JSON, tracesKey , evaluatorVersion stamp
ScoreRevision	kind (evaluation · failure · rescore · reevaluation · resubmission · edit · cancelled · legacy), the score and 18 metrics at that moment, note , by
SubmissionCollaborator	notifiedAt (invited), acceptedAt (public), inviteToken for the e-mail links
DryRun	its own status/lock, result JSON, log — never touches blinded data, never on the leaderboard

Tables that stand alone:

TABLE	ROLE
WorkerHeartbeat	One row per worker process (hostname-pid), refreshed every 15 s: liveness, what it is busy with, its runtimes, a pending admin command, machine diagnostics, the last 200 console lines
EvalSettings	A single row (id = 1): evaluation timeout 360 min, dry-run timeout 10 min, submissions per day 3. Read with a 15 s cache by both website and worker
ScoringConfig	Append-only weight overrides; the newest row is active; absent = the published defaults
AdminEvent	The activity feed on Admin → Overview — also the memory for outage alerting
RateLimitHit	One row per rate-limited attempt; counted per key per window

TABLE	ROLE
ContactMessage	The feedback inbox
Contest , ContestEntry	A time-boxed event with its own frozen leaderboard, and who registered for it

Two conventions that recur everywhere



✦ **Files to open, in order:** [prisma/schema.prisma](#) (read it top to bottom once — it is the best map of the system) → [src/lib/db.ts](#) → [src/lib/history.ts](#).

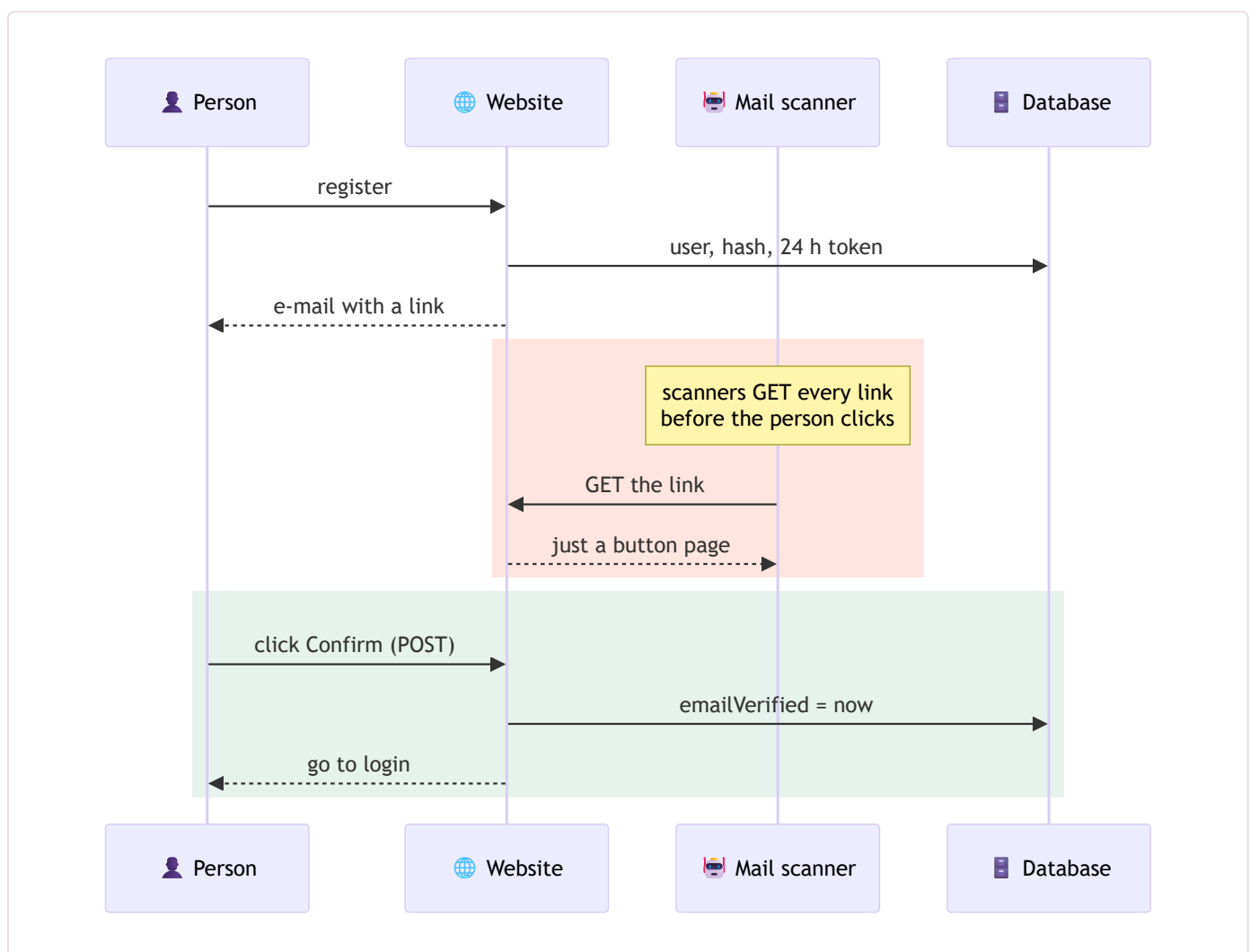


Authentication and accounts

💡 **Plain English.** Register with e-mail and password, prove you own the e-mail address by clicking a confirmation, then sign in. Passwords are stored scrambled, never in the clear. Administrators are either promoted on the admin page or listed in a configuration variable. Every sensitive action is limited to a few attempts per hour so nobody can guess passwords by brute force.

Files: [auth.ts](#), [auth.config.ts](#), [middleware.ts](#), [\(auth\)/actions.ts](#), [verify/actions.ts](#), [rate-limit.ts](#).

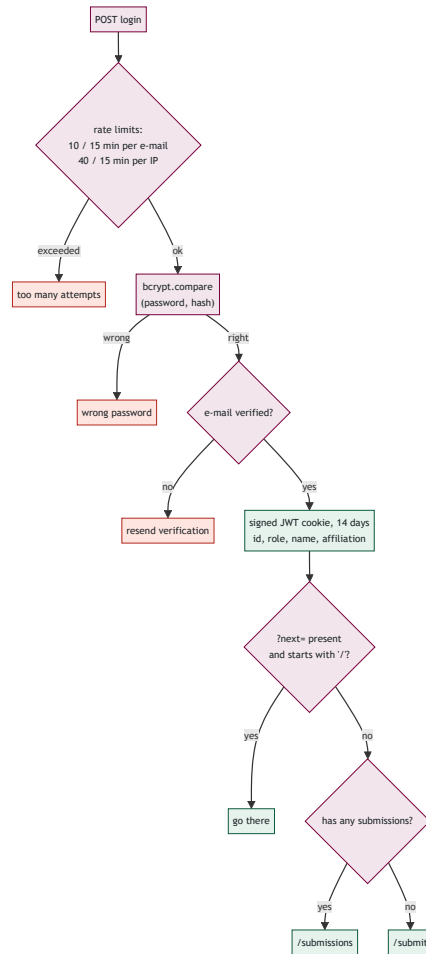
Register and verify — and why the verification link is a button



The GET is side-effect free on purpose: before this change, mail scanners (Microsoft Safe Links, Proofpoint, Gmail) were consuming the one-time token before the person clicked. The token is also deliberately **left in place after success**, so a scanner firing after the

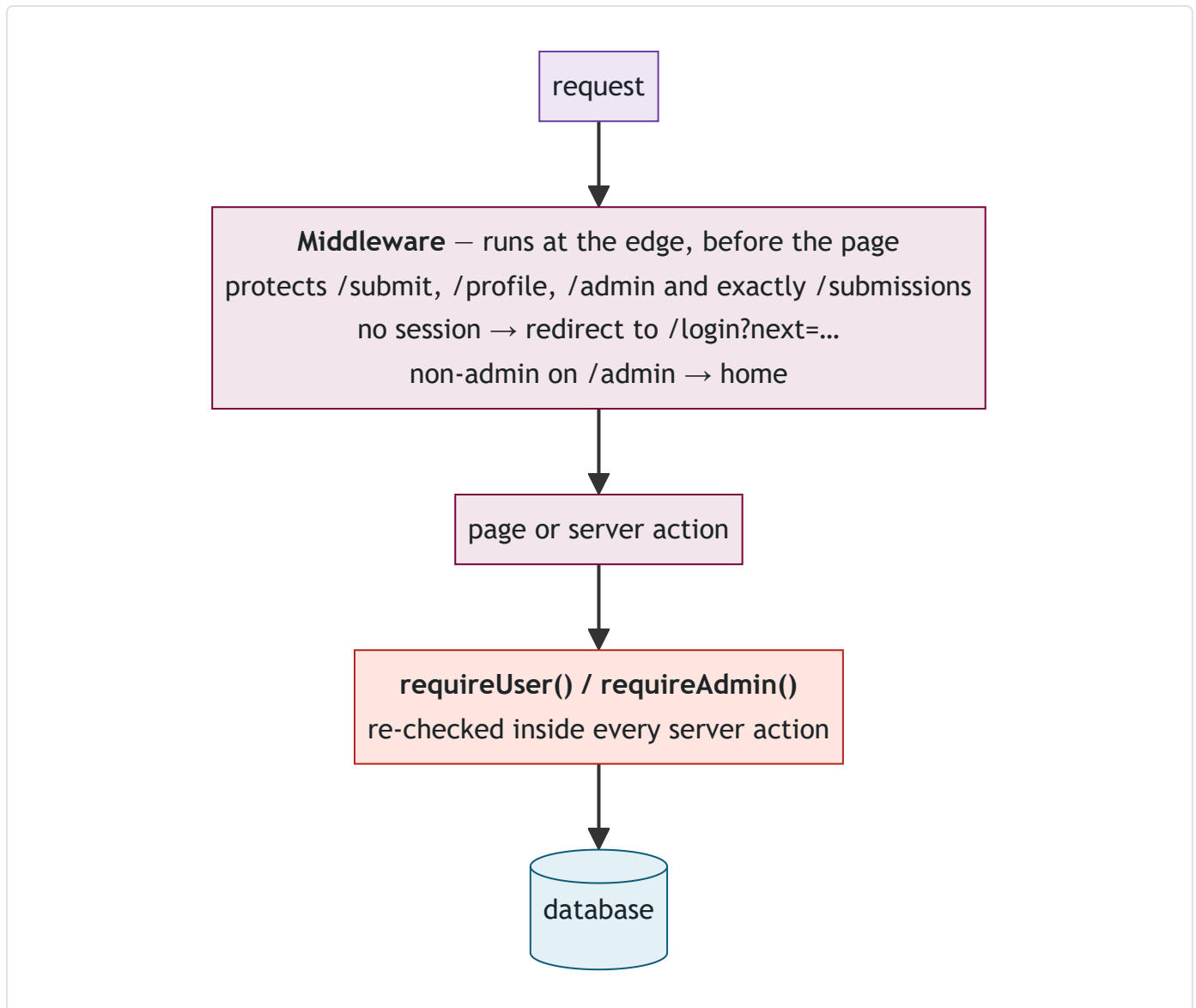
click, or a double-click, lands on "already verified" instead of an error. The page even tells the user why: "This extra click keeps automated e-mail security scanners from activating accounts on your behalf."

Login, and where you land afterwards



The `startsWith('/')` check blocks open redirects to other sites. Sessions are signed JWT cookies — no session table, so the database is not touched to verify a request. The `next` parameter is how a deep link survives login: open `/submissions/abc` while signed out and you come back to exactly that page.

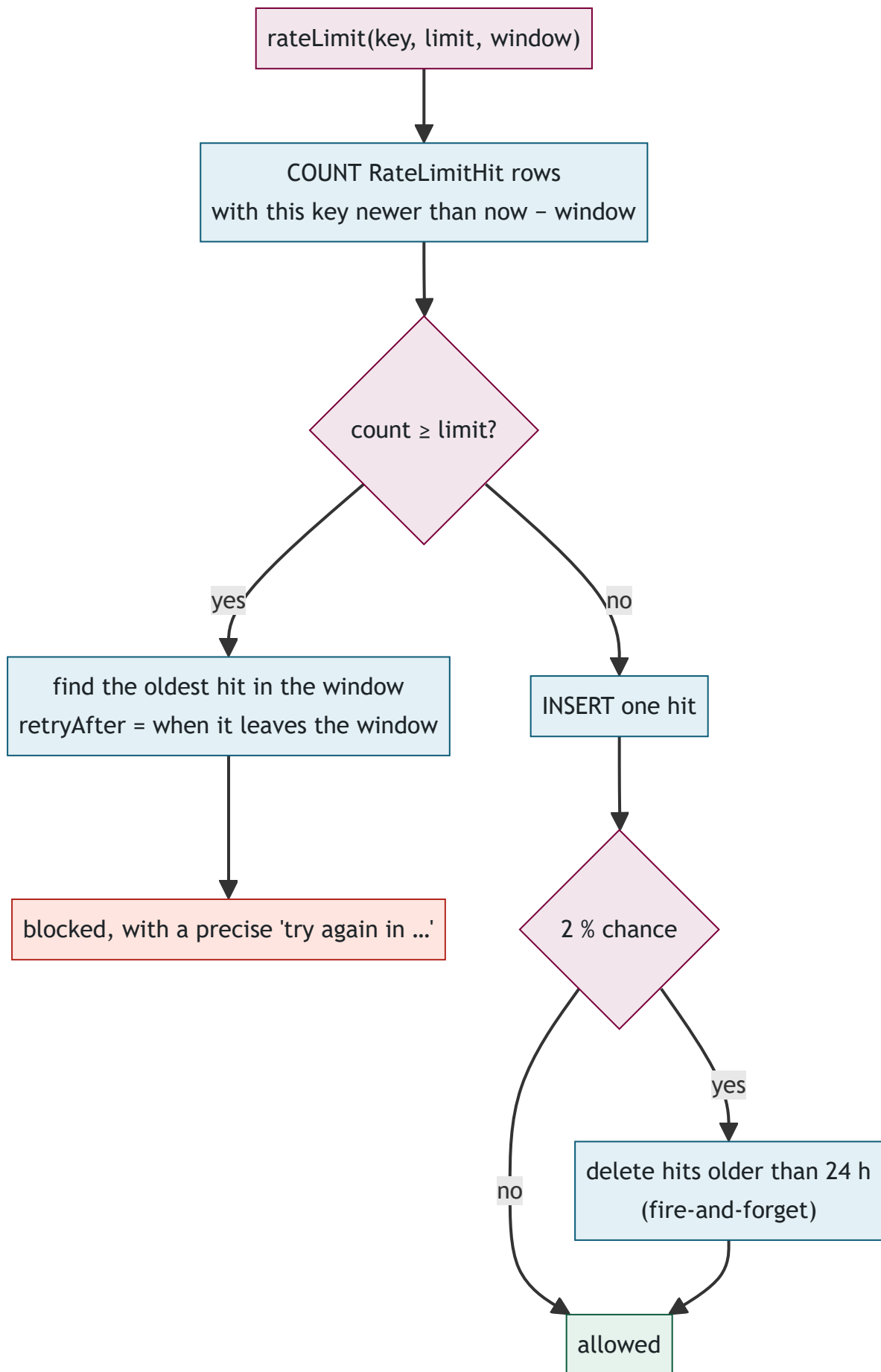
🛡️ Authorization has two layers, and the first is not the boundary



Middleware is a convenience that keeps signed-out users off the pages; the real check is inside each action. Two details worth knowing: `auth.config.ts` is split from `auth.ts` because middleware runs at the *edge* where Prisma cannot load; and the `ADMIN_EMAILS` variable is **re-read on every call** and applied in the session callback, so adding an address there makes that person an admin on their next request — no waiting for a token to expire.

🔄 Rate limiting without a memory

Vercel functions are stateless — an in-memory counter would reset on every request — so the counter is the database.



The 2 % probabilistic prune means there is no clean-up cron to run or forget. Configured limits:

ACTION	LIMIT
Register	5 / hour / IP
Resend verification	3 / hour / e-mail
Login	10 / 15 min per e-mail and 40 / 15 min per IP
Password reset request	3 / hour / e-mail, 10 / hour / IP
Contact form	5 / hour / IP
Upload URLs	30 / hour / user
Resend a collaborator invite	1 / 12 h per person
Dry runs (separate quota, from real rows)	5 / hour; admins unlimited

Other account details

- Password policy: ≥ 8 characters with an upper-case letter, a lower-case letter and a digit. Hash: bcrypt, cost 11 (about a tenth of a second per check — trivial for one login, ruinous for a million guesses).
- Tokens: 256-bit random, one live token per purpose per user; verification 24 h, reset 1 h. A successful password reset also sets `emailVerified` — proving mailbox control is treated as equivalent to clicking the verification link.
- Resend-verification and forgot-password always answer "if that address is registered..." so they cannot be used to discover accounts. Registration reports a duplicate e-mail explicitly — a deliberate usability trade-off.
- Avatars are resized in the browser to 256 px JPEG, capped at 400 KB on the server, stored as bytes in Postgres, served with a 24 h cache and an ETag derived from `avatarUpdatedAt`.

🔴 **Files to open, in order:** [auth.config.ts](#) → [auth.ts](#) → [middleware.ts](#) → [\(auth\)/actions.ts](#) → [verify/actions.ts](#) → [rate-limit.ts](#).



The web tier: submitting, results, leaderboard

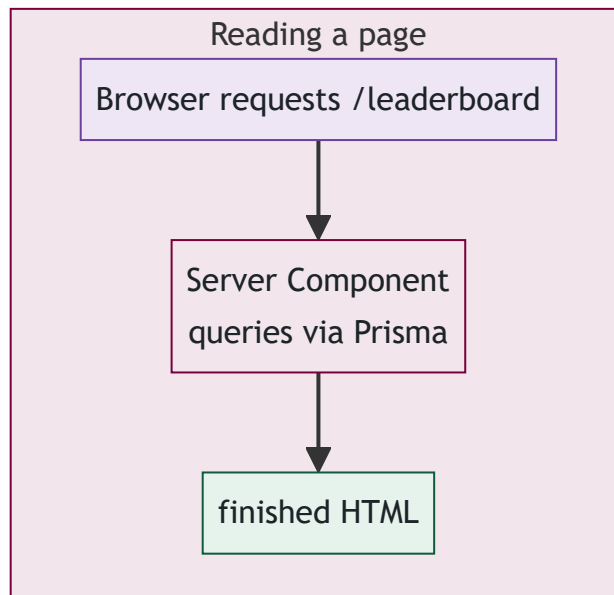
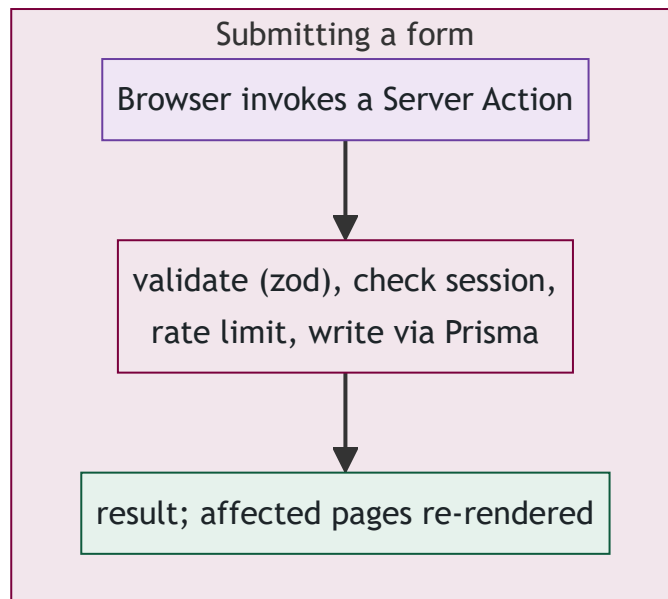


Plain English. The submit page lets you test a package for free before spending one of your three daily submissions. The results page explains the score in plain language first, then shows the numbers, then the charts. The leaderboard ranks public models; your private ones show a "ghost" rank so you can see where you would stand without displacing anyone.



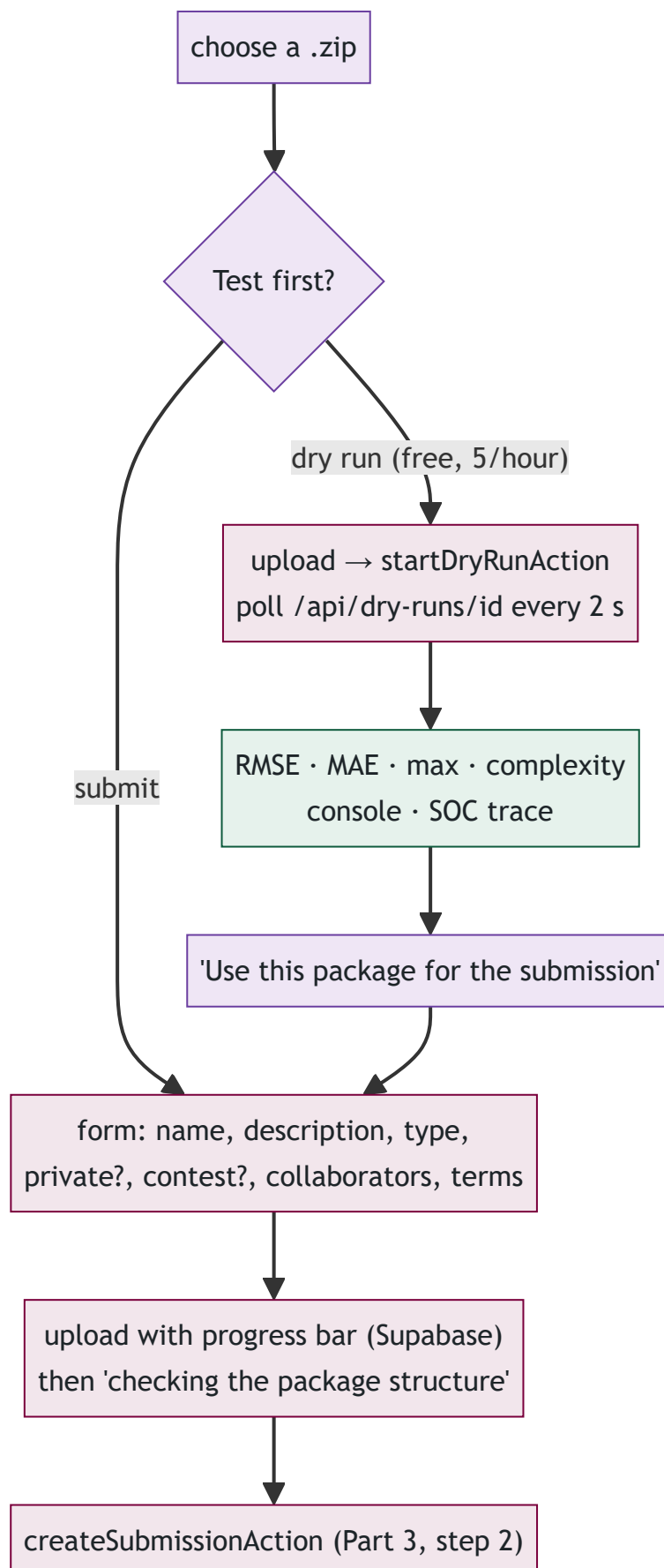
There is no REST API

Pages read the database directly in Server Components and return finished HTML. Forms call Server Actions. This is the pattern that surprises people coming from an API-first codebase:



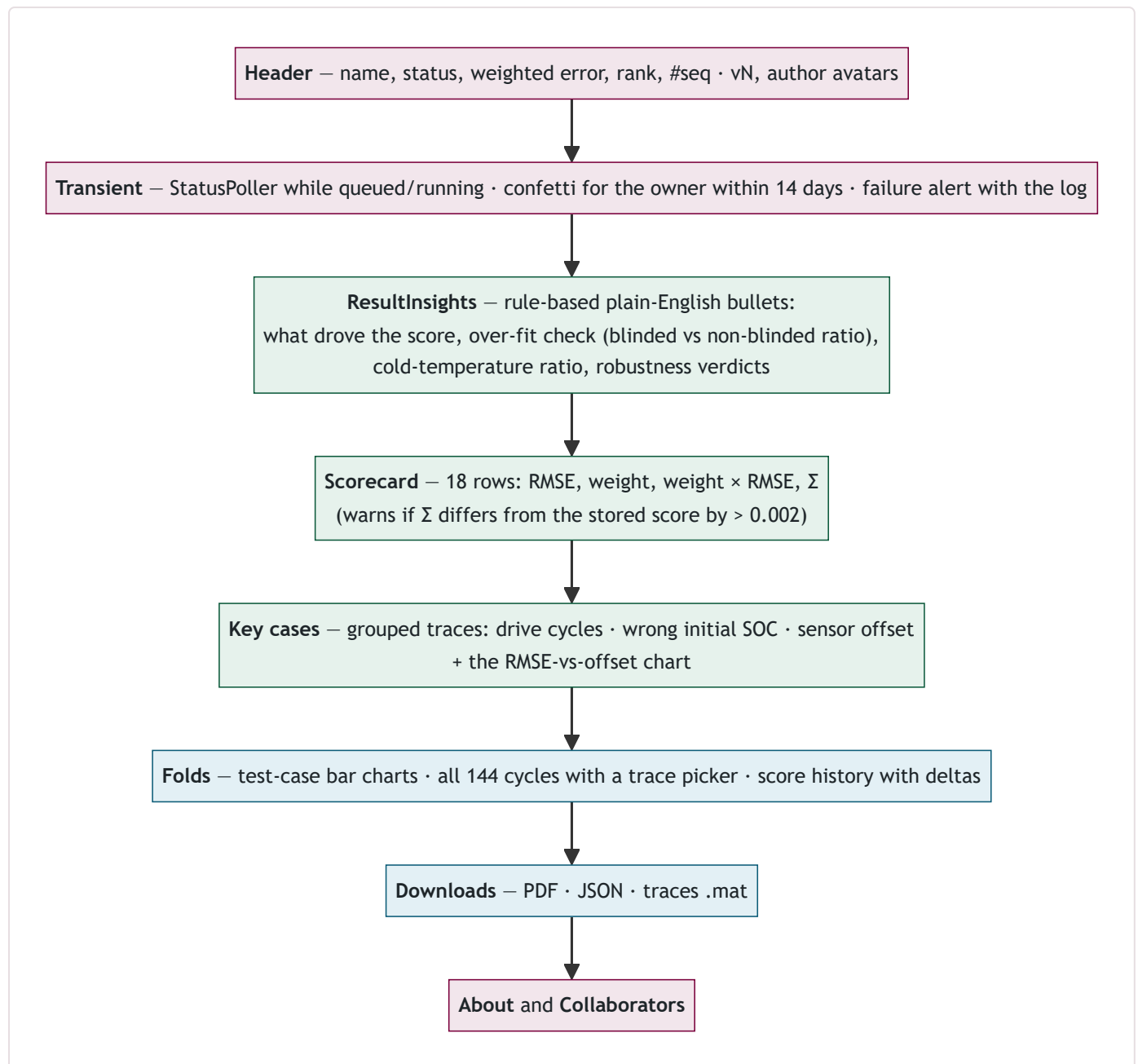
Route handlers under `src/app/api/` exist only for the handful of things that genuinely need a URL: status polling, the PDF and JSON downloads, upload URLs, avatars, the health check.

The submit page



The form is fully controlled — React 19 resets uncontrolled inputs after an action — and validates with the *same* zod schema the server uses, so a rejection is caught before the upload. Picking a contest force-clears *private*: contest entries must stay public for the frozen contest leaderboard.

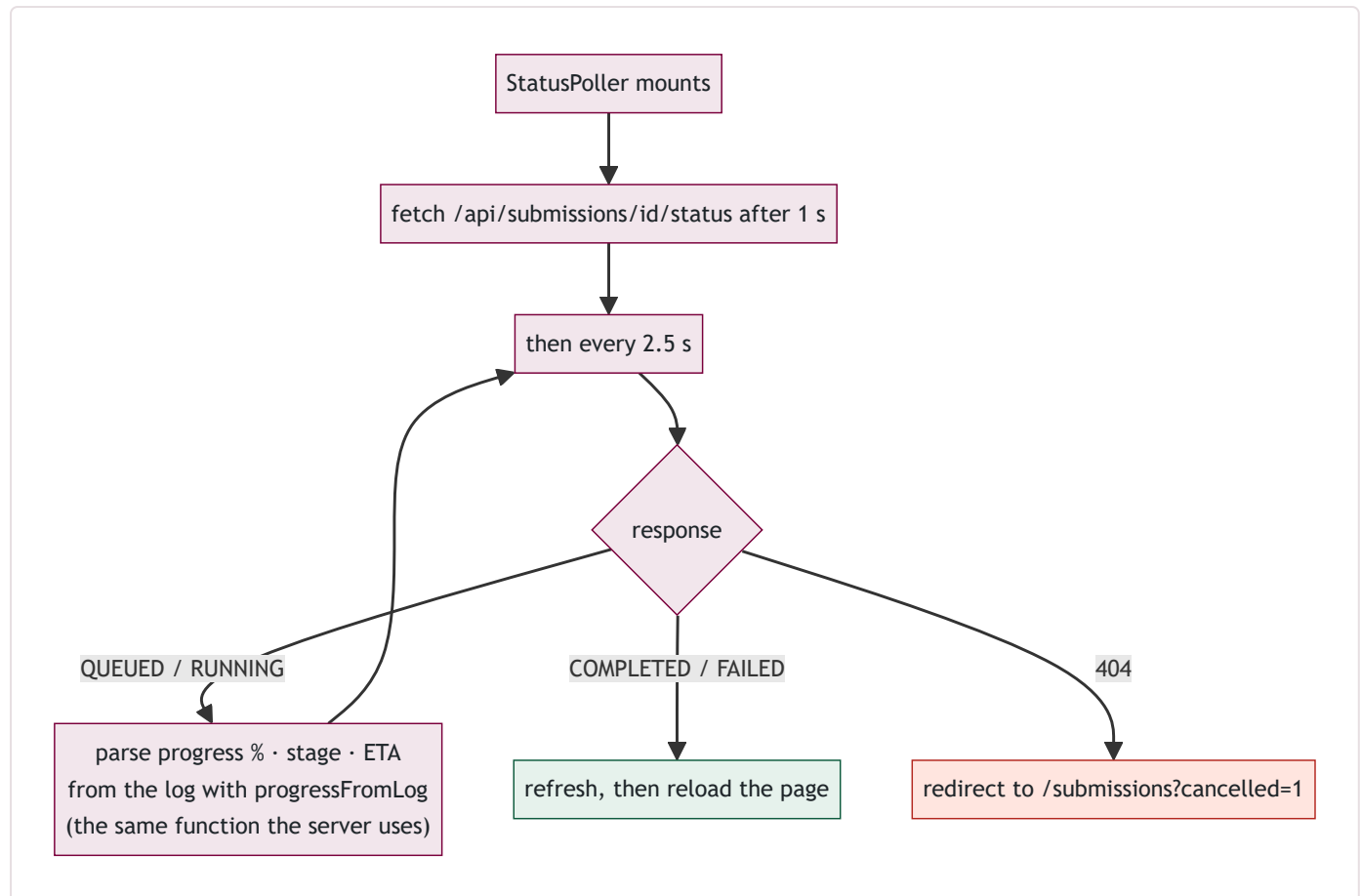
The results page, top to bottom



The order is deliberate: *why* before *what* before *detail*. A researcher who reads only the insights and the scorecard knows what to fix. Access is decided by `canViewSubmission`: admin, owner, collaborator, or public-and-not-hidden. The **Manage** menu (owner or admin) offers: edit name/description/type (name and type freeze once a contest closes); submit a new version (same submission, `version + 1`, previous

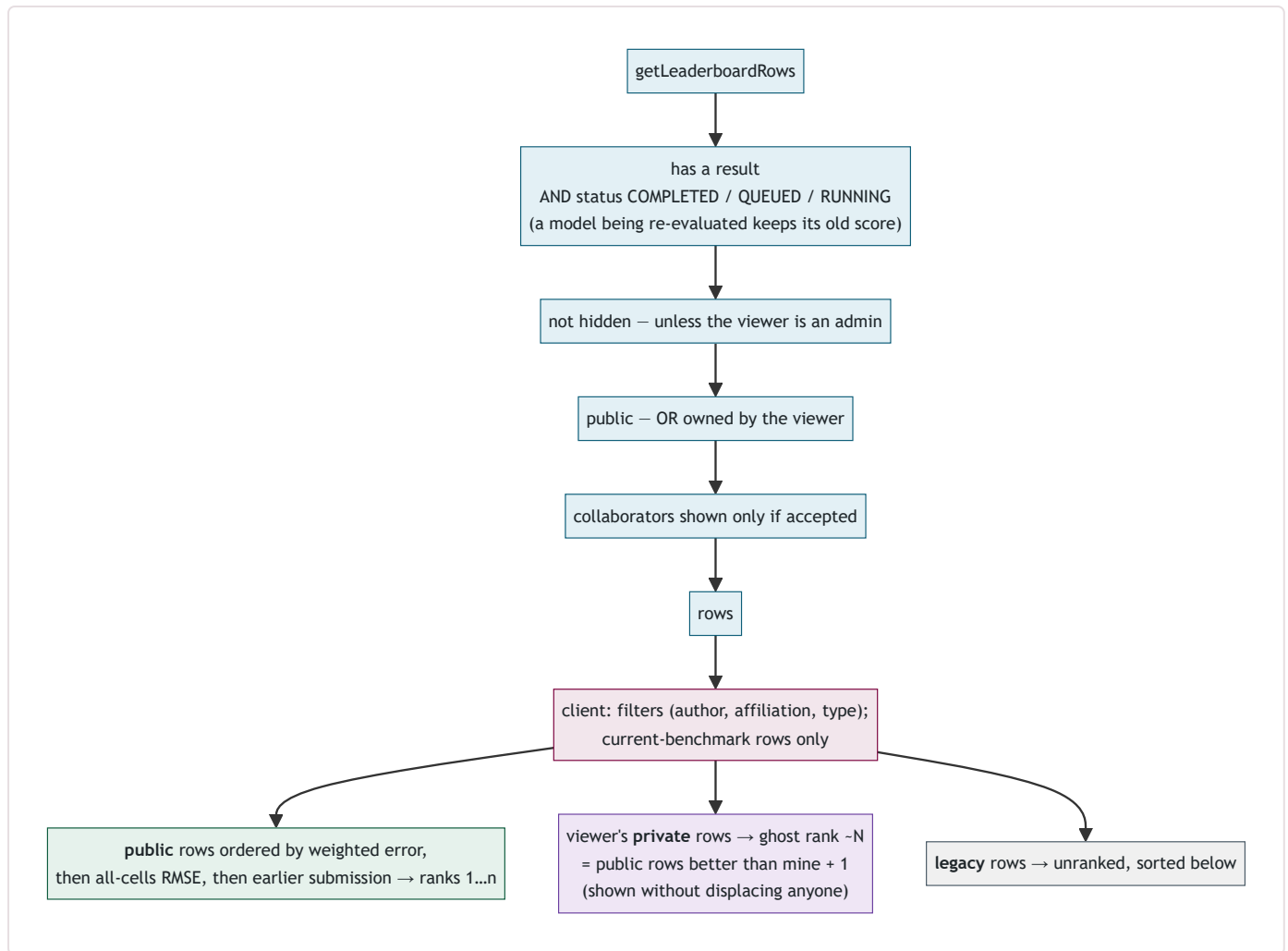
score kept in history); make private/public (hidden for contest entries); re-run when FAILED; cancel while queued/running; delete.

Live status while waiting



The progress line format is `NN.N% | cycle:m80:3`; the regex, the "validation stage pinned to 1 %" rule and the ETA formula all live in one file, [progress.ts](#), so the page and the queue estimator can never disagree. *The ETA formula*: if reaching 40 % took 8 minutes, the remaining 60 % is estimated at $8 \times 60 / 40 = 12$ minutes, minus however long since the last progress line, floored at 30 s.

The leaderboard: which rows, and how they are ranked



A worked ranking. Five models, and you are signed in as the owner of the private one:

MODEL	WEIGHTED ERROR	ALL-CELLS	SUBMITTED	PUBLIC?	SHOWN AS
A	2.10	2.4	Mar 3	yes	1
B	2.35	2.8	Mar 9	yes	2 — tie on weighted error...
C	2.35	2.6	Apr 1	yes	2 ...no: C's all-cells is lower, so C is 2 and B is 3
D	3.00	3.1	Feb 1	yes	4
Mine	2.50	2.7	Apr 5	private	~4 — three public rows are better; nobody moves

The tie-break is applied identically on the client (`rankById`) and on the server (`publicRankOf` , which produces the rank badge on a results page), so the two can

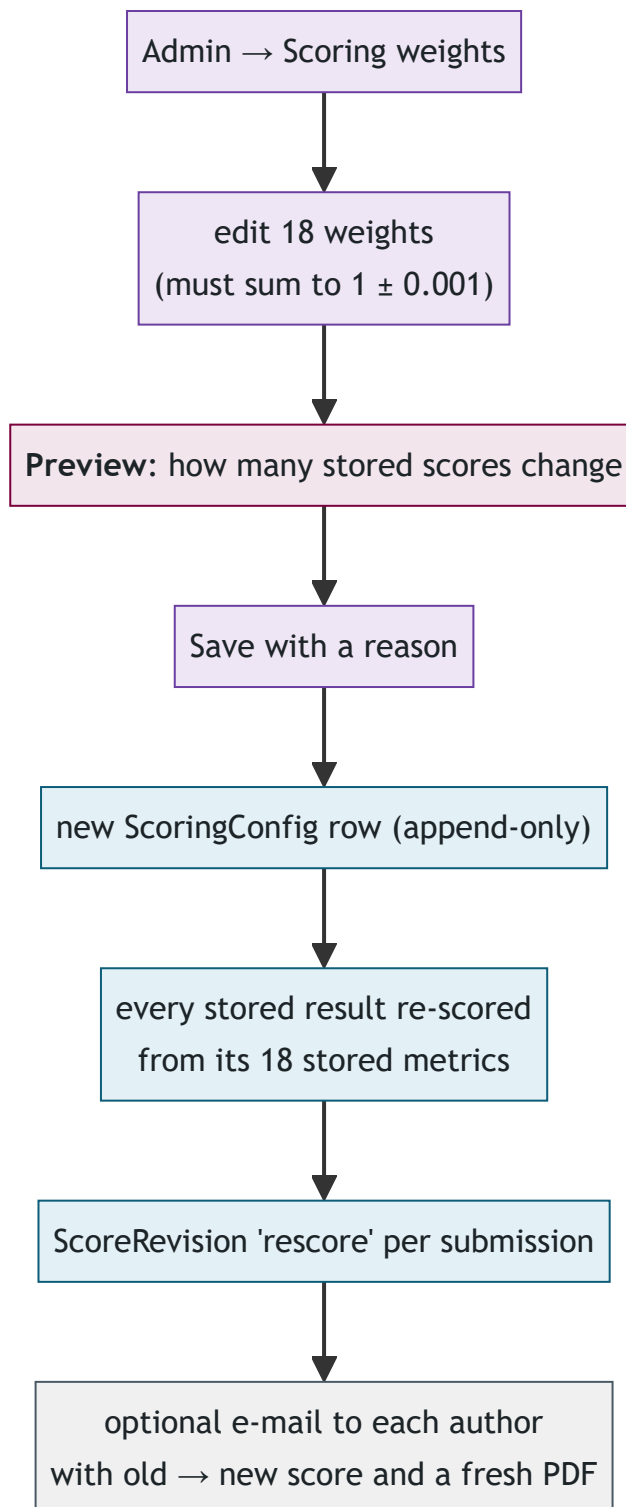
never disagree. Column visibility persists in `localStorage` ; CSV export renders rank as `N` , `~N` (private) or `unranked` (legacy scoring) .



Scoring on the web side, and changing the grading

`weightedError()` in [scoring.ts](#) is $\Sigma(w \cdot v) / \Sigma w$ over the 18 cases defined in [test-cases.ts](#).

Administrators can override the weights:

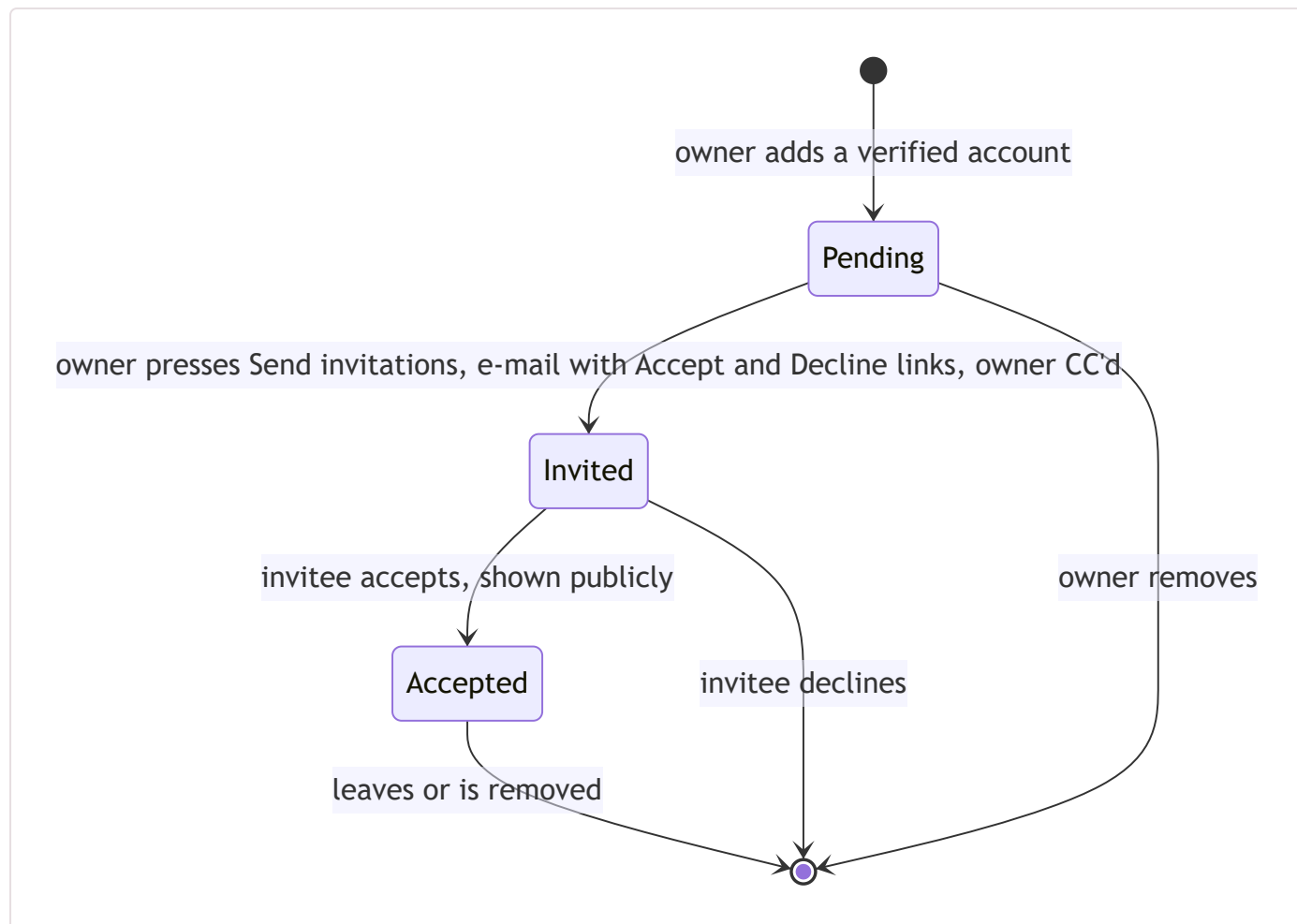


Everything that shows or computes a score — page, PDF, JSON, worker — reads `getActiveWeights()` (30 s cache), so they can never disagree.

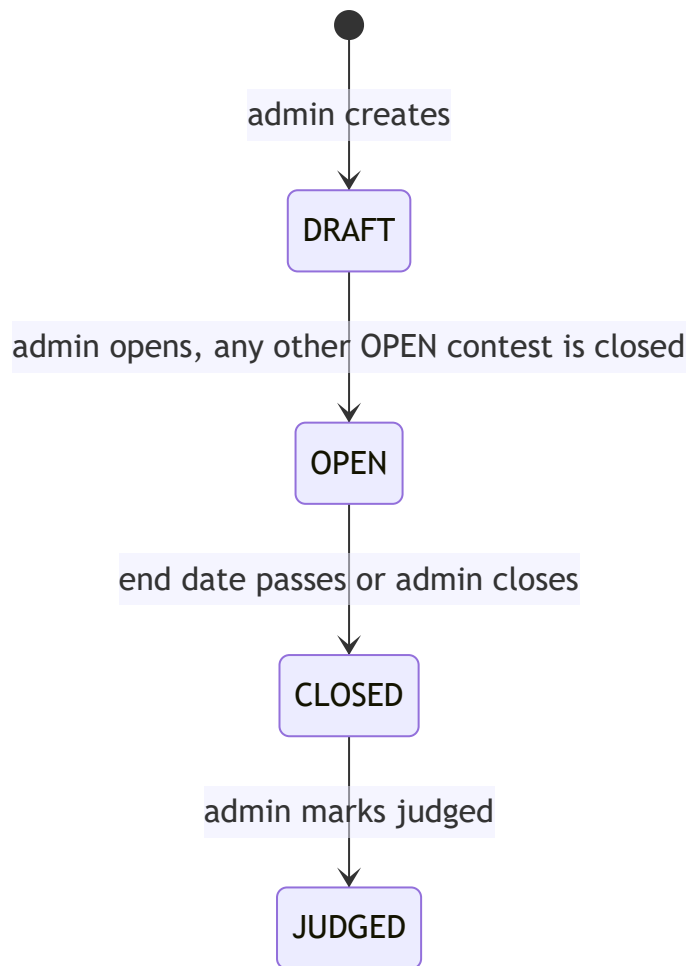
Benchmark versioning is the other kind of change. Results are stamped `socbench-eval-0.1.0/<runtime>`. If the evaluator's *maths* changes, bump `__version__` and `BENCHMARK_VERSION`; older results become "legacy · unranked" and authors are e-

mailed to resubmit. They cannot be re-run automatically because packages are deleted after evaluation on purpose — that is the price of not retaining third-party IP.

Collaborators



Links land on `/collab/[token]` ; decisions are POSTs, never GETs, so scanners cannot answer on someone's behalf; only the invited account can respond even though the owner holds the same link. Results e-mails go to the owner and every *confirmed* collaborator.



While OPEN and within its dates: register (`ContestEntry`), then submit; up to `maxSubmissionsPerUser` non-failed entries; entries must stay public. `/contest/[slug]` freezes its leaderboard by showing only submissions with `submittedAt <= endsAt`. Once closed, resubmission is blocked and name/type are frozen.

Compare

`/compare?ids=a,b,c,d` — at most 4, filtered to rows the viewer may see: a metric table with the best value per row highlighted, the same bar charts as the results page, and overlaid SOC traces when all selected models have them.

🔗 **Files to open, in order:** [submit-form.tsx](#) → [submissions/\[id\]/page.tsx](#) → [status-poller.tsx](#) → [progress.ts](#) → [queries.ts](#) → [leaderboard-table.tsx](#) → [scoring-config.ts](#).



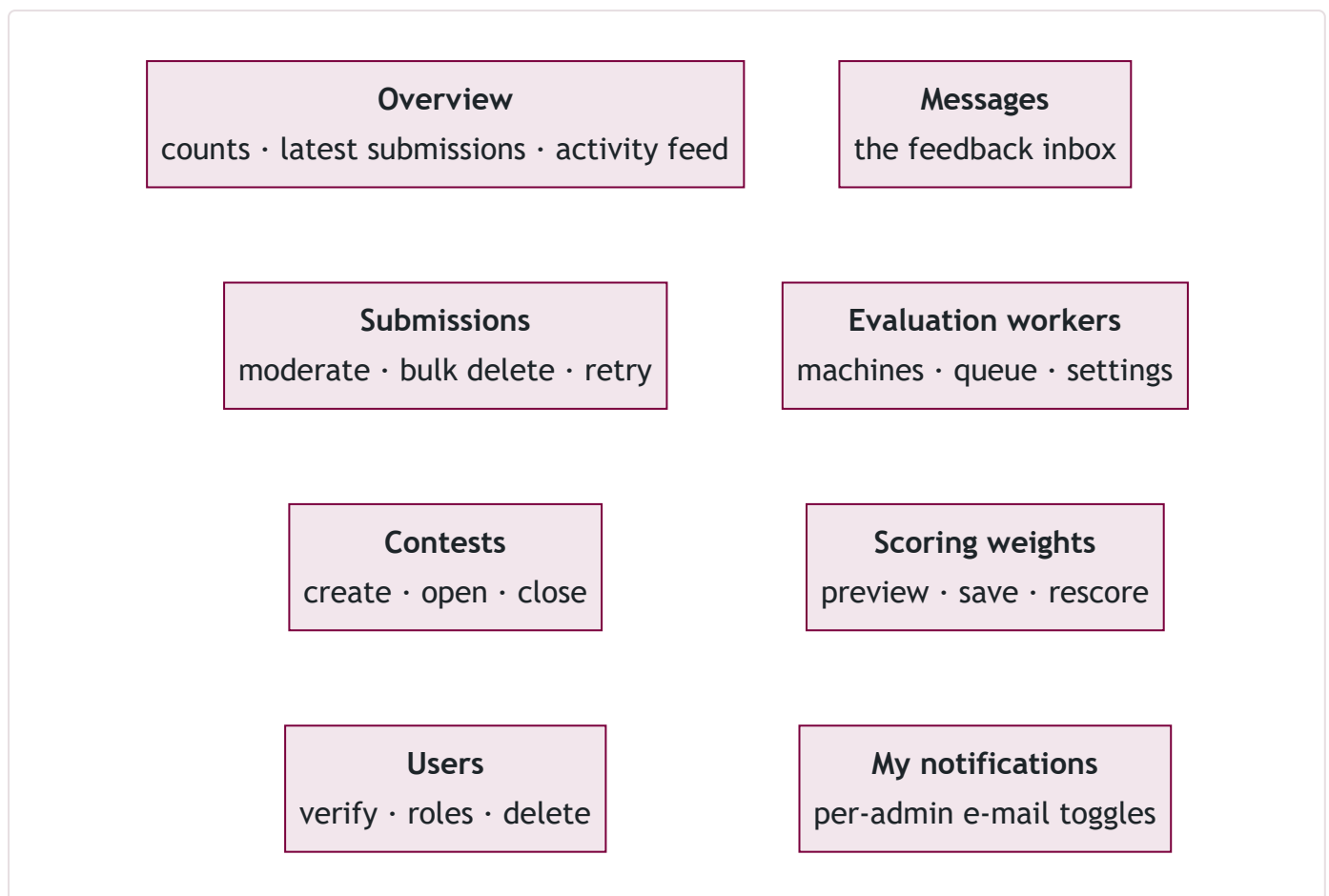
Administration, notifications, reports, monitoring



Plain English. Administrators can moderate submissions, manage users and contests, tune the evaluation limits, change the scoring weights, and watch the worker machines. Every admin action is recorded in an activity feed on the site; e-mail is only a copy of that feed, and each admin chooses which kinds of e-mail they want. A robot checks every ten minutes that a worker is alive and e-mails the admins once if it isn't, and once when it comes back.



The admin area

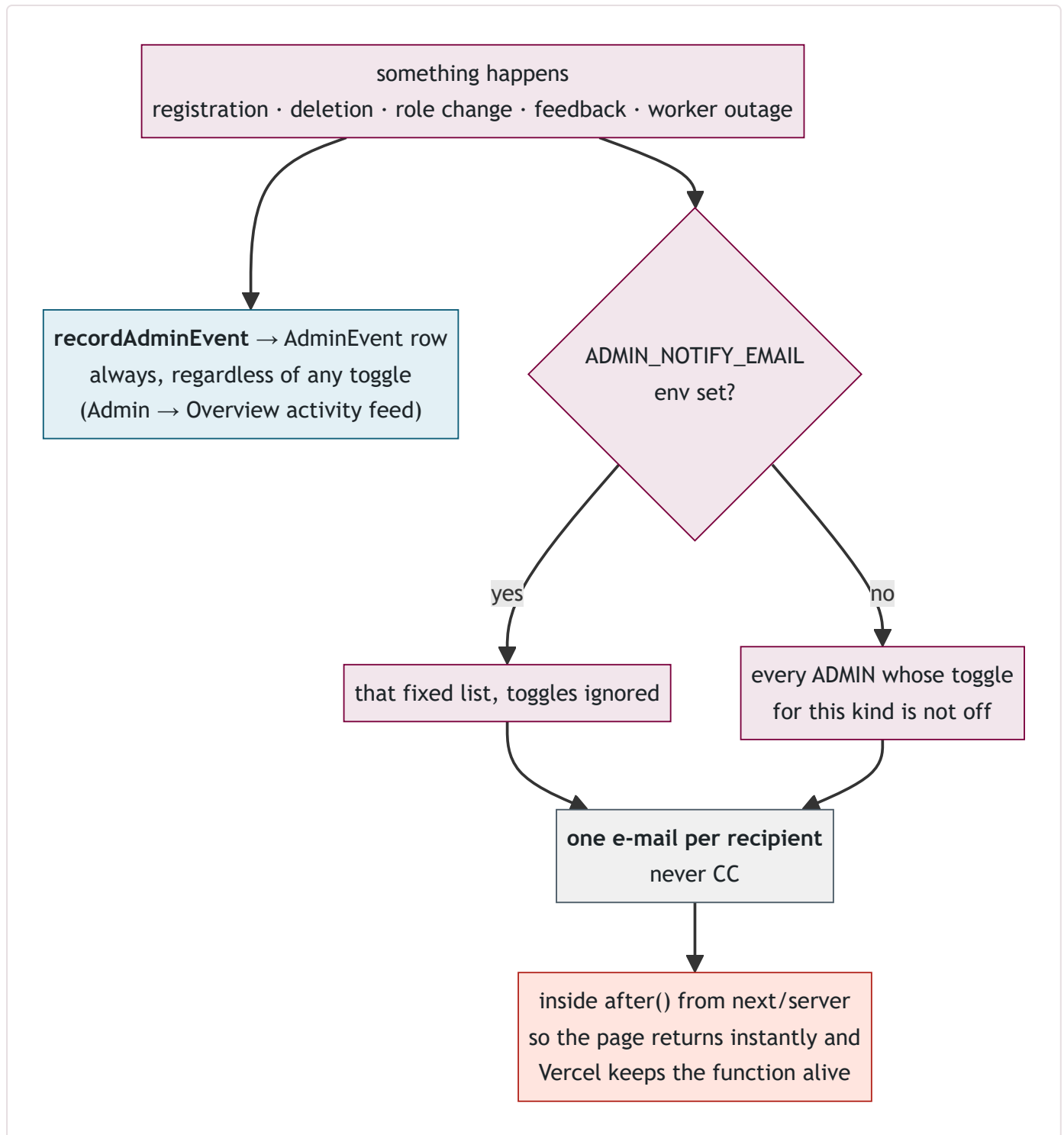


The admin actions and their safety rails

All in [admin/actions.ts](#); every one begins with `requireAdmin()`.

ACTION	RAILS
Moderate a submission (private / public / hide / unhide / delete)	reason ≥ 10 characters; delete blocked while RUNNING; private/public blocked for contest entries ("hide it instead")
Bulk delete	up to 100 ids; RUNNING ones skipped; one activity line; authors optionally e-mailed
Delete a user	no self-delete; cannot delete an admin — revoke admin first (one compromised admin cannot wipe the others); blocked if any of their submissions is RUNNING; storage objects removed, database rows cascade
Change a role	cannot change your own
Worker commands (pause / resume / stop)	written to <code>WorkerHeartbeat.command</code> , picked up at the next heartbeat (≤ 15 s); <i>forget</i> offered only for offline rows
Release a job's lock	behind a confirm — a live worker would then double-evaluate
Evaluation settings	timeout 10–1440 min, dry-run 2–60 min, per-day 1–100
Scoring weights	preview first; must sum to 1; identical-to-active rejected
Contests	only one OPEN at a time

■ Notifications: who gets told, and how



Five kinds each admin toggles for themselves on *My notifications*: feedback, accounts, roles, deletions, workers. **Never CC** because a CC leaks addresses to everyone on the thread and cannot respect per-person toggles. `after()` because Vercel freezes a serverless function the instant it returns its response — an un-awaited promise is silently dropped, which is exactly how a batch of admin notifications went missing before this was found. In code:

```
// the rule, everywhere a server action sends mail it does not wait for
import { after } from "next/server";
after(() => accountDeletedEmail(user, reason, adminName, counts, notifyUser).catch(
```

All templates live in [mail.ts](#): a maroon header, the McMaster and NSERC logo row and acknowledgement in the footer; the results e-mail embeds a confetti GIF as an inline attachment so it shows even where remote images are blocked. With no `SMTP_HOST` configured, mail goes to a throw-away Ethereum inbox and the preview link is printed — how development works.

The PDF report

Built by [report.ts](#) with pdfkit — A4, entirely vector, no browser involved.

Page 1 – title block, author, dates, evaluator version
four stat cards · error-by-test-case bars · error-vs-temperature bars

All test cases – 18-row table (test, name, data, weight, RMSE)
+ score history when there is more than one revision

How this score is computed – eight short paragraphs
+ this submission's weight $\times$ RMSE table summed to the score

Time-domain results – the illustrative cycle traces, two per row

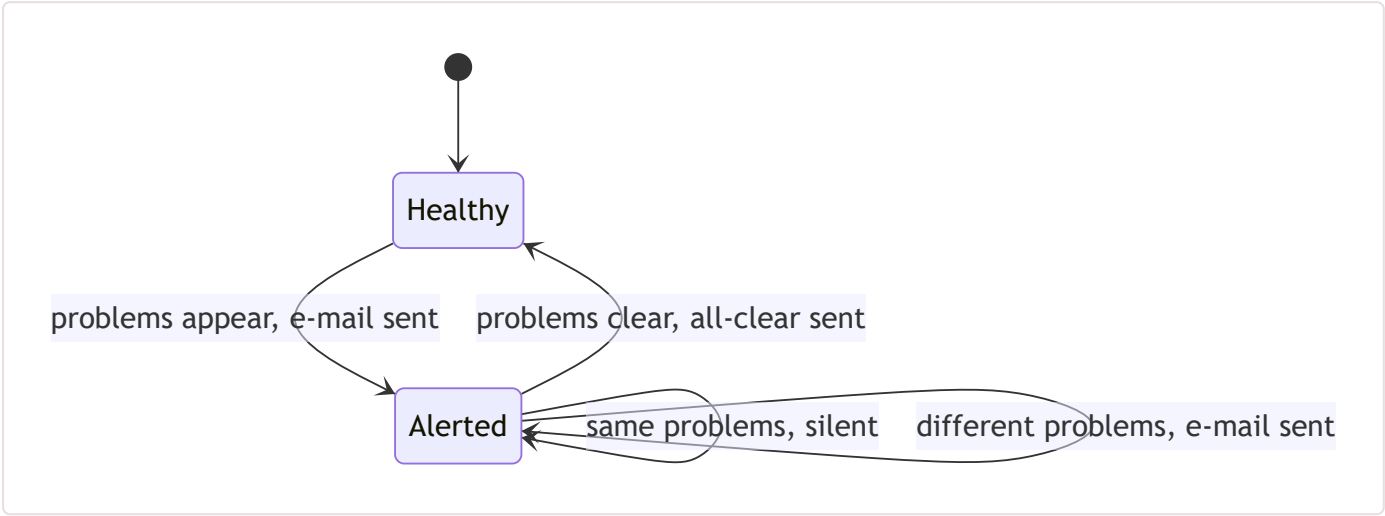
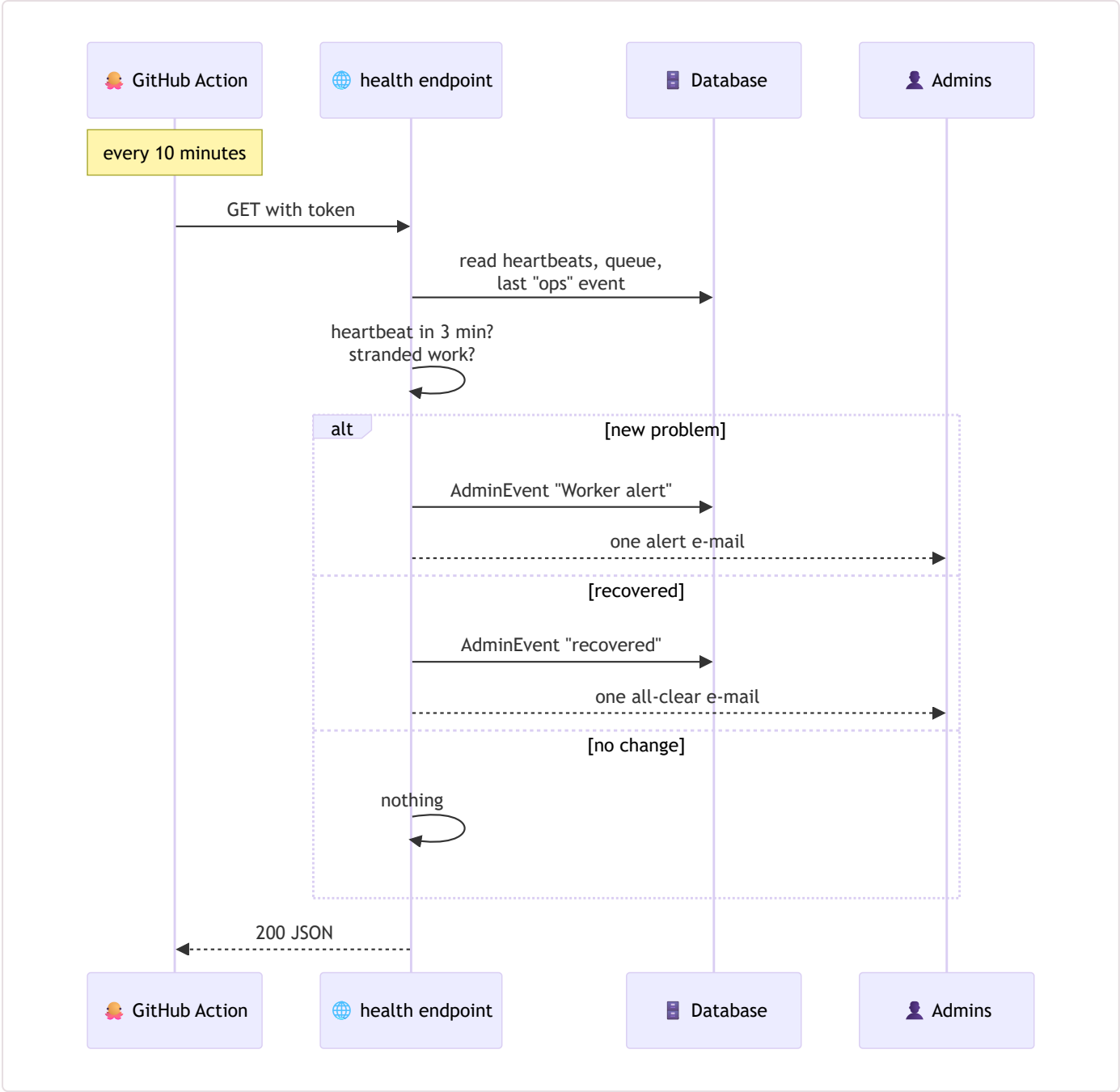
Robustness cases – initial-SOC and sensor-offset traces

Per-cycle errors – every cycle: RMSE, MAE, max, duration

footer on every page: submission URL · citation · NSERC acknowledgement · page i / n

💖 Outage monitoring

Born from the 1 September Arbutus routing outage: the worker was healthy but could not reach the database for ten hours, and nobody was told. The website is the one vantage point that always sees both the database and SMTP, so it does the watching.



Three minutes = twelve missed 15-second heartbeats — long enough that the routine 10-minute update restart never trips it. Counts (how many queued) go into the details, never into the problem text, so a growing queue cannot read as a new outage. A **red** Action run means the *website* was unreachable — GitHub reports that separately. The Workers admin page uses a stricter 60 s window for its "online" badge; the two thresholds are deliberately different (display versus alerting).

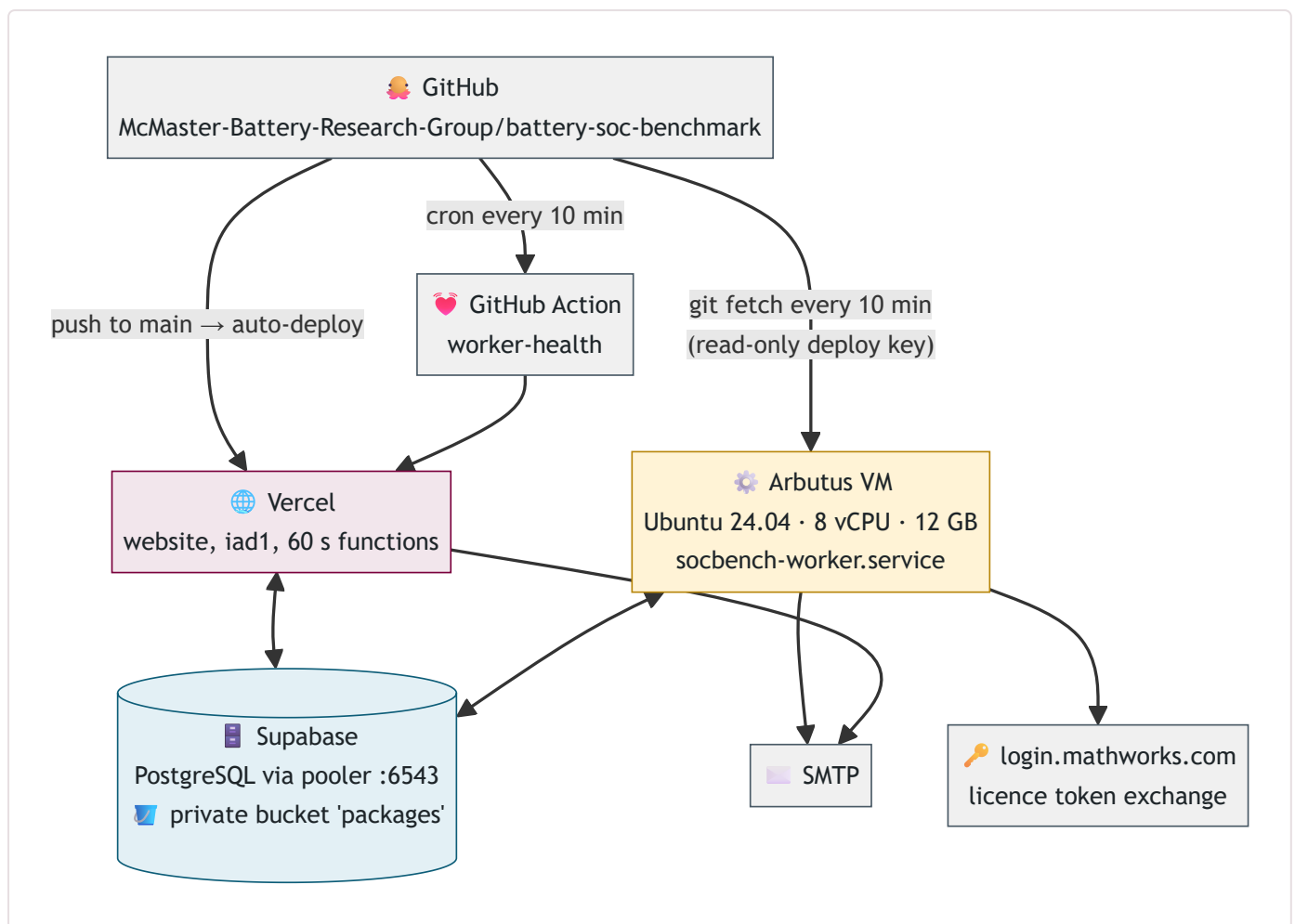
✦ **Files to open, in order:** [admin/actions.ts](#) → [admin-notify.ts](#) → [mail.ts](#) → [report.ts](#) → [api/ops/worker-health/route.ts](#) → [.github/workflows/worker-health.yml](#).



Infrastructure and operations

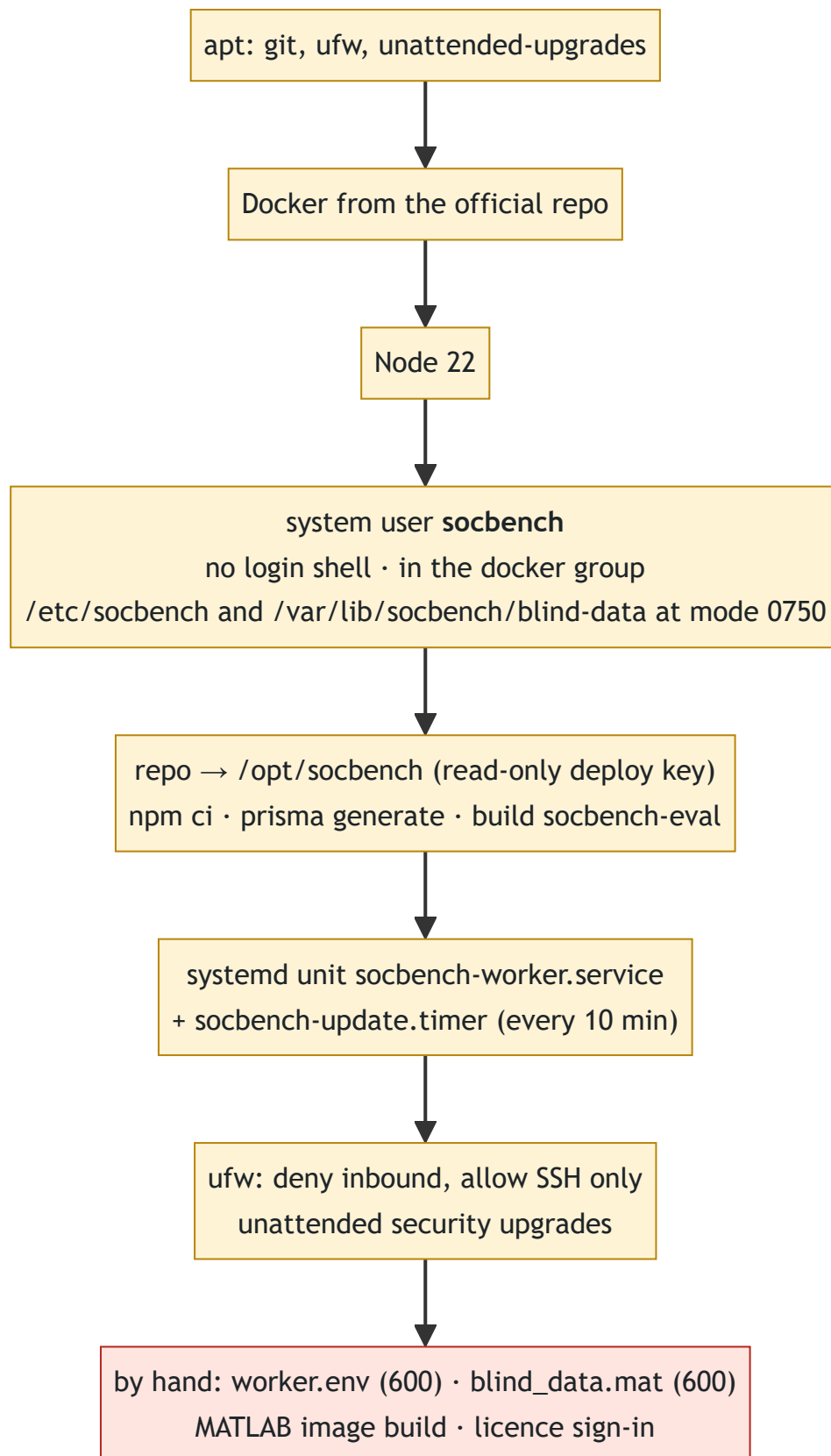
💡 **Plain English.** The website is hosted by Vercel and deploys itself whenever code is pushed. The database and file storage are hosted by Supabase. The worker is a rented Linux computer in the Alliance research cloud that updates itself every ten minutes, restarts only when idle, and runs MATLAB inside a container licensed through Dr. Kollmeyer's MathWorks account. Secrets live in files only the worker's own account can read.

Deployment topology



The VM, as provisioned by one script

[provision-arbutus-worker.sh](#) is idempotent — safe to re-run — and copies **no secrets and no blinded data**; those are placed by hand afterwards.

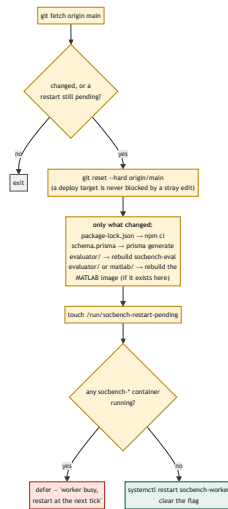


The systemd unit is hardened: `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome=read-only`, write access only to its own state directory and `/tmp`, `ConditionPathExists=/etc/socbench/worker.env` so it cannot start before secrets

land, `Restart=always` , and `KillSignal=SIGINT` so the worker's graceful shutdown hands jobs back to the queue rather than abandoning them.

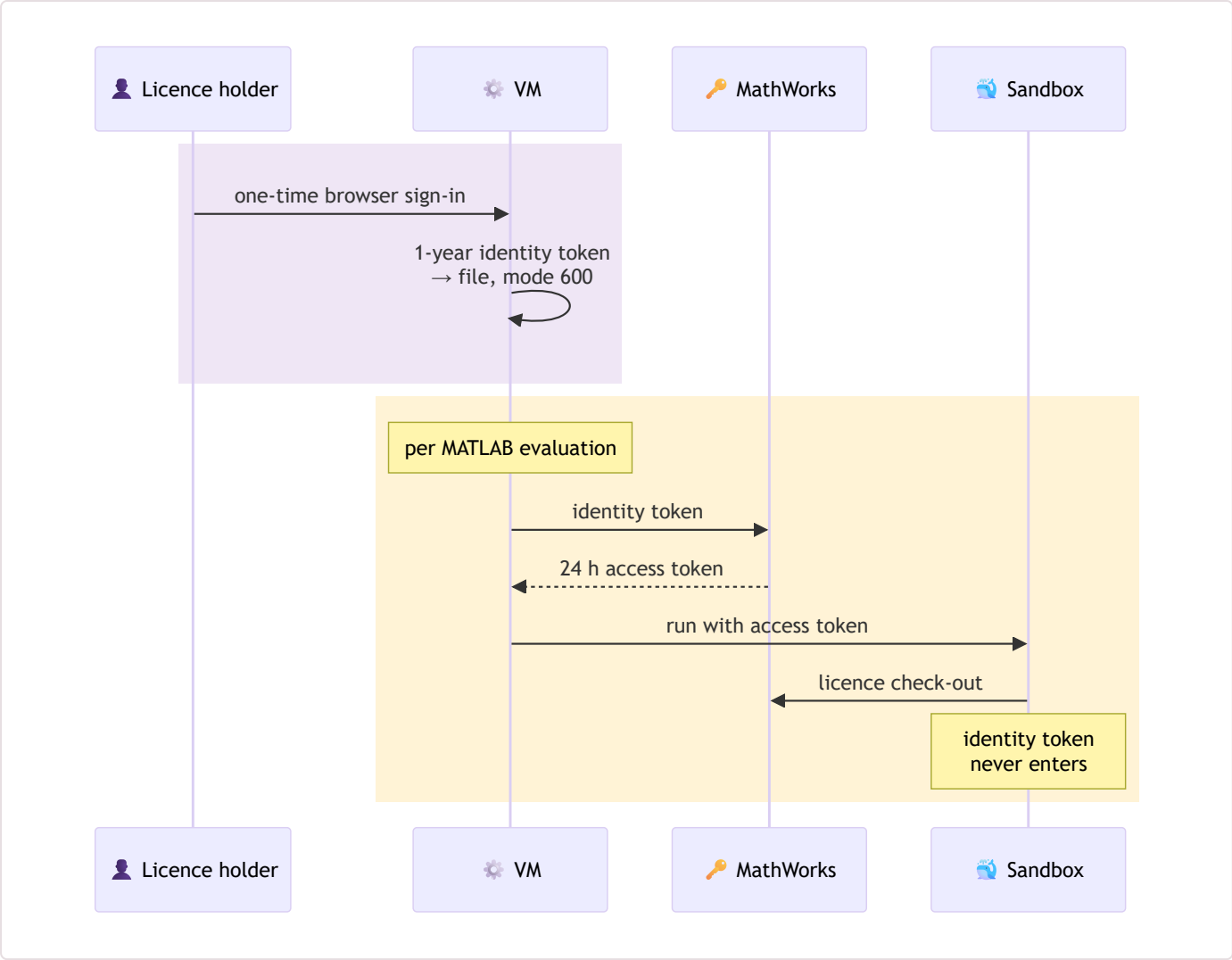
Self-update, without killing an evaluation

[`vm-update.sh`](#) runs from a timer every 10 minutes.



🔑 MATLAB inside the sandbox, and how it is licensed

The image `socbench-eval-matlab` is MathWorks' `matlab-deep-learning:r2026a` (24.5 GB, every toolbox submissions have needed) plus Python and the harness, with the image's `matlab` user **remapped to the** `socbench` **uid** so the blinded data can stay mode 600 and the licence file is owned by the same account. No licence material is baked into the image.



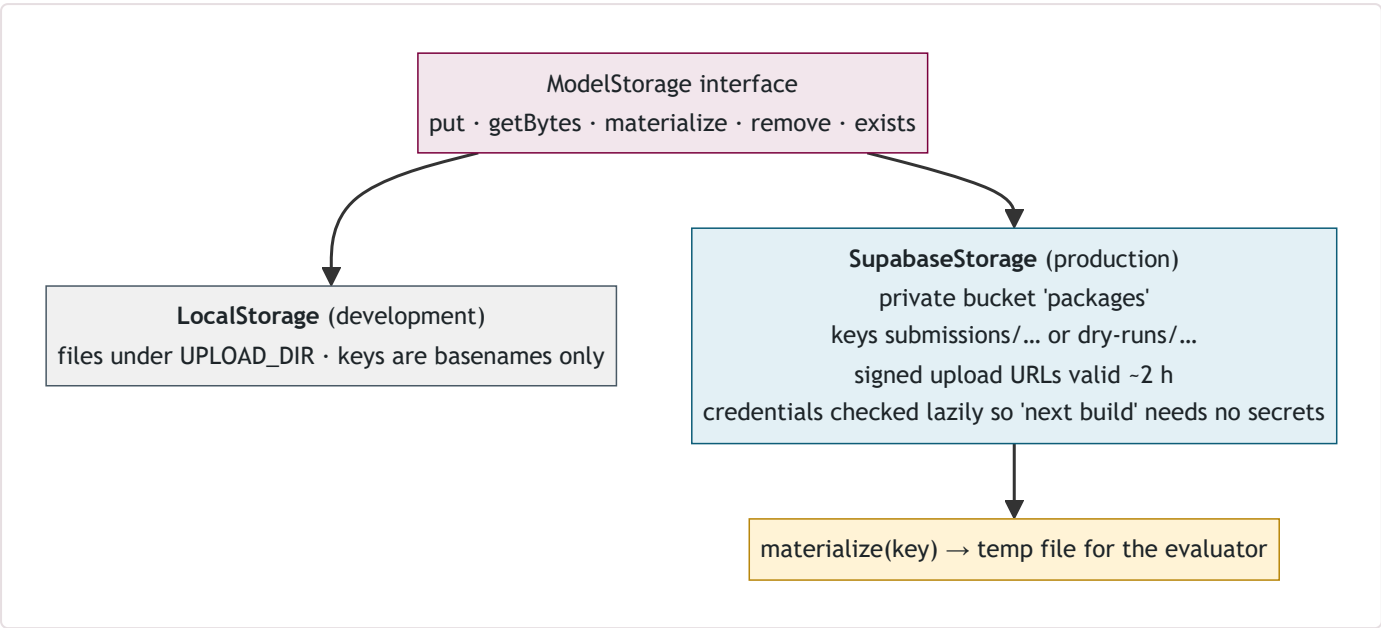
The worst a malicious MATLAB submission can do is read a token that expires within a day and licenses nothing but MATLAB. The identity token's expiry date is shown on the Workers page; renewal is repeating the sign-in.

 The two sandbox images

	SOCBENCH-EVAL (PYTHON)	SOCBENCH-EVAL-MATLAB
Base	python:3.12-slim	mathworks/matlab-deep-learning:r2026a
Adds	numpy, scipy; optional <code>--build-arg TORCH=1</code> (+ ~800 MB CPU PyTorch)	Python 3, numpy, scipy, <code>Run_Model.m</code>
User	<code>evaluator</code> , uid 1000, no login	<code>matlab</code> , remapped to the worker's uid

	SOCBENCH-EVAL (PYTHON)	SOCBENCH-EVAL-MATLAB
Entrypoint	<code>python -m socbench_eval</code>	<code>python3 -m socbench_eval</code> (Python still drives MATLAB)
Build context	<code>evaluator/</code>	repo root (needs <code>matlab/</code>)
Baked in	<code>dryrun_data.mat</code> — dry runs mount nothing	same

Storage abstraction



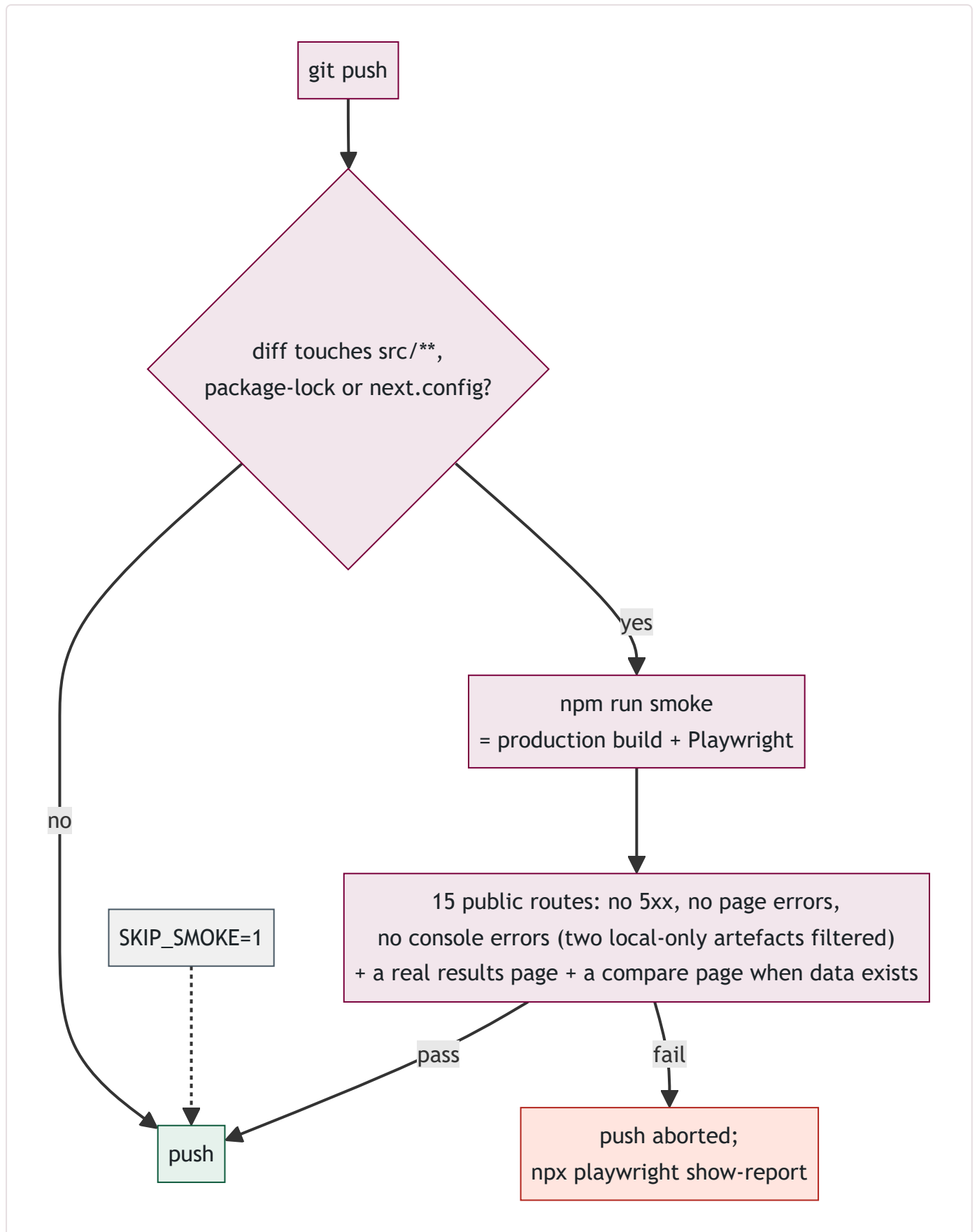
Configuration: the environment variables, by purpose

GROUP	VARIABLES
Database	<code>DATABASE_URL</code> (pooler, 6543), <code>DIRECT_URL</code> (5432, migrations only)
Web / auth	<code>AUTH_SECRET</code> , <code>AUTH_URL</code> , <code>NEXT_PUBLIC_SITE_URL</code> , <code>ADMIN_EMAILS</code> , <code>CRON_SECRET</code>
Mail	<code>SMTP_HOST/PORT/USER/PASS</code> , <code>MAIL_FROM</code> , <code>ADMIN_NOTIFY_EMAIL</code>
Storage	<code>STORAGE</code> (local / supabase), <code>UPLOAD_DIR</code> , <code>MAX_UPLOAD_MB</code> , <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_KEY</code> , <code>SUPABASE_BUCKET</code>

GROUP	VARIABLES
Evaluator	SOCBENCH_BLIND_DATA , SOCBENCH_PYTHON , MATLAB_BIN , WORKER_CONCURRENCY , WORKER_RUNTIMES
Sandbox	EVAL_SANDBOX , EVAL_SANDBOX_IMAGE , EVAL_SANDBOX_MATLAB_IMAGE , EVAL_MATLAB_MHLM_FILE , EVAL_MATLAB_NETWORK , EVAL_CPUS , EVAL_MEMORY , EVAL_PIDS
Calibration	SOCBENCH_CAL_PYTHON , SOCBENCH_CAL_MATLAB

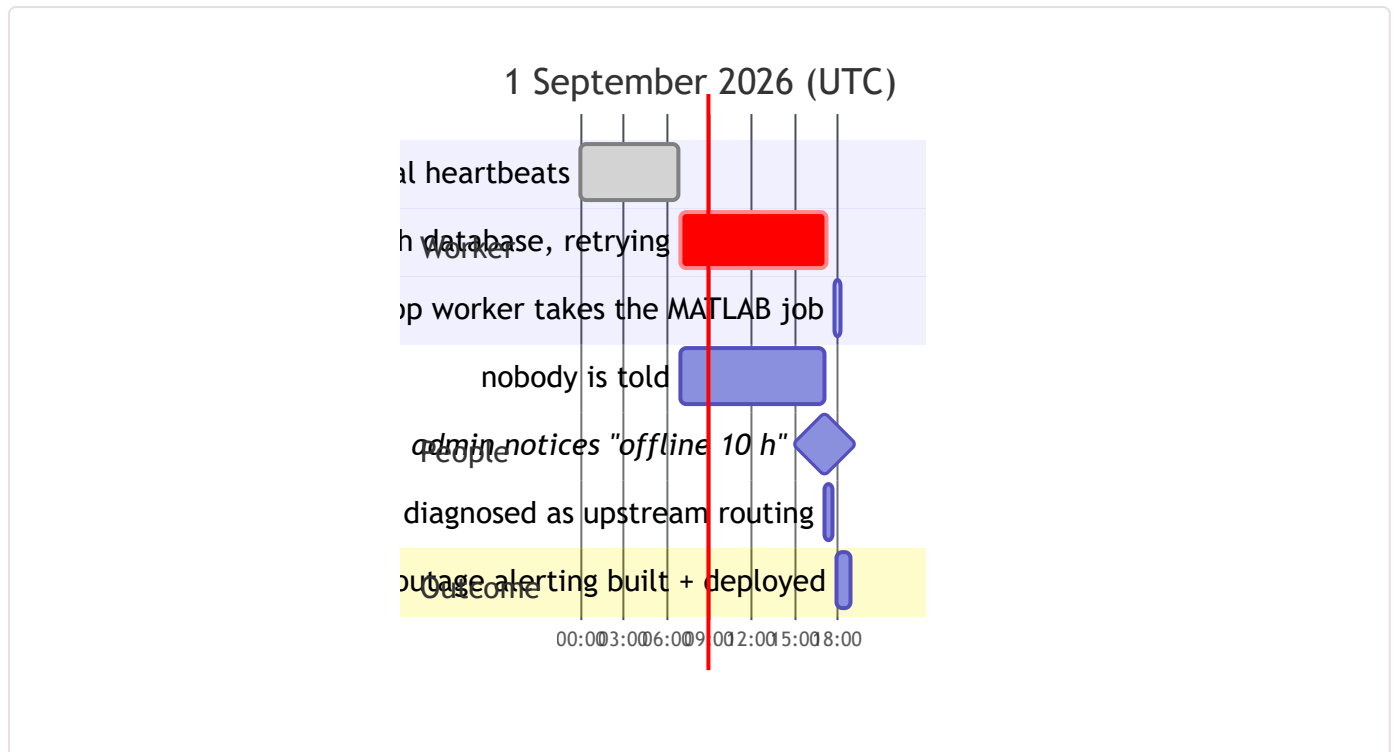
Every line is annotated in [.env.example](#). Production values live only on Vercel (website) and in `/etc/socbench/worker.env` (worker, mode 600).

✓ Tests and the push hook



⚠ The 1 September outage, as it unfolded

The event that shaped the monitoring design. Nothing on the VM was wrong; the route between the research cloud and the part of the internet where the database lives had disappeared overnight.



🚀 Developer bootstrap

```
git clone ...
cp .env.example .env
npm install
npm run setup      # Postgres in Docker on port 5433 + schema + seed accounts
npm run dev        # website
npm run worker     # in a second terminal – needs the blinded data and the sandbox i
```

The seed wipes and creates an admin, six fictional researchers and one open contest with four registrations — and **no submissions or results**, because there is no fake scorer; the leaderboard fills as real evaluations run. Two things that bite on Windows: stop the dev server and the worker before `prisma generate` (a running process locks the generated client), and PowerShell 5.1 has no `&&` — run commands on separate lines.

🔗 **Files to open, in order:** [provision-arbutus-worker.sh](#) → [vm-update.sh](#) → [Dockerfile](#) → [Dockerfile.matlab](#) → [matlab-mhlm-setup.sh](#) → [storage.ts](#) → [playwright.smoke.config.ts](#).

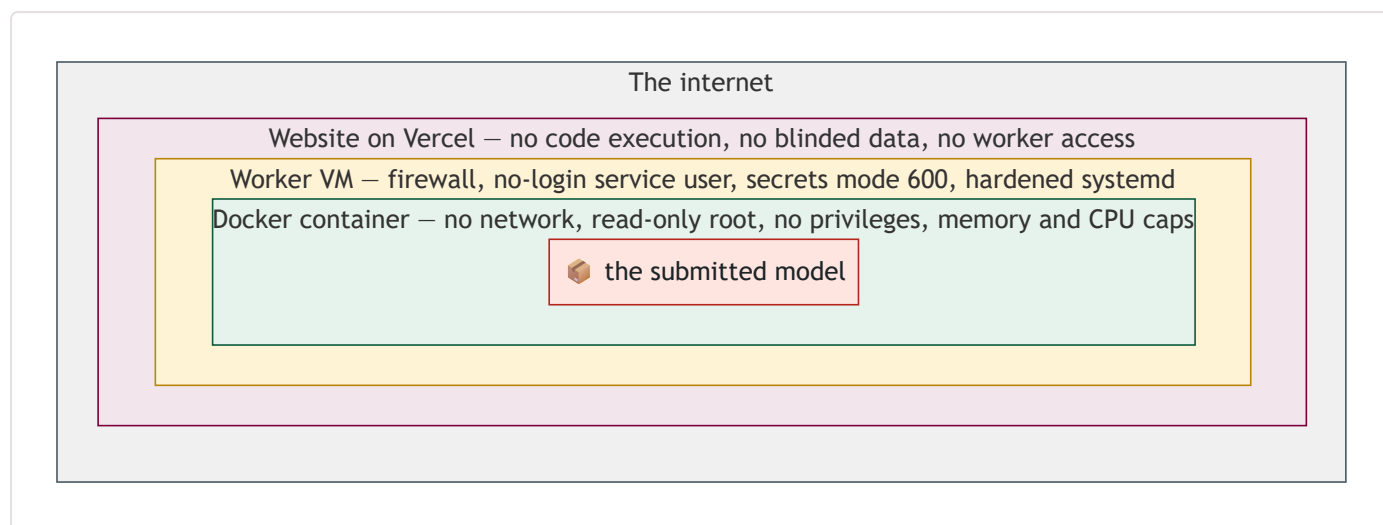


Security: threats and what stops them

💡 **Plain English.** The system runs code written by strangers, on a machine that holds a secret dataset, and it does so in public. This part lists what could go wrong and, for each, the specific thing that prevents it. Most defences are layers: a model would have to break out of several boxes in a row to do any harm.



The onion



 **Where every secret lives**

Vercel environment

DATABASE_URL · AUTH_SECRET
SMTP_PASS · SUPABASE_SERVICE_KEY
OPS_HEALTH_TOKEN · CRON_SECRET

VM — /etc/socbench, mode 600

worker.env — the same database,
storage and SMTP secrets

matlab-mhlm.json —
the 1-year MATLAB identity token

 blind-data/blind_data.mat —
the answer key

GitHub

repo secret OPS_HEALTH_URL

read-only deploy key on the VM

Never

in the repository · in a Docker image
in a job log · in an e-mail

Threats and defences

THREAT	DEFENCE	WHERE
Model exfiltrates the blinded data	<code>--network none</code> ; container destroyed after the run; only <code>/out</code> is writable and only results are read from it	<code>python-evaluator.ts</code>
Model attacks the host	read-only root, <code>--cap-drop ALL</code> , <code>no-new-privileges</code> , runs as the unprivileged worker uid, pid/memory/CPU caps	<code>python-evaluator.ts</code>
Model reads worker secrets	the container never receives the host environment; in host mode only an allow-list of variables is passed	<code>HOST_ENV_ALLOW</code>
Model steals the MATLAB licence	only a 24-hour access token enters the container; the year-long identity token never does	<code>mhlmLicenseEnv()</code>
Zip bomb / path traversal / symlink escape	entry, size and ratio limits; path and attribute checks — on the website <i>and</i> again in the evaluator	<code>package-check.ts</code> , <code>safe_extract</code>
Runaway evaluation	hard timeout inside and outside the container; <code>docker kill</code> + process-group kill	<code>killTree</code>
Queue flooding	3 submissions/day, 5 dry runs/hour, 30 upload URLs/hour	<code>eval-settings</code> , <code>dry-run-quota</code> , <code>rate-limit</code>
Password guessing	bcrypt cost 11; 10 attempts / 15 min per account, 40 per IP	<code>auth.ts</code> , <code>rate-limit.ts</code>
Account enumeration	reset and resend endpoints answer identically whether or not the address exists	<code>(auth)/actions.ts</code>
Link scanners consuming one-time tokens	verification and invitation decisions are POSTs; GETs are side-effect free	<code>verify/actions.ts</code> , <code>collab</code>
Open redirect after login	<code>next</code> must start with <code>/</code>	<code>loginAction</code>
Tampered upload key	keys must match <code>OBJECT_KEY_RE</code>	<code>submit/actions.ts</code>

THREAT	DEFENCE	WHERE
Forged form data	zod on every input; <code>requireUser</code> / <code>requireAdmin</code> inside every action, not just middleware	validation.ts, auth.ts
One compromised admin wipes the rest	admins cannot be deleted until demoted; nobody can change their own role	admin/actions.ts
Secrets in logs	licence values redacted from the docker command before it is logged; <code>CRON_SECRET</code> accepted only in a header, never a URL	python-evaluator.ts, jobs/run
Browser-side attacks	strict CSP, HSTS, <code>frame-ancestors 'none'</code> , nosniff	next.config.ts
Worker VM exposure	ufw default-deny, SSH only from listed addresses, no-login service user, <code>ProtectSystem=strict</code>	provision script
Silent outage	health endpoint + GitHub Action; one alert per transition	worker-health

⚠ What is deliberately *not* defended, and why

- **A model can burn its full time budget doing nothing.** Accepted: the cap is per submission and per day, so the cost is bounded.
- **A model can read the blinded data into memory.** Unavoidable — it has to, to be scored. The defence is that nothing it computes can leave except the SOC estimates we read back.
- **Host-mode MATLAB (the laptop fallback) is not sandboxed.** Documented as "dedicated low-privilege account only"; the production path is always the container.
- **The worker VM has general outbound network access.** On the roadmap: an egress allow-list to the database, SMTP and MathWorks only.

🔗 **Files to open, in order:** [python-evaluator.ts](#) → [package-check.ts](#) → [rate-limit.ts](#) → [next.config.ts](#) → [provision-arbutus-worker.sh](#).

? Questions you will probably get

From researchers and non-developers

"Is my model safe with you?" It is deleted the moment it has been scored. What remains is the scores, the down-sampled traces on your results page, and the full-resolution traces file you can download. Nobody at the lab opens packages; the worker is the only thing that reads them.

"Can other people see my model or my results?" Never your model. Your results are public if you submit publicly; if you tick *private*, only you, your collaborators and administrators can see them, and you get a ghost rank showing where you would stand.

"Why only three submissions a day?" A full evaluation occupies a machine for up to an hour. The dry run is free and unlimited within reason (five an hour) — use it to iterate, and submit when you are confident.

"What does the complexity number mean for me?" Roughly, would this fit on a real battery controller? A 2 is a few arithmetic operations per second; a 9 is a neural network that would need a much bigger processor. It never affects your rank.

"Why is the blinded cell separate?" `m448` is the one cell no data was ever published for. A model that does well on the other three but badly on it has probably memorised the open data rather than learned the physics. That gap is the first thing the plain-English insights point at.

"Why does my score differ from the number in the paper?" The paper reports all-cells RMSE; the leaderboard ranks on weighted error, which up-weights the hard cases (cold, heavy load, robustness). Both are on your results page.

"Why do I have to click a button in the verification e-mail?" Corporate mail scanners open every link before you do; the button stops them activating your account on your behalf.

From developers

"How do you stop someone's model from stealing the blinded data?" It can read it — it has to, to be evaluated — but it cannot send it anywhere: no network in the container, read-only filesystem, no secrets in the environment, container destroyed afterwards. And it cannot reach the website or the database at all; those are on a different machine.

"What if two workers grab the same job?" They cannot. Claiming is an `UPDATE ... WHERE lockedAt = <what I just read>`; exactly one worker sees one row updated. No transactions, no advisory locks — a compare-and-swap on one column.

"What if a worker dies mid-evaluation?" Its lock stops being refreshed (every log line refreshes it). After 30 quiet minutes another worker can reclaim the job; there are two attempts per job. A graceful stop hands the job back immediately without consuming an attempt.

"Why Python for the maths when the lab's tool was MATLAB?" Numerically identical — verified to 0.000 against the historical leaderboard — much faster, no licence needed to *score*, and it runs anywhere. MATLAB is kept only to execute `.m/.p` models: 40 lines that do nothing else.

"Why is there no REST API?" Pages read the database directly in Server Components; forms call Server Actions. Route handlers exist only where a URL is genuinely needed: status polling, downloads, upload URLs, the health check.

"Are Python and MATLAB submissions scored the same?" Accuracy: identical — same data, same scoring code, models called one sample at a time in both. Complexity: calibrated per runtime, informational only, never enters the weighted error.

"What happens if you change the grading?" Weights only: an admin changes them, every stored result is re-scored from its 18 stored values, authors are e-mailed. Anything deeper: bump the benchmark version; old results become "legacy · unranked" and authors resubmit, because packages are deleted after evaluation on purpose.

"How do you know the worker is alive?" It writes a heartbeat row every 15 s with diagnostics and its console tail; the Workers page shows it. A GitHub Action polls a health endpoint every 10 minutes and e-mails admins on the transition to "no heartbeat for 3 min" and back.

"Why does the browser upload straight to the bucket?" Vercel's 4.5 MB body cap. Signed URL, ~2 h validity; the server re-downloads and validates before creating anything.

"Why run a validation cycle first?" So a broken model fails in seconds with the real error instead of after 45 minutes.

"Where is the per-sample loop?" `_iterate_py` in `runner.py` for Python; the `for i = 2:size(X,1)` loop in `Run_Model.m` for MATLAB. Both carry `z` forward and call once per sample — the BMS contract.

"Why is the queue a database table and not Redis?" One fewer service to run and pay for, and the compare-and-swap gives the same guarantee. Throughput is a few jobs an

hour; a queue service would be solving a problem we do not have.

"Why serverless for the website but a VM for the worker?" The website is bursty and stateless — serverless is free and scales itself. The worker needs the blinded data on disk, Docker, MATLAB and multi-hour runs — none of which serverless allows.

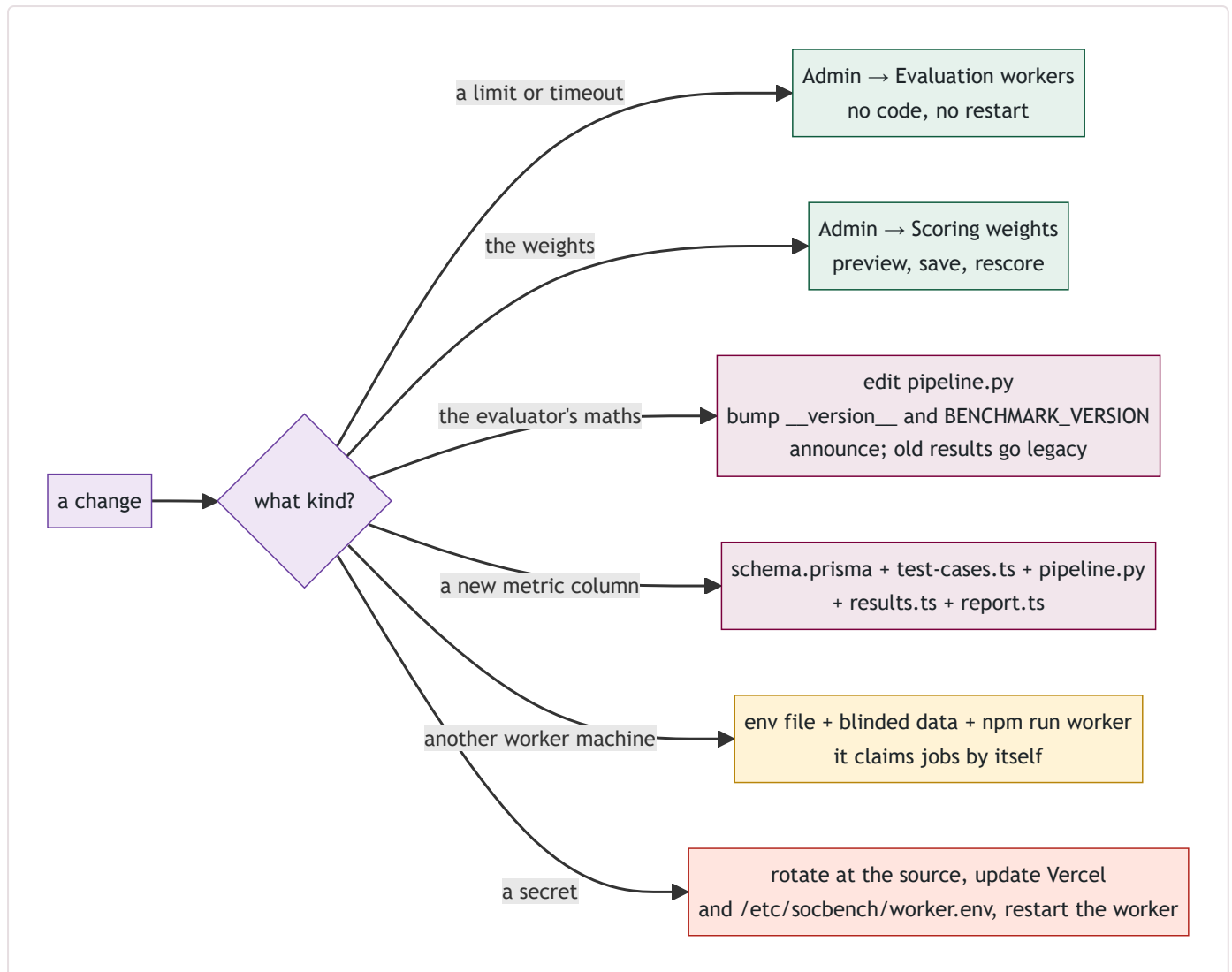
"Why `after()` everywhere in server actions?" Vercel freezes the function the instant it returns; an un-awaited promise is dropped. `after()` keeps the function alive until the background work finishes. This is written down because it silently lost admin e-mails once.

Rough edges to know about before someone finds them: `.env.example` defines `ADMIN_NOTIFY_EMAIL` and `MAIL_FROM` twice and omits `WORKER_RUNTIMES`; the Workers page (60 s) and the health endpoint (3 min) use different "online" windows on purpose; complexity bins for loop-heavy MATLAB code can drift ± 1 between hosts; the leaderboard tie-break was only made explicit on 11 September (all-cells RMSE, then earlier submission).



How to change things — recipes

Plain English. The most common changes, each as a short checklist. If a change is not here, the *Files to open* lists at the end of each part say where to look.



Change a limit (timeout, dry-run timeout, submissions per day): Admin → Evaluation workers → Evaluation settings. Takes effect within 15 s on the website and the worker. No deploy.

Change the scoring weights: Admin → Scoring weights → edit → Preview (see how many scores move) → Save with a reason → optionally notify authors. Append-only; the previous weights remain in `ScoringConfig` history.

Change the evaluator's maths (a metric definition, padding, a sweep): edit `pipeline.py`; re-run the four reference packages and confirm the scores you *expect* to change changed and nothing else did; bump `__version__` in `socbench_eval/__init__.py` and

`BENCHMARK_VERSION` in [benchmark-version.ts](#); push (the VM rebuilds the images within 10 minutes); run `npx tsx scripts/announce-benchmark-version.ts --note "..."` `--send` so affected authors are told to resubmit.

Add a metric column: `EvaluationResult` in [schema.prisma](#) → `npm run db:push` → the entry in [test-cases.ts](#) (key, label, weight, group) → compute it in `score()` → it flows through `METRIC_KEYS` to `results.ts`, the scorecard, the PDF and the CSV automatically. Re-check the weights still sum to 1.

Add a model type: the `ModelType` enum in [schema.prisma](#) → `db:push` → `MODEL_TYPES` in [validation.ts](#).

Add a worker machine: a machine with Docker (or MATLAB), the blinded data at mode 600, and a copy of `worker.env` → `WORKER_RUNTIMES` set to what it can run → `npm run worker:prod`. It registers a heartbeat and starts claiming jobs; nothing else changes. On Linux, use the provisioning script.

Rotate a secret: rotate at the source (Supabase, Gmail/Resend, MathWorks) → update the Vercel environment and redeploy → update `/etc/socbench/worker.env` on the VM → `systemctl restart socbench-worker`. The MATLAB identity token is the exception: repeat the browser sign-in and `matlab-mhlm-setup.sh`.

Add an e-mail: a function in [mail.ts](#) using `layout()` and `sendMail()` → if it is an admin notification, route recipients through `adminNotifyTargets(kind)` and record it with `recordAdminEvent` → call it inside `after()` from any server action.

Add an admin page: a folder under `src/app/(admin)/admin/` → the layout already enforces `requireAdmin()` → add the tab to `admin-nav.tsx` → every action in `actions.ts` starts with `await requireAdmin()`.

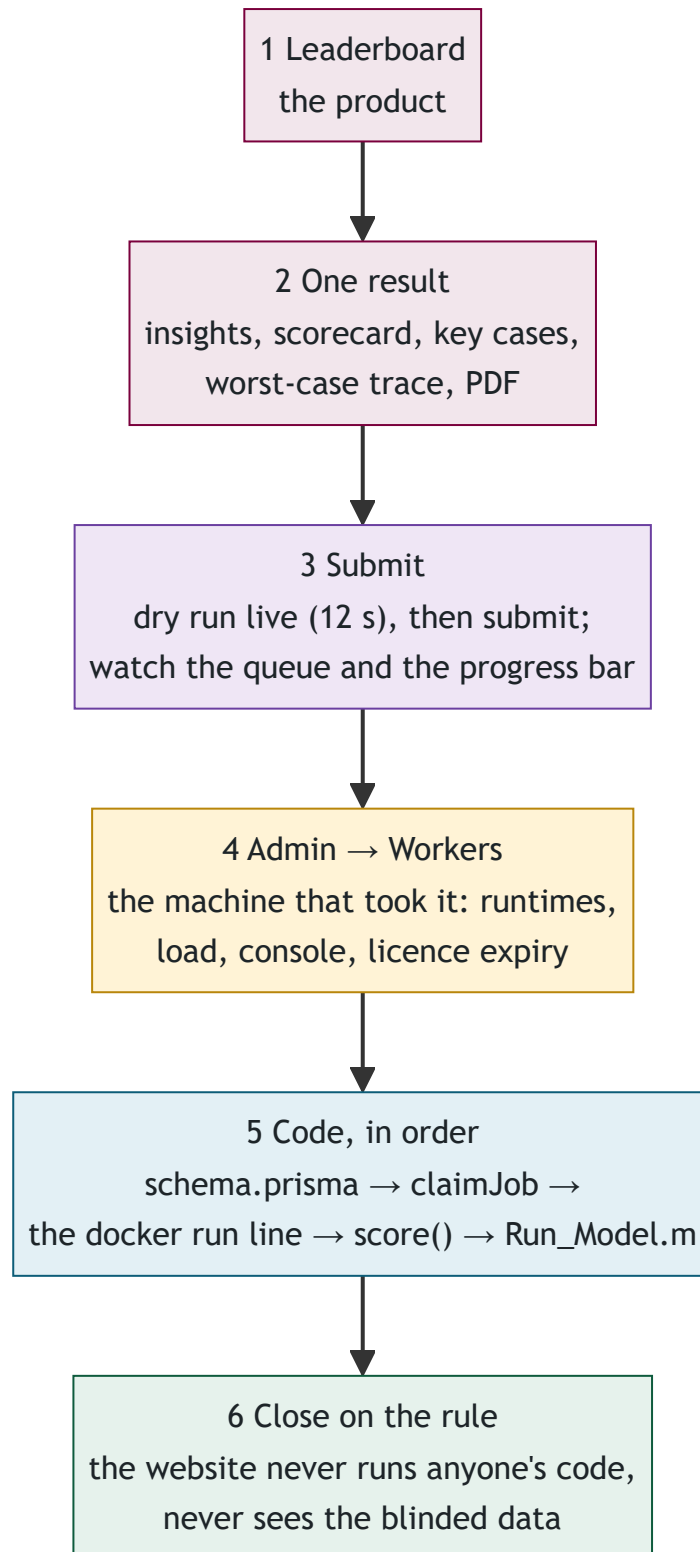
Change who is an admin: Admin → Users → Make admin / Revoke admin. Or add the address to `ADMIN_EMAILS` on Vercel — effective on their next request.

Renew the MATLAB licence (yearly): the Workers page shows the expiry; repeat the browser sign-in through the SSH tunnel and run `scripts/matlab-mhlm-setup.sh` on the VM.



A demo order that tells the story

About ten minutes.



1. **Leaderboard** — rank, weighted error, complexity, the column groups.

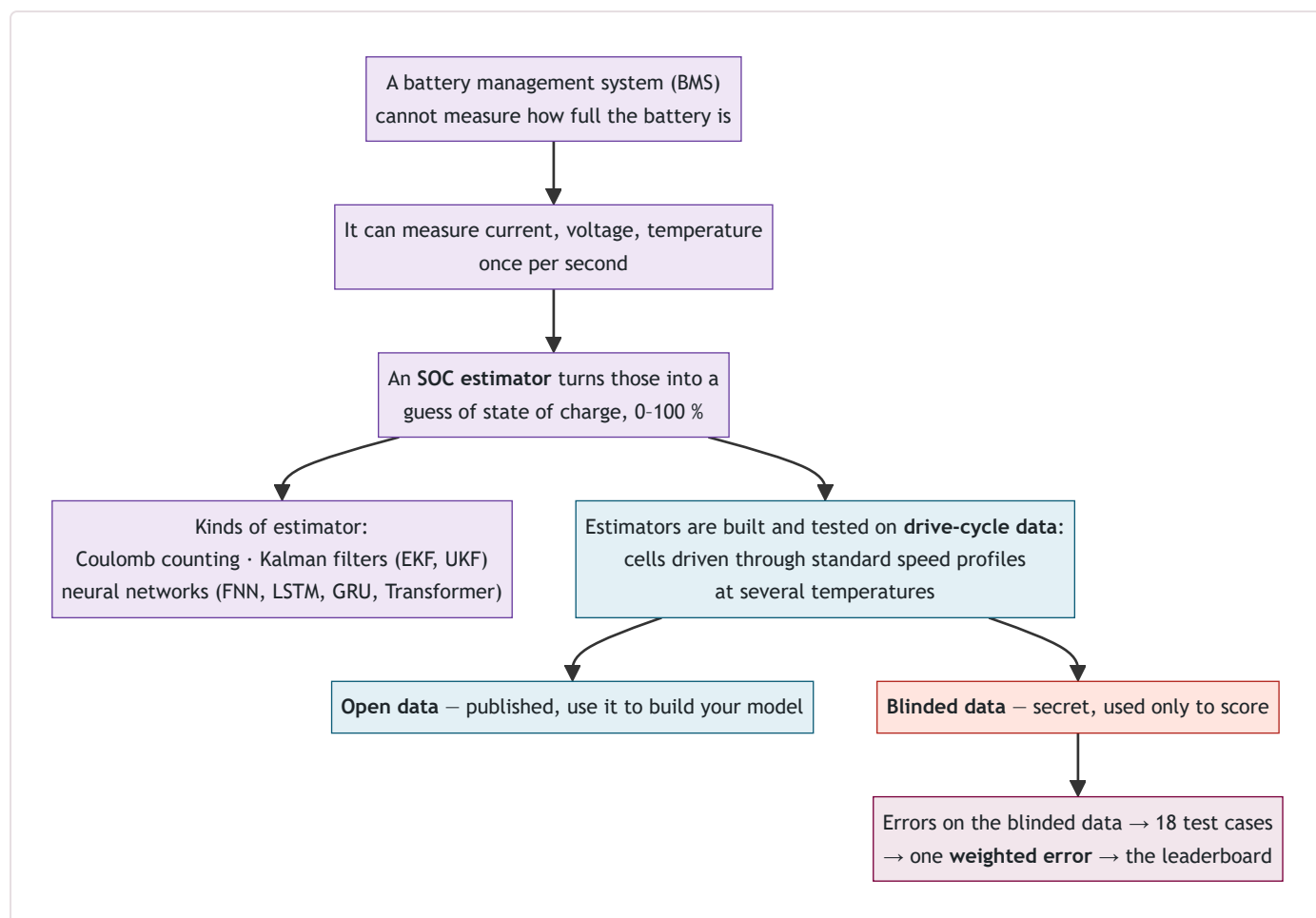
2. **Open one result** — the plain-English insights, the 18-row scorecard summing to the score, key cases (blinded vs non-blinded cell, cold temperature, wrong initial SOC, sensor offset), the worst-case trace, the PDF.
 3. `/submit` **with a reference package** — run a dry run live, watch the console stream, then submit. Show the queue position and the live progress bar.
 4. **Admin** → **Workers** — the machine that just picked it up. Mention the outage alerting.
 5. **Code, in this order:** `schema.prisma` (the tables you just saw) → `run-job.ts` `claimJob` (the compare-and-swap) → `python-evaluator.ts` (the `docker run` line) → `pipeline.py` `score()` (the matrix R and the weights) → `Run_Model.m` (all 40 lines of MATLAB).
 6. **Close on the design rule.**
-



Glossary

Plain English. Two vocabularies meet in this project: battery science and web software. Nobody is expected to know both. This part defines every word the rest of the document uses, in one line each, and shows how the concepts hang together.

How the battery concepts fit together



Battery and benchmark terms

TERM	MEANING
SOC — state of charge	How full a battery is, 0–100 %. The fuel gauge. It cannot be measured directly; it must be <i>estimated</i> from what can be measured: current, voltage and temperature. Every model in the benchmark is an SOC estimator.

TERM	MEANING
BMS — battery management system	The electronics in an electric vehicle or phone that watch the battery. The SOC estimator runs inside it, one measurement at a time. Our evaluator calls models the same way, which is why a model cannot "look ahead".
Cell	One physical battery. The dataset has four Tesla 2170 cells, named after the vehicle-mass profile each was driven with: <code>m80</code> , <code>m448</code> , <code>m448N</code> , <code>m1000</code> (80 kg, 448 kg, a second 448 kg variant, 1000 kg — heavier means larger currents). <code>m448</code> is the <i>fully blinded</i> cell: no data about it was ever published, so it is the purest test of whether a model generalises.
Drive cycle	A standard speed-vs-time profile a car is tested against, converted into the current a cell would see. Standard ones: UDDS (urban stop-and-go), HWFET (steady highway), LA92 (aggressive urban), US06 (aggressive highway). Custom ones the model has never seen in the open data: HWCUST , HWGRADE (highway with road grade — long high-current stretches and regeneration).
Temperature	Each cycle was run at -20, -10, 0, 10, 25 and 40 °C. Cold is hard: internal resistance rises and the voltage response becomes strongly non-linear.
Open data / blinded data	<i>Open</i> = published on Borealis; researchers use it to build and train models. <i>Blinded</i> = kept secret and used only to score. The file <code>blind_data.mat</code> is the answer key; it exists only on the evaluation machine.
Coulomb counting	The simplest estimator: integrate the current over time. Cheap and it drifts badly. Used as the <i>reference point</i> for the complexity scale.
EKF / UKF	Extended / Unscented Kalman Filter — classical estimators that fuse a physics model of the cell with the measurements.
FNN / LSTM / GRU / Transformer	Neural-network estimators: a plain feed-forward network and three kinds of sequence model that keep memory of past samples.
RMSE / MAE / max error	Root-mean-square error, mean absolute error, and the worst single-sample error between estimated and true SOC, in % SOC. RMSE is the per-cycle headline; max error tells you the worst moment.
Test case	One of the 18 published scoring categories (blinded cell, charging, each temperature, sensor offset, ...). Each has a weight.
Weighted error	The single leaderboard score: the sum of weight × test-case RMSE. Lower is better.

TERM	MEANING
Robustness sweep	Deliberately breaking the model's assumptions: starting it at the wrong initial SOC (test 10) or feeding it current with a constant sensor offset (test 11). The cases that separate good estimators from lucky ones.
Padding	One hour of the first measurement repeated before every cycle, so models with internal memory settle before scoring starts.
Complexity	A 1–10 bin of how much computation the model needs per sample, relative to a Coulomb counter. Informational; never part of the score.
Dry run	A free pre-submission test on <i>open</i> data: does the package run, what error does it make, how expensive is it. No leaderboard entry.
Package	The <code>.zip</code> a researcher uploads: <code>Model.py</code> (Python) or <code>Model.m</code> / <code>Model.p</code> (MATLAB) at the top level plus any parameter files.
Legacy result	A score produced by an older version of the evaluator's maths. Still shown, but unranked; the author is asked to resubmit.



Software terms

TERM	MEANING
Repository (repo)	The folder of source code, tracked by git so every change is recorded and reversible. Hosted on GitHub .
Front end / back end	Front end = what runs in your browser. Back end = what runs on a server. In this project the line runs <i>inside</i> Next.js.
Server	A computer that answers requests. Serverless = the hosting provider starts a tiny short-lived server <i>per request</i> and freezes it the instant it answers. Cheap, scales itself, and has quirks this document points out.
Container / Docker / image	A container is a lightweight isolated box a program runs in, with its own filesystem and no view of the host. Docker runs them. An image is the frozen template a container starts from.
Sandbox	A container configured to be as harmless as possible: no network, read-only files, no privileges. Where submitted models run.
Database / table / row	PostgreSQL stores everything as tables of rows, like spreadsheets with strict columns.

TERM	MEANING
ORM (Prisma)	A library that lets TypeScript talk to the database with typed function calls instead of hand-written SQL, and keeps one schema file as the description of every table.
Migration / db push	Applying a schema change to a real database.
API / endpoint / route	A URL a <i>program</i> calls to get or send data, as opposed to a page a human reads. REST is one common style of API; this project mostly does not use one.
JSON	The text format programs use to exchange structured data: <code>{"tmse": 3.2}</code> .
Environment variable / .env	Configuration and secrets handed to a program from outside its code, so the same code runs in development and production with different settings.
Secret / token / key	Any string that grants access: a database password, an API key, a licence token. Never in the repo; always in environment variables or files with restricted permissions.
Session / JWT / cookie	After login the browser holds a signed cookie — a JSON Web Token — proving who you are; the server verifies the signature on every request instead of looking you up.
Hash (bcrypt)	A one-way scramble of a password. We store the scramble, never the password; bcrypt is deliberately slow so guessing is expensive.
Rate limit	"At most N attempts per time window" — the defence against brute force and abuse.
Queue / job / worker	A queue is a list of work waiting to be done; a job is one item; a worker is the program that takes jobs off the queue and does them.
Lock / atomic / compare-and-swap	Ways to make sure two workers never take the same job. <i>Atomic</i> = happens in one indivisible step. Compare-and-swap = "update this row only if it still looks the way I last saw it".
Heartbeat	A periodic "I am alive" signal a worker writes so everyone else can tell if it died.
Polling	Asking "anything new?" every few seconds, rather than being pushed a notification.

TERM	MEANING
Cron / scheduled job	Something that runs on a timer.
CI — continuous integration	Automated checks that run on the code-hosting platform (GitHub Actions here).
SMTP	The protocol for sending e-mail.
Object storage / bucket / signed URL	A cloud service for storing <i>files</i> (not tables). A bucket is a named store; a signed URL is a temporary pre-authorised link that lets a browser upload directly without our server in the middle.
VM — virtual machine	A rented computer in the cloud. Ours is on Arbutus , the Digital Research Alliance of Canada's cloud at the University of Victoria, built on OpenStack .
systemd / service	Linux's way of running a program as a background service that starts on boot and restarts on crash.
SSH	Secure remote terminal access to a machine.
uid / gid / mode 600	Linux file ownership (user id, group id) and permissions. <code>600</code> = only the owner can read or write.
tmpfs	A folder that lives in RAM and vanishes when the container stops.
TypeScript / Node.js / npm / tsx	TypeScript is JavaScript with types. Node.js runs JavaScript outside a browser. npm installs libraries and runs scripts. tsx runs TypeScript files directly with no build step.
Zod schema	A declaration of what a form input must look like, checked at runtime, reused in the browser and on the server.

12 Numbers worth remembering

4 cells × 6 temperatures × 6 cycles	144 blinded test cycles, plus charging cycles
Robustness runs	9 initial-SOC + 18 sensor-offset
Scoring	18 test cases; weights sum to exactly 1

Daily cap	3 submissions per person per rolling 24 h (dry runs free, 5 per hour)
Evaluation limit	360 min ; dry run 10 min
Worker rhythm	polls every 2 s , heartbeat every 15 s
Job safety	lock goes stale after 30 min of silence; 2 attempts per job
Website "online" window	60 s ; outage alert after 3 min
Upload cap	50 MB ; browser uploads straight to the bucket because Vercel accepts only 4.5 MB